



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA**

TRABAJO FIN DE GRADO

**Desarrollo de un generador configurable
de modelos de características UVL
integrado en UVLHub**

Realizado por

Jiménez Ortega, Antonio

Para la obtención del título de

Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por

Benavides Cuevas, David

Romero Organvández, David

En el departamento de

Departamento de Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

DECLARACIÓN DE AUTORÍA

D./Dña. **Jiménez Ortega, Antonio**, autor/a del Trabajo Fin de Grado titulado «Desarrollo de un generador configurable de modelos de características UVL integrado en UVLHub», presentado en Sevilla, octubre 2026 para la obtención del título de Grado en Ingeniería Informática - Ingeniería del Software, declara bajo su responsabilidad que:

- El trabajo es original y ha sido realizado por el/la autor/a en colaboración con su/s tutor/es académicos.
- Se han citado adecuadamente todas las fuentes consultadas, tanto bibliográficas como digitales.
- No se ha plagiado contenido de otros autores ni se han utilizado materiales sin la correspondiente autorización.
- Autoriza a la Universidad a utilizar este trabajo con fines académicos y de difusión, citando siempre su autoría.

Sevilla, a 3 de agosto de 2026

Fdo.: Jiménez Ortega, Antonio

AGRADECIMIENTOS

Quiero expresar mi profundo agradecimiento a mis tutores, David Benavides y David Romero, por la orientación, supervisión y apoyo proporcionados durante el desarrollo de este Trabajo de Fin de Grado. Sus indicaciones y experiencia han sido fundamentales para comprender el contexto del proyecto, tomar las decisiones funcionales y técnicas adecuadas, y afrontar los distintos retos que han ido apareciendo durante su realización.

También quiero agradecer al grupo de DiversoLab y a todas las personas que han contribuido al desarrollo del entorno tecnológico sobre el que se ha construido este trabajo por facilitar un marco en el que aplicar los conocimientos adquiridos durante mis años en el grado.

Finalmente, dedico este trabajo a mi familia y a todo mi entorno cercano. Su acompañamiento y confianza han sido indudablemente esenciales para completar este proyecto y afrontar el esfuerzo que supone un trabajo de esta magnitud. Abuela, dondequiera que estés, ya estoy un paso más cerca de convertirme en ingeniero.

Mairena del Alcor, agosto 2026

ACKNOWLEDGEMENTS

I would like to express my deep gratitude to my mentors, David Benavides and David Romero, for the guidance, supervision, and support they provided throughout the development of this Bachelor's Thesis. Their advice and experience have been essential in helping me understand the context of the project, make the appropriate functional and technical decisions, and face the various challenges that arose during its completion.

I would also like to thank the DiversoLab team and everyone who contributed to the development of the technological environment on which this project was built for providing a framework in which to apply the knowledge I acquired during my undergraduate studies.

Finally, I dedicate this project to my family and all those close to me. Their support and trust have undoubtedly been essential to completing this project and tackling the effort required for a project of this magnitude. Grandma, wherever you are, I'm one step closer to becoming an engineer.

Mairena del Alcor, august 2026

RESUMEN

Este Trabajo de Fin de Grado aborda la evolución e integración de un generador configurable de modelos de características en el entorno UVLHub, una plataforma orientada a la gestión y manipulación de modelos escritos en Universal Variability Language (UVL). La necesidad de este componente surge de la escasez de modelos UVL expresivos y configurables que permitan evaluar herramientas de análisis de variabilidad software, especialmente aquellos que incorporan niveles avanzados del lenguaje como atributos y restricciones complejas. Para ello, se ha desarrollado una arquitectura modular del generador como plugin del ecosistema Flamapy, separando la definición de los modelos, su configuración y las operaciones de generación. Además, se ha adaptado el módulo generador de UVLHub para delegar la lógica de generación en este nuevo componente independiente, mejorando la reutilización, mantenibilidad y capacidad de evolución del sistema. La solución ha sido implementada utilizando Flask (Python) y desplegada mediante contenedores Docker para garantizar un entorno reproducible. Como resultado, se ha conseguido desacoplar el generador de la aplicación principal manteniendo la funcionalidad existente y adaptando la infraestructura de pruebas asociada al módulo. El sistema cuenta además con una batería de 134 pruebas automatizadas superadas y pruebas de interfaz mediante Pytest y Selenium que validan el flujo completo de generación. La principal aportación del trabajo consiste en proporcionar una base más flexible y mantenible para la generación de modelos UVL destinados a la evaluación y desarrollo de herramientas de análisis de variabilidad.

Palabras clave: SPL, UVL, UVLHub, Flamapy, Modelos de Características, generación automática de modelos, Python, Docker, pruebas automatizadas.

ABSTRACT

This project addresses the development and integration of a configurable feature model generator within the UVLHub environment, a platform designed for the management and manipulation of models written in the Universal Variability Language (UVL). The need for this component arises from the scarcity of expressive and configurable UVL models that enable the evaluation of software variability analysis tools, especially those that incorporate advanced language features such as attributes and complex constraints. To address this, a modular architecture for the generator has been developed as a plugin for the Flamapy ecosystem, separating model definition, configuration, and generation operations. In addition, the UVLHub generator module has been adapted to delegate the generation logic to this new independent component, improving the system's reusability, maintainability, and scalability. The solution was implemented using Flask (Python) and deployed via Docker containers to ensure a reproducible environment. As a result, the generator has been decoupled from the main application while maintaining existing functionality and adapting the testing infrastructure associated with the module. Plus, the system includes a suite of 134 successfully passed automated tests and interface tests using Pytest and Selenium that validate the entire generation workflow. The main contribution of this work is to provide a more flexible and maintainable foundation for generating UVL models intended for the evaluation and development of variability analysis tools.

Keywords: SPL, UVL, UVLHub, Flamapy, Feature Models, automatic model generation, Python, Docker, automated testing.

ÍNDICE GENERAL

Declaración de autoría	ii
Agradecimientos	iv
Acknowledgements	v
Resumen	vii
Abstract	viii
Índice de tablas	xvi
Índice de figuras	xix
Lista de acrónimos	xxi
1. Introducción	1
1.1. Contexto	1
1.2. Problema	2
1.3. Motivación	3
1.4. Objetivos	4
1.5. Alcance	8
1.6. Estructura de la memoria	9
2. Estado del arte	12
2.1. Ingeniería de Líneas de Productos Software (SPL)	13
2.1.1. Líneas de Productos Software	13
2.1.2. Modelos de características	14
2.1.3. Análisis automático y motores de razonamiento	15
2.2. Universal Variability Language (UVL)	18
2.2.1. Estructura básica de un modelo UVL	19
2.2.2. Sintaxis	19
2.2.3. Diseño del lenguaje	20
2.2.4. Niveles del lenguaje	23
2.3. Ecosistema de herramientas y aplicación en variabilidad software	27
2.3.1. Flamapy como framework de análisis de variabilidad	27
2.3.2. UVLHub como plataforma de gestión de modelos UVL	29
2.3.3. Generación automática de modelos de características	31

2.4. Trabajos relacionados	32
2.4.1. S.P.L.O.T.	33
2.4.2. FeatureIDE	33
2.4.3. Comparativa de trabajos relacionados	34
3. Planificación del proyecto	35
3.1. Acta de constitución del proyecto	35
3.1.1. Interesados del proyecto	37
3.2. Metodología	37
3.2.1. Metodología de desarrollo	37
3.2.2. Metodología de validación y evaluación	39
3.3. Estructura de descomposición del trabajo (EDT)	40
3.4. Cronograma e hitos (línea base)	41
3.5. Plan de comunicaciones y seguimiento	44
3.6. Gestión de riesgos	46
3.6.1. Seguimiento de riesgos al cierre	47
3.7. Control de cambios	50
3.8. Plan de calidad	53
3.9. Presupuesto	55
3.10. Cierre del proyecto y lecciones aprendidas	59
4. Análisis de requisitos	61
4.1. Descripción del sistema	61
4.2. Requisitos de información	63
4.2.1. Requisitos de información	63
4.3. Requisitos funcionales	69
4.4. Requisitos no funcionales	71
4.5. Reglas de negocio	73
4.6. Restricciones del proyecto	81
4.7. Casos de uso	82
4.7.1. Diagrama general de casos de uso	82
4.7.2. Diagrama general de casos de uso	82
4.7.3. Especificación de casos de uso principales	83
4.8. Matriz de trazabilidad	88
4.8.1. Requisitos funcionales $\leftrightarrow$ casos de uso	88
4.8.2. Requisitos $\leftrightarrow$ pruebas	89
5. Diseño de la solución	92
5.1. Modelo conceptual del dominio	92
5.1.1. Estructura del modelo	93

5.1.2.	Restricciones del dominio	95
5.1.3.	Espacio de soluciones / configuraciones	96
5.2.	Arquitectura del sistema	97
5.2.1.	Visión general	97
5.2.2.	Diagrama de componentes	98
5.2.3.	Diagrama de secuencia del flujo principal	99
5.3.	Patrones de diseño aplicados	101
5.4.	Diseño de la interfaz de usuario	102
5.4.1.	Mapa de navegación	102
5.4.2.	Wireframes y mockups	111
5.4.3.	Decisiones de diseño visual	115
5.5.	Esquema de base de datos	117
5.5.1.	Modelo entidad-relación	117
5.5.2.	Migraciones / versionado del esquema	118
5.6.	Flujo de datos principal	118
5.6.1.	Estructura de la representación intermedia	119
5.6.2.	Mapeo entre niveles	120
5.6.3.	Trazabilidad e identificadores	121
6.	Implementación	122
6.1.	Análisis de tecnologías	122
6.2.	Tecnologías utilizadas	125
6.3.	Módulo de validación / lógica principal	130
6.3.1.	Validación sintáctica / comprobaciones básicas	130
6.3.2.	Validación semántica / restricciones de dominio	133
6.4.	Módulo de generación de artefactos	136
6.4.1.	Cálculos o transformaciones específicas	136
6.4.2.	Síntesis del artefacto final	139
6.4.3.	Operaciones complementarias	140
6.5.	API REST / capa de servicios	140
6.5.1.	Endpoints implementados	140
6.5.2.	Descarga / exportación de artefactos	142
6.6.	Integración continua	142
7.	Pruebas y validación	144
7.1.	Estrategia de pruebas	144
7.2.	Pruebas unitarias	145
7.3.	Pruebas de integración	146
7.4.	Pruebas de extremo a extremo	147
7.5.	Cobertura y métricas de calidad	149

8. Resultados y discusión	152
8.1. Diseño de la evaluación	152
8.2. Resultados	153
8.2.1. Resultados sobre el objetivo O1	154
8.2.2. Resultados sobre el objetivo O2	157
8.3. Comparación con trabajos previos	160
8.4. Discusión	162
9. Conclusiones y trabajo futuro	164
9.1. Conclusiones	164
9.2. Limitaciones	167
9.3. Trabajo futuro	168
Bibliografía	171
A. Apéndice	173
B. Manual de instalación y despliegue	175
B.1. Requisitos previos	175
B.2. Obtención del código fuente	176
B.3. Instalación de dependencias y ejecución	176
B.4. Despliegue en producción	176
B.5. Verificación de la instalación	177
B.6. Verificación de la instalación	177
C. Manual de usuario	178
C.1. Acceso al sistema	178
C.2. Funcionalidad principal	179
C.3. Mensajes de error y resolución de incidencias	180
C.4. Contacto y soporte	181

ÍNDICE DE TABLAS

1.1. Criterios de éxito de los objetivos y capítulos donde se verifican.	7
2.1. Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.	18
2.2. Comparativa de trabajos relacionados con la generación automática de modelos de características.	34
3.1. Acta de constitución del proyecto.	36
3.2. Registro de interesados del proyecto.	37
3.3. Estructura de descomposición del trabajo (EDT).	41
3.4. Fases del desarrollo e hitos asociados (línea base de esfuerzo y calen- dario vs. valores reales).	42
3.5. Fases del desarrollo e hitos asociados (línea base de esfuerzo y calen- dario vs. valores reales).	43
3.6. Fases de redacción de la memoria (estimadas vs. reales).	44
3.7. Resumen global del proyecto (estimadas vs. reales).	44
3.8. Plan de comunicaciones y seguimiento del proyecto.	45
3.9. Plan de comunicaciones y seguimiento del proyecto.	45
3.10. Riesgos identificados, exposición y planes de mitigación.	46
3.11. Riesgos identificados, exposición y planes de mitigación.	47
3.12. Seguimiento de riesgos al cierre: materialización, respuesta y residuales.	48
3.13. Seguimiento de riesgos al cierre: materialización, respuesta y residuales.	50
3.14. Registro de control de cambios sobre la línea base.	51
3.15. Registro de control de cambios sobre la línea base.	53
3.16. Costes de capital (CapEx).	55
3.17. Costes de capital (CapEx).	56
3.18. Desglose de costes de personal (OpEx).	56
3.19. Desglose de costes de personal (OpEx).	57
3.20. Costes de herramientas y servicios de desarrollo (OpEx).	57
3.21. Costes de herramientas y servicios de desarrollo (OpEx).	58
3.22. Resumen del presupuesto del proyecto ($TCO = CapEx + OpEx$).	58
3.23. Resumen del presupuesto del proyecto ($TCO = CapEx + OpEx$).	59
3.24. Lecciones aprendidas al cierre del proyecto.	59
3.25. Lecciones aprendidas al cierre del proyecto.	60
4.1. Atributos del requisito de información RI-01.	63

4.2. Atributos del requisito de información RI-02.	63
4.3. Atributos del requisito de información RI-01.	64
4.4. Atributos del requisito de información RI-02.	65
4.5. Atributos del requisito de información RI-03.	65
4.6. Atributos del requisito de información RI-04.	66
4.7. Atributos del requisito de información RI-05.	67
4.8. Atributos del requisito de información RI-06.	68
4.9. Atributos del requisito de información RI-07.	69
4.10. Requisitos funcionales del sistema.	70
4.11. Requisitos funcionales del sistema.	71
4.12. Requisitos no funcionales del sistema.	72
4.13. Requisitos no funcionales del sistema.	73
4.14. Reglas de negocio del dominio.	74
4.15. Reglas de negocio del dominio.	80
4.16. Restricciones del proyecto.	81
4.17. Restricciones del proyecto.	82
4.18. Especificación del caso de uso CU-01.	83
4.19. Especificación del caso de uso CU-01.	84
4.20. Especificación del caso de uso CU-02.	85
4.21. Especificación del caso de uso CU-03.	86
4.22. Especificación del caso de uso CU-04.	87
4.23. Especificación del caso de uso CU-05.	88
4.24. Matriz de trazabilidad entre requisitos funcionales y casos de uso. . .	89
4.25. Matriz de trazabilidad entre requisitos funcionales y casos de uso. . .	89
4.26. Matriz de trazabilidad entre requisitos y pruebas.	89
4.27. Matriz de trazabilidad entre requisitos funcionales y pruebas.	91
5.1. Esquema de la tabla <i>[nombre_tabla_1]</i>	117
5.2. Esquema de la tabla <i>[nombre_tabla_2]</i>	117
6.1. Bloques de configuración de <code>FmgeneratorModel</code>	133
7.1. Resumen de la suite de pruebas por nivel de la pirámide.	149
8.1. Tiempos obtenidos durante la evaluación temporal del generador. Elaboración propia	159
9.1. Grado de cumplimiento de los objetivos y evidencia asociada.	165
9.2. Grado de cumplimiento de los objetivos. Elaboración propia	166

ÍNDICE DE FIGURAS

2.1. Representación de una línea de productos software basada en activos comunes. Extraído de [11].	14
2.2. Ejemplo de modelo de características. Extraído de [15].	15
2.3. Ejemplo de modelo de características. Elaboración propia.	16
2.4. Funcionamiento de cada motor de razonamiento. Elaboración propia.	17
2.5. Arquitectura general de UVLHub y sus integraciones principales. Extraído de [17]	30
2.6. Arquitectura general de UVLHub y sus integraciones principales. Extraído de [17]	31
3.1. Algunos commits del proyecto. Elaboración propia.	39
3.2. Arquitectura general de UVLHub y sus integraciones principales. Elaboración propia.	54
4.1. Diagrama general de casos de uso del sistema. Elaboración propia	83
5.1. Modelado conceptual del sistema. Elaboración propia	93
5.2. Diagrama de componentes del sistema. Elaboración propia	99
5.3. Interfaz del Step 1: Batch. Elaboración propia	103
5.4. Interfaz del Step 2: Language levels. Elaboración propia	104
5.5. Interfaz del Step 3: Feature tree. Elaboración propia	105
5.6. Interfaz del Step 3: Feature tree. Elaboración propia	106
5.7. Interfaz del Step 4: Constraints. Elaboración propia	107
5.8. Interfaz del Step 4: Constraints. Elaboración propia	107
5.9. Interfaz del Step 5: Attributes. Elaboración propia	108
5.10. Interfaz del Step 5: Attributes. Elaboración propia	109
5.11. Interfaz del Step 6: Output. Elaboración propia	110
5.12. Interfaz del Step 6: Output. Elaboración propia	110
5.13. Interfaz del Step 6: Output. Elaboración propia	111
5.14. Mapa de navegación del generador UVL. Elaboración propia	111
5.15. Wireframe inicial del Step 1 del generador. Elaboración propia	112
5.16. Wireframe inicial del Step 2 del generador. Elaboración propia	113
5.17. Wireframe inicial del Step 3 del generador. Elaboración propia	114
5.18. Wireframe inicial del Step 4 del generador. Elaboración propia	114
5.19. Wireframe inicial del Step 5 del generador. Elaboración propia	115
5.20. Componente selector de distribuciones. Elaboración propia	116

5.21. Flujo de datos principal. Elaboración propia	119
5.22. Ejemplo de uso de la <i>session</i> . Elaboración propia	120
6.1. Estructura del módulo generator de UVLHub. Elaboración propia . .	126
6.2. Estructura del plugin generador basado en Flamapy. Elaboración propia	127
6.3. Plantillas HTML utilizadas para los pasos del wizard. Elaboración propia	128
6.4. Scripts y dependencias utilizadas para la ejecución con Pyodide. Ela- boración propia	128
6.5. Configuración de Docker en UVLHub. Elaboración propia	129
6.6. Suite de pruebas del módulo generator. Elaboración propia	129
8.1. Configuración usada para verificar O1. Elaboración propia	155
8.2. Ejemplo 1 para verificar O1. Elaboración propia	156
8.3. Ejemplo 2 para verificar O1. Elaboración propia	157
8.4. Resultado temporal de la generación de modelos. Elaboración propia	159
8.5. Resultado de la ejecución de la suite de pruebas automatizadas. Ela- boración propia	160
B.1. Salida de consola tras el arranque correcto del contenedor web. Elabo- ración propia	177
C.1. Pantalla inicial de UVLHub. Elaboración propia	179
C.2. Ejemplo de error durante la generación de modelos. Elaboración propia	181

LISTA DE ACRÓNIMOS

[Lista alfabética de los acrónimos y siglas utilizados en la memoria. Mantén la lista ordenada y elimina las entradas que no aparezcan finalmente en el documento. Si un término solo se usa una vez, valora si compensa introducir el acrónimo o desarrollarlo en el cuerpo del texto. Convención de primera mención: todo acrónimo se desarrolla la primera vez que aparece en el texto —«Ingeniería del Software Basada en Evidencias (EBSE)»— y a partir de entonces se usa solo la sigla. Esta lista es una referencia de consulta: no sustituye a esa primera mención desarrollada.]

SPL	Software Product Lines
UVL	Universal Variability Language
FLOSS	Free/Libre Open Source Software
TASOVA	Red en nuevas Tendencias en Arquitectura Software y Variabilidad
SAT	Boolean Satisfiability Problem
BDD	Binary Decision Diagrams
DOI	Digital Object Identifier

GUÍA DE ESTILO Y REVISIÓN (NO APARECE EN LA VERSIÓN FINAL)

Este capítulo es una guía para ti: solo se imprime en modo borrador y desaparece de forma automática al compilar con `\draftfalse`. Léelo antes de empezar a redactar y vuelve a él durante la revisión final. Una memoria con buen contenido pero mal escrita pierde puntos fáciles; las convenciones siguientes evitan los errores más frecuentes.

Registro y voz

Elige una voz y mantenla en toda la memoria: la forma impersonal o pasiva refleja («se ha desarrollado», «se propone») o la primera persona del plural («hemos desarrollado»). Ambas son aceptables; mezclarlas no. La única excepción son los agradecimientos, que van en primera persona del singular.

Usa el tiempo pasado (pretérito perfecto) para el trabajo realizado («se ha implementado un módulo de autenticación») y el presente para describir lo que el sistema es o hace («el sistema valida cada petición»). Reserva el futuro y el condicional para el trabajo futuro. Evita los coloquialismos y muletillas («un montón», «básicamente», «a día de hoy») y no te dirijas al lector en segunda persona.

Vocabulario técnico

Un concepto, un término: si llamas «usuario» a quien usa la aplicación, no alternes con «actor» o «cliente» para referirte a lo mismo. Haz una pasada final de búsqueda para comprobar la consistencia. Los anglicismos sin traducción asentada van en cursiva con `\textit`: *framework*, *endpoint*, *backend*. Si existe un término español habitual (prueba, despliegue, biblioteca, servidor), úsalo, y úsalo siempre igual.

Los comandos, el código, las rutas y los identificadores van en `\texttt`: `main.tex`, `POST /api/login`, `docker compose up`. Todo acrónimo se desarrolla en su primera mención en el texto —«Ingeniería del Software Basada en Evidencias (EBSE)»— y a partir de entonces se usa solo la sigla; además, debe quedar recogido en la lista de acrónimos.

Prosa antes que viñetas

Las listas-guía de esta plantilla son andamiaje: indican qué contenido debe aparecer, no cómo debe presentarse. El texto final debe ser prosa argumentada, con párrafos conectados mediante conectores («Sin embargo», «Por tanto», «En consecuencia»). Una sección no puede ser solo una sucesión de viñetas; reserva las listas para enumeraciones reales (requisitos, objetivos, pasos de un procedimiento). Las citas textuales van siempre con `\enquote{...}` (el paquete `csquotes` ya está cargado), nunca con comillas tecleadas a mano.

Figuras y tablas

Toda figura y toda tabla se referencia desde el texto **antes** de aparecer («como muestra la Tabla 3.1...», usando `Tabla~\ref{...}`) y se interpreta: no basta con insertarla, hay que decir al lector qué debe ver en ella. Los pies deben ser autoexplicativos, de modo que se entiendan sin leer el cuerpo del texto («Comparación del tiempo medio de respuesta (ms) de las tres alternativas evaluadas» en lugar de «Resultados»).

Checklist de revisión final

- ☐ Corrector ortográfico y gramatical pasado a todo el documento (el integrado en Overleaf y, como complemento, LanguageTool o similar).
- ☐ Lectura completa del PDF de principio a fin, mejor impresa o en voz alta: delata las frases que no fluyen.
- ☐ Terminología consistente: el mismo término para el mismo concepto en toda la memoria; anglicismos en cursiva; código y comandos en `\texttt`.
- ☐ Acrónimos desarrollados en su primera mención y recogidos en la lista de acrónimos (sin entradas sobrantes).
- ☐ Compilado con `\draftfalse` y revisado el log de compilación: ningún aviso «Placeholder sin rellenar» pendiente.
- ☐ Ningún título de sección ni caption de tabla/figura sigue siendo un placeholder (en modo final dejarían entradas vacías en el índice o pies vacíos).
- ☐ Toda entrada de la bibliografía se cita en el texto y toda afirmación tomada de una fuente externa lleva su cita.

- ☐ Todas las figuras y tablas se referencian desde el texto y tienen pie autoexplicativo.
- ☐ Metadatos del PDF completos: bloque «DATOS DEL TRABAJO» de `main.tex` relleno (título, autor/a, titulación, tutores, ciudad y fecha).

1 INTRODUCCIÓN

[Este capítulo presenta el trabajo. Da al lector el contexto mínimo para entender qué se aborda, por qué es relevante, qué se pretende conseguir y qué queda fuera. No se entra todavía en detalles técnicos: eso pertenece al estado del arte y al diseño. Fija aquí el registro que mantendrás en toda la memoria: redacción impersonal (“se ha desarrollado”) o primera persona del plural (“hemos desarrollado”), pero consistente de principio a fin, y prosa académica sin coloquialismos (consulta la Guía de estilo incluida al inicio del documento en modo borrador).]

1.1 Contexto

[Describe el dominio en el que se enmarca el TFG y por qué es relevante. Aporta el trasfondo necesario para que un lector ajeno al tema entienda el resto del documento. Un lector tras leer esta sección debería poder responder a: “¿de qué va este trabajo y en qué entorno se sitúa?”.]

[Sugerencia de contenido:]

- *[Sector o disciplina (industria, investigación, educación...)].*
- *[Tendencias o cambios recientes que motivan el problema.]*
- *[Conceptos básicos imprescindibles para entender el resto.]*

El desarrollo de software actual debe dar respuesta a sistemas cada vez más configurables, capaces de adaptarse a distintas necesidades, plataformas y contextos de uso. Dentro de este escenario, las Líneas de Productos Software (Software Product Lines, SPL) [1] ofrecen un enfoque para gestionar familias de productos relacionados a partir de los elementos que comparten y de aquellos que pueden variar entre unas configuraciones y otras. Esto permite tratar la variabilidad de forma explícita y facilita tareas como la configuración de productos, la reutilización de componentes o el análisis automático de las distintas combinaciones posibles.

Una de las formas más extendidas de representar esta variabilidad son los modelos de características (Feature Models) [2]. Estos modelos describen las características que puede incluir una familia de productos y las relaciones que existen entre ellas, permitiendo establecer qué elementos son obligatorios u opcionales, cuáles pueden combinarse y qué restricciones deben respetarse para obtener una configuración válida.

A medida que los modelos han ganado en complejidad, también ha surgido la necesidad de disponer de formatos que permitan representarlos de manera común.

Universal Variability Language (UVL) [3] nace con este propósito y proporciona un lenguaje para describir modelos de variabilidad con diferentes grados de expresividad. Además de las relaciones habituales entre características, UVL permite incorporar construcciones más avanzadas, como atributos y restricciones, ampliando así los escenarios que pueden representarse y analizarse.

En torno a estos modelos se han desarrollado distintas herramientas destinadas a facilitar su procesamiento y análisis. Entre ellas se encuentra Flamapy [4], un framework desarrollado en Python que proporciona soporte para el análisis automático de modelos de características. Su arquitectura extensible, basada en plugins, permite incorporar nuevos formatos, operaciones y técnicas de análisis sin necesidad de modificar el núcleo de la herramienta, lo que facilita su evolución y su uso en diferentes líneas de investigación.

El presente Trabajo Fin de Grado se desarrolla dentro de este ecosistema y se enmarca en la actividad investigadora de DiversoLab y en TASOVA Plus (Red en nuevas Tendencias en Arquitectura Software y Variabilidad) [5]. Este entorno promueve el desarrollo de herramientas relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, siguiendo además principios asociados al software libre y a la ciencia abierta (Open Science).

Otro de los componentes de este ecosistema es UVLHub [6], una plataforma orientada a la publicación, gestión y exploración de modelos expresados en UVL. La plataforma ofrece un espacio en el que investigadores y desarrolladores pueden compartir modelos y reutilizarlos en distintos experimentos o herramientas. Flamapy, UVL y UVLHub conforman así buena parte del entorno tecnológico sobre el que se desarrolla este trabajo.

1.2 Problema

[Plantea el problema concreto que el TFG resuelve. Conviene formularlo en términos de limitaciones reales del estado actual: qué no se puede hacer hoy o qué se hace mal, y qué consecuencias tiene.]

[Estructura recomendada (1–3 problemas claramente delimitados):]

1. **[Limitación 1:]** *[descripción del problema y de su impacto.]*
2. **[Limitación 2:]** *[descripción del problema y de su impacto.]*
3. **[Limitación 3:]** *[descripción del problema y de su impacto.]*

La evolución de los lenguajes de modelado y de los motores de razonamiento ha permitido trabajar con formas de variabilidad cada vez más complejas. Sin embargo, disponer de herramientas capaces de procesar esa complejidad no resulta suficiente

por sí solo: para poder evaluarlas también es necesario contar con modelos que hagan uso de esas capacidades.

Buena parte de los modelos de características disponibles se concentra en construcciones relativamente básicas. Estos modelos siguen siendo útiles para numerosos análisis, pero resultan menos adecuados cuando se pretende experimentar con elementos más avanzados del lenguaje o estudiar cómo se comportan los motores de razonamiento ante distintos grados de complejidad. Se produce así una diferencia entre la expresividad que pueden ofrecer los lenguajes y herramientas actuales y la variedad de modelos disponibles para ponerlos a prueba.

Es por ello que se identifican de forma explícita las siguientes limitaciones:

1. Expresividad limitada de los modelos de características disponibles:

Muchos de estos modelos se basan principalmente en relaciones estructurales básicas entre características. Aunque estos modelos permiten realizar análisis fundamentales, no siempre permiten representar escenarios más complejos donde intervienen elementos avanzados del lenguaje, lo que reduce la capacidad de explorar nuevas técnicas de razonamiento y análisis.

2. Complicaciones a la hora de evaluar y mejorar motores de razonamiento avanzados sobre modelos más complejos:

Esto es consecuencia directa del problema anteriormente descrito. Sin estos modelos más avanzados, la capacidad de validar nuevas aproximaciones se ve limitada, por lo que es más complicado estudiar hasta qué punto estos motores pueden aprovechar construcciones más avanzadas del lenguaje de modelado.

3. Necesidad de ampliar el conjunto de modelos disponibles para investigación en variabilidad software

La investigación en este ámbito requiere disponer de modelos que permitan experimentar con diferentes escenarios, niveles de complejidad y características del lenguaje. Sin una base amplia y diversa de modelos, resulta mucho más difícil comparar técnicas, evaluar resultados y avanzar en el desarrollo de herramientas más potentes.

1.3 Motivación

[Justifica por qué merece la pena resolver el problema y por qué con este enfoque. Conecta el problema (sección anterior) con la solución que se va a proponer (sin entrar todavía en cómo se implementa).]

[Puntos típicos:]

- *[Por qué el enfoque elegido es adecuado.]*
- *[Qué aporta respecto a soluciones existentes.]*

- *[Encaje del trabajo en un contexto académico, profesional o de investigación (grupo de investigación, asignatura, empresa, publicación...)].*

La disponibilidad de modelos de características con distintos niveles de expresividad resulta fundamental para continuar avanzando en el desarrollo y evaluación de técnicas de análisis automático de variabilidad software. Como se ha comentado anteriormente, la evolución de los lenguajes de modelado permite representar escenarios cada vez más complejos, pero esta capacidad debe estar acompañada de modelos que permitan estudiar y validar el comportamiento de las herramientas encargadas de procesarlos.

Abordar esta necesidad dentro de un entorno de investigación permite trabajar sobre problemas que aparecen de forma directa durante el desarrollo de nuevas técnicas y herramientas. En este sentido, el contexto proporcionado por TASOVA Plus y DiversoLab ofrece un marco adecuado para contribuir al avance de soluciones relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, lo que además permite seguir principios asociados al software libre y la reutilización del conocimiento científico.

Desde una perspectiva académica y profesional, este Trabajo Fin de Grado representa también una oportunidad para aplicar los conocimientos adquiridos durante el grado en un proyecto software real. Durante su desarrollo, he tenido la oportunidad de trabajar dentro de un ecosistema ya existente, enfrentándome a aspectos habituales en la ingeniería del software, como la comprensión de código desarrollado previamente, la integración de nuevas funcionalidades, la documentación o la validación mediante pruebas automatizadas.

Además, la participación en un proyecto vinculado a investigación me ha permitido adquirir experiencia en un contexto diferente al de los desarrollos software tradicionales. La necesidad de analizar información técnica, consultar documentación especializada y tomar decisiones fundamentadas ha favorecido el desarrollo de habilidades relacionadas con la búsqueda autónoma de conocimiento y la resolución de problemas abiertos.

1.4 Objetivos

*[Define el objetivo general del trabajo en una o dos frases, y a continuación los objetivos específicos en forma de lista. Formula cada objetivo específico siguiendo los criterios **SMART**: **S**pecific (específico: una sola cosa, sin ambigüedad), **M**easurable (medible: con una métrica o criterio de éxito verificable), **A**chievable (alcanzable: realista con los recursos y el plazo de un TFG), **R**elevant (relevante: conectado con una de las limitaciones de la Sección 1.2) y **T**ime-bound (acotado en el tiempo: realizable dentro del calendario del Capítulo 3).]*

[Usa verbos de resultado, no de actividad: evita “estudiar”, “aprender” o “profundizar en” (no se puede verificar cuándo terminan) y prefiere “implementar X que soporte Y”, “alcanzar una cobertura del X %”, “reducir el tiempo de Z por debajo de N segundos”. Ejemplo de objetivo bien formulado: “Implementar un módulo de búsqueda que resuelva consultas sobre 10 000 registros en menos de 200 ms, dando respuesta a la Limitación 1”.]

[Objetivo general: “Desarrollar/implementar/diseñar ... con el fin de ...”.]

- O1.** *[Objetivo específico 1:] [descripción concreta y medible.]*
- O2.** *[Objetivo específico 2:] [descripción concreta y medible.]*
- O3.** *[Objetivo específico 3:] [descripción concreta y medible.]*
- O4.** *[Objetivo específico 4:] [descripción concreta y medible.]*
- O5.** *[Objetivo específico 5 (opcional):] [descripción concreta y medible.]*

El objetivo principal de este trabajo es dar apoyo a la mejora de los motores de razonamiento existentes para el análisis de modelos de características mediante la obtención de una cantidad significativa y variada de modelos con expresividad avanzada. Como propuesta para conseguirlo, se pretende desarrollar un generador de modelos de características aleatorio y completamente configurable para poder cubrir todas las posibilidades en las que un modelo de características pueda ser diseñado [7].

De forma más específica, podemos desgranar los objetivos de este trabajo de la siguiente forma:

- O1.** Diseñar e implementar un generador automático de modelos de características capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad.
- O2.** Desarrollar un sistema de configuración flexible que permita controlar las características de los modelos generados y adaptarlos a distintos escenarios de experimentación.
- O3.** Incorporar soporte para la generación de modelos que utilicen construcciones avanzadas del lenguaje UVL, permitiendo ampliar la variedad de casos de estudio disponibles para el análisis de variabilidad.
- O4.** Integrar la solución desarrollada dentro del ecosistema Flamapy y facilitar su utilización desde UVLHub como herramienta orientada a la generación y gestión de modelos de características.

- O5.** Evaluar el comportamiento del generador bajo escenarios de generación con un volumen elevado de modelos, analizando el coste asociado a la inicialización del entorno y al proceso de generación.
- O6.** Validar el funcionamiento del generador mediante pruebas automatizadas que permitan comprobar la corrección de los modelos producidos.

De forma paralela, el trabajo persigue una serie de objetivos formativos que se pretenden conseguir con la realización del proyecto:

- O7.** Reutilizar el conocimiento científico en el desarrollo de herramientas software (objetivo heredado de la red TASOVA Plus)
- O8.** Adquirir experiencia en el desarrollo y mantenimiento de proyectos software reales mediante la participación en un entorno de investigación y código abierto.
- O9.** Profundizar en el uso de tecnologías empleadas en el proyecto, especialmente Python, Flask, Docker y herramientas asociadas a integración y despliegue continuo (CI/CD).
- O10.** Mejorar la capacidad de búsqueda, análisis y selección de información técnica relevante para resolver problemas dentro de un contexto de investigación.
- O11.** Aplicar buenas prácticas de planificación, gestión y documentación durante el desarrollo individual de un proyecto software de mayor duración y complejidad.

*[Resume en la tabla siguiente el criterio de éxito de cada objetivo, cómo se verificará y en qué capítulo. Esta tabla es la columna vertebral de la memoria: el Capítulo 8 dará el veredicto sobre **todos** los criterios, recapitulando con \ref la evidencia de aquellos que se verifican en capítulos anteriores, y el Capítulo 9 sintetizará el grado de cumplimiento final. Si el criterio es cuantitativo, fíjalo también como RNF en el Capítulo 4 con el **mismo** umbral: un único umbral, dos vistas. Conviene rellenar la columna «Cap.» con \ref (p. ej. Capítulo 7) para que la numeración se mantenga sola.]*

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O1	El generador permite producir modelos UVL automáticamente incorporando diferentes niveles de complejidad y expresividad del lenguaje, ampliando la variedad de modelos disponibles para el análisis de variabilidad.	Generación de modelos con configuraciones variadas y validación de que los modelos producidos cumplen la sintaxis y estructura esperada de UVL.	6, 7
O2	El generador dispone de un sistema de configuración que permite modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.	Revisión de las opciones de configuración disponibles y ejecución de pruebas con diferentes configuraciones para comprobar su efecto sobre los modelos generados.	6, 7
O3	La solución desarrollada queda integrada dentro del ecosistema Flamapy y puede ser utilizada desde UVLHub como herramienta de generación de modelos.	Verificación de la integración del plugin en Flamapy y comprobación del flujo completo de generación desde UVLHub.	6, 7
O4	Evaluar el comportamiento temporal del generador en escenarios de generación con un volumen elevado de modelos de características.	Medición del tiempo de inicialización del generador y del tiempo requerido para generar un conjunto elevado de modelos UVL bajo una configuración representativa, analizando el coste asociado a cada fase del proceso.	8
O5	El generador cuenta con una batería de pruebas automatizadas que permita validar su correcto funcionamiento y detectar errores en los modelos producidos.	Ejecución de la suite de pruebas automatizadas y análisis de los resultados obtenidos.	7

Tabla 1.1: Criterios de éxito de los objetivos y capítulos donde se verifican.

1.5 Alcance

*[Delimita lo que el trabajo **sí** cubre y, sobre todo, lo que **no** cubre. Es la sección que protege al autor de expectativas desmedidas durante la defensa.]*

[Una posible estructura:]

[Lo que sí se aborda:]

- *[Funcionalidad / dominio / tipo de entradas que el sistema acepta.]*

- *[Restricciones cuantitativas (rangos, tamaños, plataformas...)]*

[Quedan explícitamente fuera del alcance:]

- *[Funcionalidad excluida 1 (con una breve justificación).]*

- *[Funcionalidad excluida 2 (con una breve justificación).]*

- *[Funcionalidad excluida 3 (con una breve justificación).]*

Este TFG comprende el desarrollo de un generador automático de modelos de características integrado dentro del ecosistema de herramientas asociado a Flamapy y orientado principalmente a su utilización desde UVLHub. La solución desarrollada permite generar modelos expresados en UVL mediante una configuración definida por el usuario, incorporando diferentes niveles de complejidad y expresividad con el objetivo de ampliar la disponibilidad de modelos para tareas de análisis e investigación en variabilidad software.

Dentro del alcance del proyecto se incluye el diseño e implementación del generador como un nuevo plugin de Flamapy, siguiendo la arquitectura extensible del framework. Asimismo, se contempla su integración dentro del módulo *generator* de UVLHub, permitiendo que los usuarios de la plataforma puedan interactuar con la funcionalidad desarrollada mediante una interfaz adaptada al flujo existente de la aplicación. Esta integración incluye la definición de los mecanismos necesarios para configurar el proceso de generación, la incorporación de soporte para diferentes construcciones del lenguaje UVL y la validación del funcionamiento del generador mediante pruebas automatizadas destinadas a comprobar la corrección de los modelos producidos.

En cuanto a lo que queda **fuera** del alcance del proyecto, se consideran los siguientes aspectos:

- La creación de una aplicación web independiente para el generador. Aunque la integración realizada en UVLHub incorpora una interfaz que permite configurar y ejecutar el proceso de generación, el generador como componente reutilizable no incluye una interfaz propia. En caso de utilizarse fuera de UVLHub, sería necesario desarrollar una capa de interacción adaptada al contexto concreto de uso.

- La modificación o ampliación de otros módulos de UVLHub distintos al módulo *generator*, ya que la intervención realizada se limita a la incorporación del generador dentro del módulo correspondiente.
- El desarrollo de nuevos motores de razonamiento o algoritmos de análisis de variabilidad. El objetivo del generador es proporcionar modelos con diferentes características y niveles de expresividad que puedan servir como base experimental para este tipo de herramientas, pero esto es algo que se pretende conseguir indirectamente con su uso.

1.6 Estructura de la memoria

[Cierra la introducción con un mapa de lectura: una o dos frases por capítulo. No describas solo el contenido (“se presentan los requisitos”), sino el papel de cada capítulo en el argumento de la memoria (“se establece qué debe hacer el sistema, base contractual del diseño”).]

- **El Capítulo 2** *[revisa los fundamentos teóricos y los trabajos relacionados, identificando el hueco que este trabajo cubre.]*
- **El Capítulo 3** *[describe la metodología, las fases, los riesgos y el presupuesto con los que se ha gestionado el proyecto.]*
- **El Capítulo 4** *[especifica qué debe hacer el sistema: requisitos, casos de uso y su trazabilidad, base contractual del resto del trabajo.]*
- **El Capítulo 5** *[presenta cómo se estructura la solución: arquitectura, modelos y decisiones de diseño que responden a los requisitos.]*
- **El Capítulo 6** *[detalla cómo se materializa el diseño: tecnologías elegidas y módulos construidos.]*
- **El Capítulo 7** *[muestra cómo se ha verificado que el sistema cumple los requisitos y con qué estrategia de pruebas.]*
- **El Capítulo 8** *[contrasta los resultados obtenidos con los criterios de éxito de la Tabla 1.1 y los discute frente a los trabajos relacionados.]*
- **El Capítulo 9** *[da el veredicto sobre el cumplimiento de los objetivos, reconoce las limitaciones y propone líneas de trabajo futuro.]*

[Remata el roadmap con un párrafo puente de 2–4 líneas que lance al lector al siguiente capítulo. Por ejemplo: «Definidos el problema, los objetivos y el alcance,

el Capítulo 2 revisa los fundamentos y las soluciones existentes sobre los que se construye la propuesta».]

El resto de la memoria se organiza como sigue:

- El **Capítulo 2** presenta el **estado del arte** relacionado con la ingeniería de líneas de productos software, los modelos de características, el lenguaje UVL y las herramientas que forman parte del ecosistema en el que se desarrolla el trabajo. Además, se revisan trabajos relacionados con el objetivo de identificar las necesidades que justifican la propuesta desarrollada.
- El **Capítulo 3** describe la **planificación y gestión del proyecto**, incluyendo la metodología seguida durante el desarrollo, la organización de las tareas, la gestión de riesgos y los aspectos relacionados con la evolución del trabajo a lo largo del tiempo.
- El **Capítulo 4** define los **requisitos** que debe cumplir la solución desarrollada. En este capítulo se establecen las funcionalidades esperadas del generador, sus restricciones y la relación existente entre los requisitos y las pruebas destinadas a validarlos.
- El **Capítulo 5** presenta el **diseño de la solución** propuesta, describiendo la arquitectura general del sistema, la organización de sus componentes y las decisiones tomadas para integrar el generador dentro del ecosistema de Flamapy y UVLHub.
- El **Capítulo 6** detalla la **implementación** de la solución, incluyendo las tecnologías utilizadas, la estructura del plugin desarrollado, los mecanismos empleados para la generación de modelos y la integración realizada con la plataforma UVLHub.
- El **Capítulo 7** describe la estrategia de **pruebas** aplicada para verificar el correcto funcionamiento del sistema. En él se presentan las pruebas unitarias, de integración y de extremo a extremo utilizadas para validar la generación de modelos y la integración de la funcionalidad desarrollada.
- El **Capítulo 8** analiza los **resultados** obtenidos durante la evaluación del trabajo. Se contrastan los resultados con los criterios de éxito definidos para los objetivos y se discuten las aportaciones realizadas frente al contexto presentado en capítulos anteriores.
- El **Capítulo 9** recoge las **conclusiones** del trabajo, valorando el grado de cumplimiento de los objetivos planteados, las limitaciones encontradas y las posibles líneas de trabajo futuro derivadas de la solución desarrollada.

Definidos el contexto, el problema, los objetivos y el alcance del trabajo, el siguiente capítulo establece los fundamentos teóricos y tecnológicos necesarios para comprender el entorno de variabilidad software sobre el que se desarrolla la propuesta.

2 ESTADO DEL ARTE

[Este capítulo sitúa el trabajo en su contexto académico y técnico. Su función es doble: (1) introducir al lector los conceptos teóricos que necesita para entender el resto del documento y (2) demostrar que el autor conoce las soluciones existentes y dónde se sitúa la suya.]

[Estructura recomendada: 3–4 secciones temáticas que cubran los pilares conceptuales del TFG, más una sección final de Trabajos relacionados donde se comparan soluciones existentes con la propia.]

[Regla de oro de citación: toda definición, dato o afirmación que no sea tuya lleva su cita con \cite junto a la propia afirmación, no acumulada al final del párrafo. Prefiere la fuente primaria (el artículo o libro que introdujo el concepto) y combínala con publicaciones recientes (últimos cinco años) y con tipos de fuente diversos (artículos, congresos, libros, documentación oficial, software). Patrón de ejemplo: «El concepto X fue introducido por Apellido et al. [8] y desde entonces se ha aplicado a ... ». Consulta `bibliography.bib`: incluye un modelo de cada tipo de entrada.]

- 2.1 Ingeniería de Líneas de Productos Software (SPL)
 - 2.1.1 Líneas de Productos Software
 - 2.1.2 Modelos de características
 - 2.1.3 Análisis automático y motores de razonamiento / herramientas y ecosistema (según veamos cómo encaje mejor)
- 2.2 Universal Variability Language (UVL)
 - 2.2.1 Basic structure of a UVL model
 - 2.2.2 Syntax: Language specification
 - 2.2.3 Lenguaje design
 - 2.2.4 Language levels
- 2.3 Ecosistema de herramientas y aplicación en variabilidad software
 - 2.3.1 Flamapy como framework de análisis de variabilidad
 - 2.3.2 UVLHub como plataforma de gestión de modelos UVL
 - 2.3.3 Generación automática de modelos de características

2.1 Ingeniería de Líneas de Productos Software (SPL)

[Primer pilar teórico del trabajo. Por ejemplo: el paradigma, la metodología o el tipo de sistema sobre el que se construye la solución, apoyado en su fuente primaria, típicamente un libro de referencia [9].]

[Cubre aquí:]

- *[Definición y origen del concepto.]*
- *[Por qué es relevante para este TFG.]*
- *[Conceptos auxiliares que el lector necesita conocer.]*

2.1.1 Líneas de Productos Software

A lo largo de la historia, el desarrollo de software se ha basado tradicionalmente en la construcción de sistemas individuales orientados a resolver necesidades concretas, donde cada nuevo producto software suele desarrollarse como un proyecto independiente, incluso cuando comparte una parte significativa de sus funcionalidades, arquitectura o componentes con otros sistemas existentes. Esta estrategia puede resultar adecuada cuando el número de productos es reducido; sin embargo, hoy en día esto presenta limitaciones importantes para la gran mayoría de organizaciones, ya que surge de forma paralela la necesidad de adaptar un determinado sistema base a una gran cantidad de clientes con problemas ligeros o enormemente diferentes.

Debido a este hecho surge la Ingeniería de Líneas de Productos Software, definidas por Clements y Northrop como "*a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way*"[10]. Gracias a ellas es posible identificar, gestionar y reutilizar aquellos elementos comunes entre diferentes productos, a la vez que se mantienen explícitamente aquellos aspectos que pueden variar entre las distintas configuraciones posibles, haciendo así factible y mucho más viable la gestión de dichos elementos en lugar de construir cada producto desde cero.

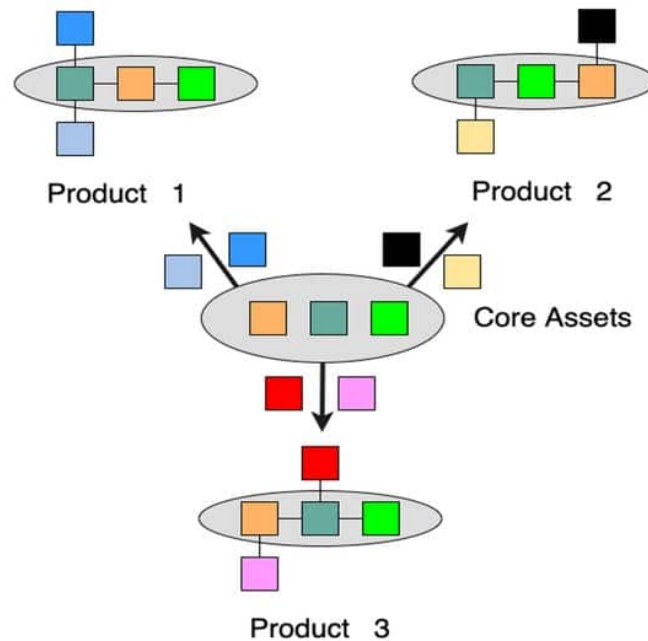


Figura 2.1: Representación de una línea de productos software basada en activos comunes. Extraído de [11].

La existencia de elementos comunes introduce el concepto de variabilidad: "*...is the ability of a software system or software artifact to be changed, customized, or configured for use in a particular context*"[12]. Esta debe ser identificada, representada y gestionada durante el proceso de desarrollo, ya que permite definir qué elementos pueden formar parte de cada producto y bajo qué condiciones pueden combinarse.

La gestión de esta variabilidad requiere distinguir entre dos actividades complementarias. Por una parte, la ingeniería de dominio (*domain engineering*) se encarga de analizar un determinado dominio, identificar sus elementos comunes y variables, y construir los activos reutilizables que formarán la base de la línea de productos. Por otra parte, la ingeniería de aplicación (*application engineering*) utiliza dichos activos para derivar productos concretos mediante la selección de las características necesarias y la resolución de las variabilidades definidas.

No obstante, este paradigma introduce consigo retos relacionados con la representación y gestión de la complejidad. A medida que aumenta el número de características variables y configuraciones posibles, resulta necesario disponer de mecanismos que permitan gestionar esta variabilidad de forma íntegra y precisa, lo cual da surgimiento a técnicas como los modelos de características.

2.1.2 Modelos de características

Los modelos de características (*Feature Models*) son considerados uno de los mecanismos más utilizados para representar la variabilidad dentro de las líneas de

productos software. Originalmente fueron introducidos como parte del método FODA (*Feature-Oriented Domain Analysis*) por Kang et al., con el objetivo de describir las características comunes y variables existentes dentro de una familia de sistemas software [13]. Benavides et al. los describen de la siguiente forma: "*a feature model represents the information of all possible products of a software product line in terms of features and relationships among them*"[14]. Estos modelos permiten representar el espacio de configuraciones de una línea de productos mediante las características que pueden formar parte de cada producto y las relaciones existentes entre ellas.

Para entenderlo mejor, estos modelos se pueden representar mediante una estructura jerárquica en forma de árbol, donde cada nodo representa una característica del sistema. Las relaciones existentes entre características permiten establecer dependencias entre ellas, indicando qué elementos son obligatorios, opcionales o alternativos dentro de una configuración concreta. Además, pueden incorporar restricciones adicionales que expresan condiciones que no pueden representarse únicamente mediante la jerarquía del modelo.

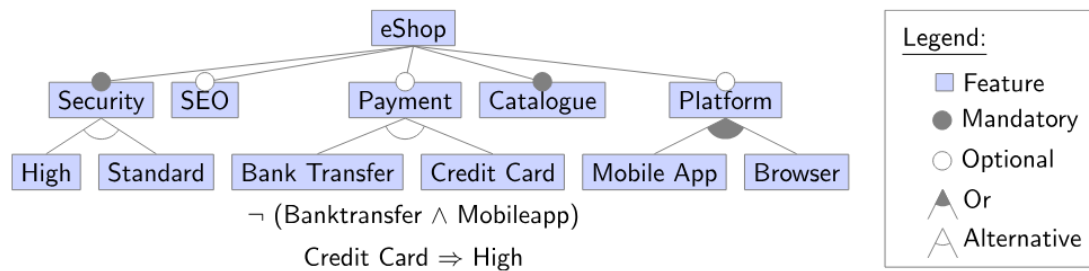


Figura 2.2: Ejemplo de modelo de características. Extraído de [15].

Aunque los modelos de características tradicionales permiten representar una gran cantidad de escenarios de variabilidad, inicialmente estaban orientados principalmente a expresar relaciones estructurales entre características. Con el aumento de la complejidad de los sistemas software surgió la necesidad de incorporar mecanismos adicionales, como atributos, restricciones aritméticas o diferentes tipos de dominios, que permitan representar escenarios más cercanos a los sistemas reales. A su vez, al incrementar la expresividad de los modelos, también se incrementa la complejidad de los mismos, lo cual hace necesaria la existencia de herramientas capaces de procesar estos modelos automáticamente y verificar propiedades sobre las configuraciones posibles.

2.1.3 Análisis automático y motores de razonamiento

Una vez definidos los modelos de características como mecanismo para representar la variabilidad de una SPL, surge la necesidad de disponer de técnicas capaces de analizar automáticamente la información contenida en dichos modelos. El análisis

automático permite obtener información sobre las configuraciones representadas, comprobar propiedades del modelo y realizar operaciones de razonamiento sin necesidad de explorar manualmente cada una de las posibles variantes.

Los motores de razonamiento (*reasoning engine*) se encargan de realizar este análisis. Procesan los modelos de variabilidad y realizan operaciones como la comprobación de satisfacibilidad, la detección de inconsistencias o la identificación de características que no pueden formar parte de ninguna configuración válida. Por ejemplo, en un restaurante un cliente puede elegir en su primer plato Pasta o Arroz (obligatoriamente uno de ellos), y opcionalmente Salsa. No se puede establecer como restricción que se deba elegir Pasta y Arroz a la vez, ya que no tendría sentido. Los motores de razonamiento se encargan de comprobar que esto no ocurra, y lo categorizarían como no satisfacible.

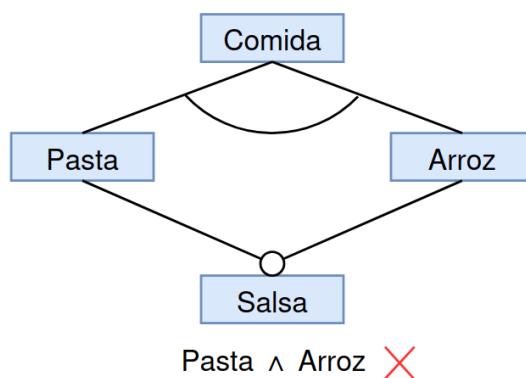


Figura 2.3: Ejemplo de modelo de características. Elaboración propia.

Una de las aproximaciones más utilizadas para realizar análisis automático sobre modelos de características consiste en transformar dichos modelos en fórmulas proposicionales que puedan ser procesadas mediante solucionadores SAT (*Boolean Satisfiability Problem*). El problema SAT consiste en determinar si existe una asignación de valores para un conjunto de variables booleanas que haga verdadera una fórmula lógica dada. En el contexto de las líneas de productos software, las características y restricciones de un modelo pueden representarse mediante variables booleanas y expresiones lógicas equivalentes, permitiendo utilizar solucionadores SAT para responder consultas relacionadas con la validez de configuraciones o la existencia de productos derivados [14].

Para aplicar esta técnica, el modelo se traduce normalmente a una fórmula lógica equivalente que conserva las restricciones definidas entre sus elementos. Por ejemplo, una relación obligatoria entre dos características puede expresarse mediante una implicación lógica, mientras que las restricciones de exclusión entre características pueden representarse mediante cláusulas que impidan determinadas combinaciones. Una vez realizada esta transformación, el solucionador SAT puede determinar si

existe una configuración válida o identificar aquellas asignaciones que cumplen las restricciones del modelo.

Otra técnica ampliamente utilizada en el análisis automático de modelos de características son los Diagramas de Decisión Binarios (*Binary Decision Diagrams*, BDD). Introducidos originalmente por Bryant como una estructura eficiente para representar funciones booleanas, los BDD permiten representar dichas funciones mediante grafos dirigidos acíclicos cuyos nodos internos corresponden a variables de decisión y cuyos nodos terminales representan los valores posibles de la función [16]. Así es posible modelar de forma compacta y comprensible el espacio de configuraciones válidas de un modelo de características.

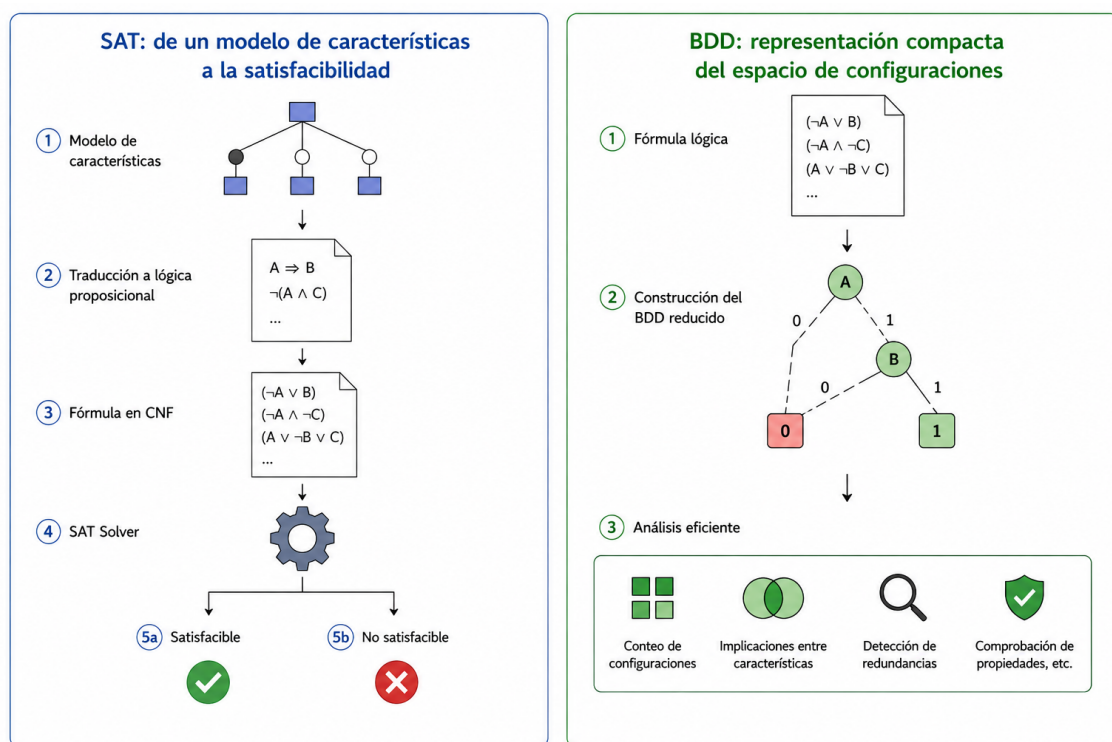


Figura 2.4: Funcionamiento de cada motor de razonamiento. Elaboración propia.

La siguiente tabla muestra una comparativa que describe los aspectos principales de ambos *solvers*:

Criterio	SAT	BDD
Representación del problema	Transformación del modelo de características a una fórmula lógica que debe ser evaluada por un solucionador SAT.	Construcción de una estructura gráfica que representa las posibles asignaciones de la función booleana.
Tipo de análisis	Orientado principalmente a responder consultas concretas sobre la existencia de configuraciones válidas.	Permite realizar análisis globales sobre el espacio completo de configuraciones representado.
Resultado obtenido	Determina si existe una asignación que satisface las restricciones y, en algunos casos, proporciona una configuración válida.	Proporciona una representación compacta del conjunto de configuraciones válidas.
Ventaja principal	Gran eficiencia para comprobar propiedades concretas del modelo y resolver problemas de satisfacibilidad.	Permite realizar múltiples operaciones de análisis sobre el modelo una vez construido el BDD.
Limitación principal	Requiere resolver consultas individuales y no mantiene necesariamente una representación completa del espacio de configuraciones.	La construcción del diagrama puede resultar costosa para modelos con elevada complejidad.

Tabla 2.1: Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.

2.2 Universal Variability Language (UVL)

A continuación, se ofrecerán unas nociones básicas sobre UVL [3], un lenguaje diseñado para proporcionar una forma común de representar modelos de características y facilitar su procesamiento dentro del ecosistema de ingeniería de líneas de productos software. Entender la estructura y funcionamiento de este lenguaje es esencial, ya que supone la base teórica por la que funciona la solución propuesta.

El modelo completo utilizado como ejemplo se incluye en el Apéndice A.

2.2.1 Estructura básica de un modelo UVL

Un archivo UVL está compuesto principalmente por dos bloques:

- **features**: define la estructura jerárquica de las características del modelo, constituyendo el núcleo principal del documento.
- **constraints**: define las restricciones lógicas existentes entre características que no pueden expresarse mediante la estructura definida en **features**. Estas restricciones deben cumplirse para garantizar la consistencia del modelo descrito, ya sea debido a reglas de negocio del producto o a otros requisitos establecidos.

Además, UVL permite incorporar dos bloques opcionales en aquellos casos donde sea necesario:

- **include**: permite importar extensiones adicionales, como los *minor levels*.
- **import**: permite importar submodelos de características situados en el mismo directorio, facilitando una organización más modular y limpia de los modelos y sus características.

Estos dos últimos bloques deben situarse al principio del archivo UVL.

2.2.2 Sintaxis

2.2.2.1. Imports

El mecanismo de importación de UVL permite dividir un modelo de características de gran tamaño en submodelos más pequeños y manejables. Mediante la palabra reservada **imports**, es posible incorporar otros archivos UVL, facilitando la organización de los modelos y favoreciendo su reutilización. Además, cada submodelo importado puede asociarse a un alias mediante la palabra reservada **as**, permitiendo acceder a sus características mediante la notación **alias.FeatureName**.

Los elementos importados pueden utilizarse tanto dentro de la estructura jerárquica de características como en la definición de restricciones, permitiendo mantener los modelos más limpios, modulares y facilitando su mantenimiento y evolución.

Por ejemplo:

```
1 imports
2     platform as pl
3     security
4
5 features
6     eShop
```

```

7      mandatory
8          security.Security
9      optional
10     pl.Platform

```

Código 2.1: Ejemplo de importación de submodelos en UVL.

En este caso, los modelos **platform** y **security** representan submodelos definidos en archivos UVL independientes situados en el mismo directorio.

2.2.2.2. Feature tree

En UVL, el árbol de características está compuesto por características y grupos de características. Cada modelo debe tener exactamente una característica raíz, a partir de la cual se define la estructura jerárquica del resto de elementos. Cada característica puede contener uno o varios grupos de características asociados.

La relación entre características y grupos se expresa mediante indentación: los grupos se escriben con respecto a su característica padre, mientras que las características se indentan respecto a su grupo correspondiente. Cada característica debe tener un nombre único que permita identificarla dentro del modelo, pudiendo utilizar comillas cuando contiene símbolos especiales o cuando su nombre está formado por varias palabras.

UVL define diferentes tipos de grupos para expresar las relaciones entre características, incluyendo **mandatory**, **optional**, **or** y **alternative**, pertenecientes al nivel booleano del lenguaje. También incorpora las cardinalidades de grupo (*group cardinality*), que permiten expresar restricciones más avanzadas sobre la selección de características.

Como detalle adicional, cuando una característica lleva mas de una palabra consigo, se suele rodear de comillas dobles `"`.

2.2.3 Diseño del lenguaje

UVL define modelos de variabilidad mediante una estructura jerárquica basada en árboles, donde cada nodo representa una característica y las relaciones entre dichos nodos permiten establecer las reglas de selección entre ellas. El núcleo del lenguaje se basa en diferentes construcciones que permiten especificar cómo pueden o deben seleccionarse las características dentro de un modelo.

2.2.3.1. mandatory

Una característica obligatoria debe ser seleccionada siempre que su característica padre forme parte de una configuración válida. Por ejemplo, en el siguiente modelo,

si se selecciona la característica *Sensors*, también debe seleccionarse obligatoriamente *Heart Rate Monitor*.

```

1 Sensors
2     mandatory
3     "Heart Rate Monitor"

```

Código 2.2: Ejemplo de característica obligatoria en UVL.

En este caso, la inclusión de *Sensors* implica necesariamente la selección de *Heart Rate Monitor*.

2.2.3.2. optional

Una característica opcional puede ser seleccionada o no dependiendo de las necesidades de una configuración concreta. Por ejemplo:

```

1 Sensors
2     optional
3     GPS
4     "Blood Oxygen Sensor" {Accuracy 95}
5     Accelerometer
6     Gyroscope

```

Código 2.3: Ejemplo de características opcionales en UVL.

En este caso, si la característica *Sensors* es seleccionada, el usuario puede decidir si incorporar o no características como *GPS*, *Blood Oxygen Sensor*, *Accelerometer* o *Gyroscope*.

2.2.3.3. alternative

Un grupo alternativo indica que, si la característica padre es seleccionada, exactamente una de las características hijas debe formar parte de la configuración. Por ejemplo:

```

1 Display
2     alternative
3     LCD
4     OLED
5     "E-Ink"

```

Código 2.4: Ejemplo de grupo alternativo en UVL.

En este caso, al seleccionar *Display*, debe escogerse exactamente una de las alternativas disponibles: *LCD*, *OLED* o *E-Ink*.

2.2.3.4. or

Un grupo **or** permite seleccionar una o más características hijas cuando su característica padre está incluida en una configuración. Por ejemplo:

```

1 Connectivity
2     or
3         Bluetooth
4         "Wi-Fi"
5         LTE

```

Código 2.5: Ejemplo de grupo OR en UVL.

En este caso, pueden seleccionarse varias opciones simultáneamente, siempre que al menos una de ellas sea seleccionada.

2.2.3.5. Restricciones entre características (*Cross-tree Constraints*)

Además de las relaciones jerárquicas definidas mediante la estructura del árbol, UVL permite establecer restricciones lógicas entre características situadas en diferentes ramas del modelo. Estas restricciones, denominadas *cross-tree constraints*, permiten expresar condiciones que no pueden representarse únicamente mediante las relaciones padre-hijo.

Estas restricciones se definen dentro del bloque **constraints** del archivo UVL y permiten utilizar diferentes operadores lógicos:

- Negación mediante **!A**: expresa que una característica no puede estar seleccionada.
- Conjunción mediante **A & B**: indica que ambas características deben estar seleccionadas.
- Disyunción mediante **A | B**: indica que al menos una de las características debe estar seleccionada.
- Implicación mediante **A =>B**: indica que si una característica A está seleccionada, entonces la característica B también debe estarlo.
- Equivalencia mediante **A <=>B**: indica que ambas características deben seleccionarse conjuntamente o ninguna de ellas debe seleccionarse.

Por ejemplo, la siguiente restricción establece que si se selecciona la característica *Fall Detection*, también deben seleccionarse *Accelerometer* y *Gyroscope*:

```
1 constraints
2   "Fall Detection" => (Accelerometer & Gyroscope)
```

Código 2.6: Ejemplo de restricción lógica en UVL.

Otro ejemplo sería:

```
1 constraints
2   "Voice Assistant" <=> "Wi-Fi"
```

Código 2.7: Ejemplo de restricción de equivalencia en UVL.

En este caso, ambas características deben aparecer juntas en una configuración válida, de forma que la selección de una de ellas implica necesariamente la selección de la otra.

2.2.4 Niveles del lenguaje

En UVL, los niveles del lenguaje permiten separar la expresividad del mismo en diferentes capas. Esta organización permite que usuarios y herramientas puedan decidir qué grado de complejidad desean soportar, además de facilitar la extensión del lenguaje sin incrementar la complejidad para aquellos usuarios que únicamente necesitan modelos sencillos.

Un usuario o herramienta que requiera modelos UVL básicos puede trabajar únicamente con el nivel booleano. Sin embargo, cuando sea necesario representar escenarios más complejos, como atributos numéricos, valores textuales u operaciones sobre ellos, pueden activarse niveles superiores.

UVL define tres niveles jerárquicos de expresividad:

- **Boolean level:** incluye características booleanas y restricciones lógicas.
- **Arithmetic level:** incorpora atributos numéricos y operaciones aritméticas.
- **Type level:** permite trabajar con atributos de diferentes tipos, como valores numéricos o textuales, así como operaciones asociadas a dichos tipos.

Cada uno de estos niveles incluye completamente las capacidades proporcionadas por los niveles anteriores. Además, UVL incorpora niveles menores (*minor levels*), que representan extensiones opcionales que añaden funcionalidades adicionales sin ser necesarias para avanzar entre los diferentes niveles principales.

2.2.4.1. Boolean level

El nivel booleano constituye el nivel principal más básico y representa el núcleo del lenguaje UVL. En él se define la estructura del árbol de características, las

relaciones entre ellas (**mandatory**, **optional**, **alternative** y **or**) y las restricciones lógicas básicas entre características mediante operadores como **!**, **&**, **|**, **=>** y **<=>**.

Por ejemplo:

```

1 features
2     Smartwatch
3         optional
4             GPS
5             "Water Resistance"
6             Accessories
7                 [1..2]
8                     "Leather Strap"
9                     "Metal Strap"
10                    "Silicone Strap"
11
12 constraints
13     Wi-Fi || Bluetooth
14     "Water Resistance" => Display

```

Código 2.8: Ejemplo de modelo UVL utilizando el nivel booleano.

En este nivel únicamente se representa la presencia o ausencia de características dentro de una configuración.

2.2.4.2. Group Cardinality

La cardinalidad de grupo (*Group Cardinality*) es un nivel menor perteneciente al nivel booleano. Permite establecer que, dentro de un grupo de características, únicamente pueda seleccionarse un número determinado de elementos comprendido entre un límite inferior y superior.

En el ejemplo anterior, la expresión:

```

1 [1..2]

```

Código 2.9: Ejemplo de cardinalidad de grupo en UVL.

indica que pueden seleccionarse entre una y dos características del grupo *Accessories*.

2.2.4.3. Arithmetic level

El nivel aritmético incluye todas las capacidades del nivel booleano y permite definir atributos numéricos asociados a características, así como restricciones basadas en operaciones aritméticas mediante operadores como **+**, **-**, *****, **/**, **=**, **<**, **>**, **<=** y **>=**.

Además, este nivel incorpora dos niveles menores: *Feature Cardinality* y *Aggregate Functions*.

Por ejemplo:

```

1 include
2     Arithmetic.feature-cardinality
3     Arithmetic.aggregate-function
4
5 features
6     Smartwatch
7         mandatory
8             Connectivity
9             or
10                Bluetooth
11                Wi-Fi
12            Battery
13                alternative
14                    Standard {Capacity 300}
15                    "Long Life" {Capacity 450}
16        optional
17            "Music Player" {Storage 2}
18            "Film Player" {Storage 10}
19            "Watch Faces" cardinality [5..20]
20
21 constraints
22     ("Music Player".Storage + "Film Player".Storage) = 12
23     avg(Capacity) >= 300

```

Código 2.10: Ejemplo de atributos y restricciones aritméticas en UVL.

2.2.4.4. Feature Cardinality

La cardinalidad de característica (*Feature Cardinality*) es un nivel menor del nivel aritmético que permite indicar un número de instancias que pueden seleccionarse de dicha característica.

Para utilizar esta funcionalidad debe incluirse explícitamente mediante:

```

1 Arithmetic.feature-cardinality

```

En el siguiente ejemplo se puede ver la forma en la que se emplea:

```

1 include
2     Arithmetic.feature-cardinality
3

```

```

4 features
5     Smartwatch
6     ...
7     optional
8         "Watch Faces" cardinality [5..20]
9
10 constraints
11     ...

```

Código 2.11: Ejemplo de atributos y restricciones aritméticas en UVL.

2.2.4.5. Aggregate Functions

El nivel menor denominado *Aggregate Functions* permite utilizar funciones agregadas como `sum()` y `avg()` dentro del bloque de restricciones.

Por ejemplo:

```

1 include
2     Arithmetic.aggregate-function
3
4 features
5     ...
6
7 constraints
8     sum(PriceEuros) > 200
9     avg(Calification) > 5

```

En este caso, la restricción definida mediante la función `sum()` establece que la suma total de las veces que aparece el atributo `Capacity` en el árbol debe ser mayor que 400. La función `avg()` funciona de forma similar: devuelve la media de sus argumentos. Asimismo, es posible usar más de un atributo o *feature* en la misma llamada a la función, separadas por coma.

2.2.4.6. Type level

El nivel de tipos incluye las capacidades proporcionadas por los niveles booleano y aritmético, incorporando además atributos con tipos específicos como `Integer`, `Float` y `String`. Gracias a ello, es posible asociar valores concretos a las características y establecer restricciones sobre dichos valores.

Este nivel también incorpora el nivel menor *String Constraints*, que permite realizar operaciones sobre atributos de tipo cadena mediante funciones como `len()`.

Por ejemplo:

```
1 include
2     Type.string-constraints
3
4 features
5     Smartwatch
6         mandatory
7             String Color
8             Float Weight
9             String "Serial Number"
10
11 constraints
12     len("Serial Number") == 10
13     Weight > 30.0
14     Color != "Pink"
```

Código 2.12: Ejemplo de atributos tipados en UVL.

En este ejemplo, se definen atributos de tipo cadena y decimal asociados a características del modelo. La primera restricción establece que el atributo *Serial Number* debe tener exactamente diez caracteres, mientras que las siguientes restricciones permiten comparar valores numéricos y textuales.

2.3 Ecosistema de herramientas y aplicación en variabilidad software

A continuación se procederá a hablar sobre herramientas y entornos que permitan trabajar con los modelos de características más allá de su simple representación, las cuales proporcionan capacidades para la creación, gestión y análisis automático de modelos, integrando diferentes técnicas de razonamiento y facilitando su adaptación a nuevos escenarios. Dentro de este ecosistema destaca Flamapy como framework orientado al análisis automático de modelos de características mediante una arquitectura extensible. Asimismo, destaca UVLHub como plataforma destinada a la gestión y compartición de modelos definidos en UVL.

2.3.1 Flamapy como framework de análisis de variabilidad

Flamapy es un framework desarrollado en Python orientado al análisis automático de modelos de características dentro del ámbito de la ingeniería de líneas de productos software [4]. Su objetivo principal es proporcionar un entorno común para trabajar con modelos de variabilidad, facilitando tanto su procesamiento como la aplicación de diferentes técnicas de análisis automático. Para ello, Flamapy puede entenderse

desde dos perspectivas complementarias: como una herramienta que permite ejecutar análisis sobre modelos de características de forma directa, y como un framework extensible que proporciona una arquitectura modular para incorporar nuevos modelos, operaciones y técnicas de análisis.

Desde la perspectiva de herramienta, Flamapy proporciona diferentes mecanismos para facilitar su utilización en distintos escenarios. Entre ellos se encuentra una interfaz de línea de comandos (*Command Line Interface*) que permite ejecutar análisis directamente sobre modelos de variabilidad, así como una fachada de Python (*Python Facade*) que simplifica su integración dentro de otros desarrollos software. Además, ofrece interfaces adicionales como servicios REST mediante Swagger y soporte para ejecución en entornos web mediante WebAssembly (WASM), permitiendo utilizar sus capacidades de análisis desde diferentes aplicaciones y contextos.

Como framework, Flamapy sigue una arquitectura modular basada en plugins, los cuales permiten extender sus capacidades sin modificar el núcleo principal del sistema. Esta arquitectura hace mucho más sencilla la incorporación de nuevos lenguajes de modelado, metamodelos o técnicas de análisis mediante componentes independientes que pueden integrarse dentro del ecosistema existente. Gracias a este enfoque, es posible adaptar Flamapy a diferentes necesidades relacionadas con la ingeniería de variabilidad, incorporando soporte para nuevos tipos de modelos o herramientas de razonamiento.

Otro de los elementos principales que ofrece Flamapy son las operaciones (*operations*), que representan las diferentes tareas de análisis que pueden ejecutarse sobre modelos de características. Cada operación define una funcionalidad concreta, como la comprobación de satisfacibilidad, la obtención de configuraciones válidas o el cálculo de propiedades del modelo, independientemente del formato en el que este haya sido definido. Esta separación permite reutilizar las mismas operaciones sobre diferentes tipos de modelos siempre que exista una representación compatible dentro del framework.

Por otra parte, las transformaciones (*transformations*) permiten convertir modelos entre diferentes representaciones dentro de Flamapy. Estas transformaciones actúan como mecanismo de conexión entre los distintos metamodelos soportados, permitiendo que los modelos puedan ser procesados por diferentes componentes del framework (o incluso no del framework) independientemente de su representación original. Por ejemplo, el componente *UVLWriter*, que es el que se ha utilizado para descargar los modelos creados por el generador diseñado, permite transformar un modelo de características almacenado internamente en Flamapy a su representación textual en formato UVL. Otras de las transformaciones existentes, por nombrar algunas, son *AFM Reader*, *AFM Writer*, *XML Reader*, *JSON Writer*, *SPLIT Writer* o

FM to BDD. Todo esto facilita la interoperabilidad entre lenguajes de variabilidad, metamodelos y herramientas de análisis.

2.3.2 UVLHub como plataforma de gestión de modelos UVL

UVLHub es una plataforma orientada a la gestión de modelos de características definidos mediante el lenguaje UVL (*Universal Variability Language*) dentro del ámbito de la ingeniería de líneas de productos software [6]. Tiene como objetivo proporcionar un repositorio especializado que permita almacenar, organizar, compartir y analizar modelos de variabilidad, facilitando tanto la reutilización de estos artefactos tanto en entornos de investigación como de desarrollo software como la colaboración entre usuarios.

La razón principal de su existencia se debe a la complicada localización de los modelos de características empleados por los usuarios, ya que estos los almacenaban en diferentes repositorios, páginas personales o proyectos independientes, utilizando además diferentes formatos de representación, dificultando su búsqueda, reutilización y análisis. UVLHub sigue los principios de ciencia abierta (*Open Science*), como accesibilidad, transparencia y colaboración, con la finalidad de compartir estos modelos de forma homogénea, haciendo más sencilla la colaboración entre usuarios.

Una de las características más relevantes de UVLHub es su capacidad de integración con otras herramientas y servicios relacionados con la gestión y análisis de modelos de variabilidad. Por una parte, la plataforma se integra con Zenodo, un repositorio abierto de investigación que permite almacenar los conjuntos de datos de forma permanente mediante la generación de identificadores DOI (*Digital Object Identifier*), garantizando que los modelos publicados puedan seguir estando disponibles aunque la plataforma dejase de mantenerse en el futuro. Por otra parte, UVLHub incorpora las capacidades de análisis automático proporcionadas por Flamapy, permitiendo obtener información relevante sobre los modelos almacenados, como el número de características, la validez del modelo o el número de configuraciones posibles que pueden derivarse de ellos.

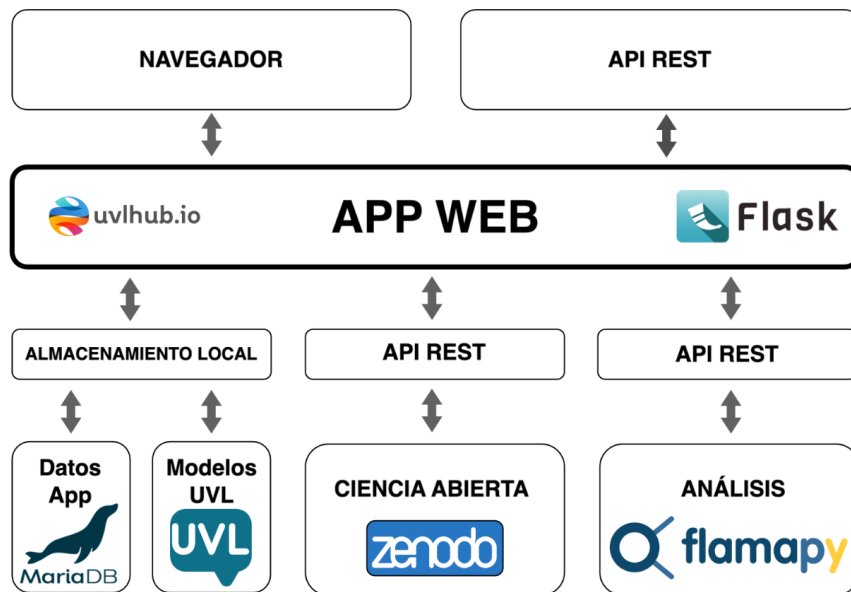


Figura 2.5: Arquitectura general de UVLHub y sus integraciones principales.
Extraído de [17]

Además de sus integraciones externas, UVLHub proporciona diferentes funcionalidades (*features*) en forma de módulos orientadas a cubrir las principales necesidades relacionadas con la gestión de modelos UVL. Algunas de las más relevantes son:

- **dataset:** representa el núcleo principal de UVLHub, permitiendo gestionar el ciclo de vida de los conjuntos de modelos UVL. Esta funcionalidad permite realizar operaciones como la subida de datasets, su almacenamiento junto con los metadatos asociados, la descarga de los modelos y la resolución de identificadores DOI generados mediante la integración con Zenodo.
- **featuremodel:** permite gestionar la información asociada a los modelos de características almacenados dentro de un dataset. Esta *feature* define elementos como el modelo de características, sus metadatos y las métricas asociadas, estableciendo la relación entre los datasets y los archivos UVL almacenados.
- **explore:** proporciona mecanismos para explorar los datasets disponibles dentro de la plataforma mediante diferentes criterios de búsqueda, permitiendo localizar modelos de características publicados previamente.
- **flamapy:** integra las capacidades de análisis automático proporcionadas por Flamapy dentro de UVLHub. Esta integración permite realizar operaciones sobre los modelos UVL, como la validación del modelo o transformaciones hacia otros formatos soportados.

- **zenodo**: permite la comunicación con la API REST de Zenodo para realizar la publicación y almacenamiento permanente de los datasets, proporcionando identificadores DOI asociados a los recursos compartidos.

La siguiente imagen representa, a grandes rasgos, los módulos actuales de UVLHub y cómo se complementan entre sí:

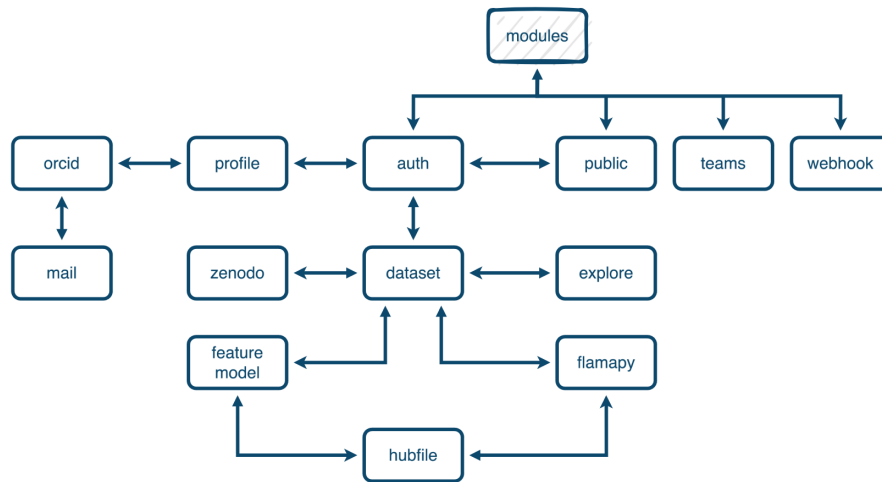


Figura 2.6: Arquitectura general de UVLHub y sus integraciones principales.
Extraído de [17]

Estos módulos son originales de UVLHub. A ellos hay que añadirle también el módulo **generator**, objetivo principal de este TFG.

2.3.3 Generación automática de modelos de características

Ya es bien sabido que los modelos de características permiten representar la variabilidad existente dentro de una línea de productos software. Sin embargo, a medida que aumenta el número de características, restricciones y posibles configuraciones, la creación manual de estos modelos puede convertirse en una tarea costosa y difícil de gestionar.

Debido a esto, surge la necesidad de disponer de mecanismos capaces de generar modelos de características de forma automática. Se ha comprobado que la generación automática es una vía bastante eficaz, ya que permite crear modelos de variabilidad siguiendo determinados parámetros o condiciones definidas previamente, reduciendo el esfuerzo necesario para construirlos y facilitando su utilización en tareas como la evaluación de herramientas de análisis o la realización de experimentos sobre diferentes escenarios de complejidad. Esta capacidad complementa las funcionalidades proporcionadas por lenguajes como UVL y frameworks de análisis como Flamapy, ya que permite no solo almacenar y analizar modelos existentes, sino también generar nuevos modelos que puedan ser posteriormente procesados dentro de estas herramientas.

2.4 Trabajos relacionados

[Sección clave del estado del arte. Aquí se comparan soluciones, proyectos o publicaciones que abordan problemas similares al que resuelve el TFG.]

[Empieza con un párrafo que sustente la exhaustividad de la búsqueda: indica qué bases de datos consultaste (Scopus, Web of Science, Google Scholar, IEEE Xplore...), con qué palabras clave y qué criterios de inclusión y exclusión aplicaste (p. ej. publicaciones de los últimos cinco años, revisadas por pares, con herramienta disponible públicamente). Un párrafo basta; no hace falta una revisión sistemática formal.]

[Plantilla de redacción:]

[En el ámbito de ..., destacan los trabajos de ..., que abordan Sin embargo, ninguno cubre ..., que es precisamente lo que motiva este trabajo.]

[En el ámbito de ..., los enfoques existentes se centran en La diferencia con la propuesta de este TFG es]

Dentro del ámbito de la ingeniería de líneas de productos software existen diferentes propuestas orientadas a la generación automática de modelos de características. Estas aproximaciones han sido utilizadas principalmente para crear modelos sintéticos destinados a la evaluación de herramientas de análisis de variabilidad, la realización de experimentos controlados o el estudio de la complejidad de diferentes técnicas de razonamiento. Para analizar el estado actual de estas soluciones, se ha tomado como punto de partida el apartado *Related Work* del paper proporcionado por el tutor de este TFG para el desarrollo de un nuevo generador de modelos de características basado en UVL. En dicho apartado se identifican diferentes propuestas existentes dentro de este ámbito, entre las que se encuentran S.P.L.O.T., FeatureIDE, BeTTy y ETHOM. A partir de esta información, se han realizado búsquedas individuales de cada generador, localizando las páginas web donde se encuentran disponibles sus respectivas publicaciones científicas, como ResearchGate para los trabajos asociados a BeTTy y S.P.L.O.T., ScienceDirect para FeatureIDE y ETHOM, así como las páginas oficiales de los propios proyectos cuando estas proporcionaban información adicional. A partir de estas fuentes se ha extraído la información necesaria para analizar las principales características de cada propuesta, prestando especial atención a sus capacidades de personalización, la expresividad de los modelos generados y su integración dentro de otros ecosistemas de variabilidad software. Como adelanto, se informa de que ninguno de los generadores existentes tienen la capacidad avanzada de expresividad de este generador, lo que lo diferencia claramente del resto.

2.4.1 S.P.L.O.T.

S.P.L.O.T. (*Software Product Lines Online Tools*) es una plataforma web orientada al desarrollo y análisis de líneas de productos software, proporcionando diferentes herramientas relacionadas con la gestión de modelos de características [18]. Entre sus funcionalidades se encuentra un generador automático de modelos de características, diseñado para crear modelos sintéticos que pueden ser utilizados posteriormente en tareas de experimentación y evaluación de técnicas de análisis de variabilidad.

Aunque S.P.L.O.T. supuso una de las primeras aproximaciones ampliamente utilizadas para la generación automática de modelos de características, sus capacidades están principalmente orientadas a construcciones pertenecientes al nivel booleano del lenguaje de variabilidad. Por tanto, permite representar estructuras jerárquicas y restricciones lógicas básicas entre características, pero no contempla elementos más avanzados como atributos numéricos, operaciones aritméticas o restricciones basadas en tipos.

2.4.2 FeatureIDE

FeatureIDE es un framework extensible orientado al desarrollo software basado en características que proporciona soporte para diferentes actividades relacionadas con la ingeniería de líneas de productos software [19]. La plataforma permite crear, editar y analizar modelos de características, integrando diferentes herramientas destinadas a facilitar el desarrollo de sistemas configurables.

Además de sus capacidades de gestión y análisis, FeatureIDE incorpora un generador automático de modelos de características que permite crear modelos sintéticos dentro de su propio ecosistema. Sin embargo, al igual que otras propuestas existentes, su generación está principalmente limitada a construcciones pertenecientes al nivel booleano de los lenguajes de variabilidad, por lo que no permite generar modelos que incorporen toda la expresividad proporcionada por lenguajes más recientes como UVL.

2.4.2.1. BeTTy

BeTTy (*Benchmarking and Testing on the Automated Analysis of Feature Models*) es un framework orientado al benchmarking y evaluación de herramientas destinadas al análisis automático de modelos de características [20]. A diferencia de aproximaciones más simples, BeTTy permite configurar diferentes parámetros relacionados con la generación de modelos, facilitando la creación de escenarios experimentales adaptados a las necesidades de evaluación.

Una de sus principales aportaciones es la capacidad de generar modelos con diferentes niveles de complejidad estructural y restricciones, permitiendo estudiar el

comportamiento de diferentes técnicas de razonamiento sobre escenarios controlados [20]. Sin embargo, aunque proporciona una mayor flexibilidad respecto a generadores anteriores, sus capacidades siguen centradas principalmente en el nivel booleano del lenguaje de variabilidad. Por ello, permite aumentar la dificultad de análisis de los modelos generados, pero no ampliar su expresividad mediante construcciones como atributos o restricciones aritméticas.

2.4.3 Comparativa de trabajos relacionados

[Resume la comparación en una tabla: una fila por trabajo (citado con \cite) y 3–4 criterios como columnas, derivados del problema que resuelve tu TFG (no genéricos). La última fila es tu propuesta: así el lector ve de un vistazo el hueco que cubre el trabajo.]

[Leyenda: Sí = lo cubre completamente; Parcial = lo cubre con limitaciones; No = no lo cubre. Comenta la tabla de forma crítica, no descriptiva: justifica cada «Parcial» y cada «No», identifica el hueco que dejan los trabajos existentes y cierra con una frase que enuncie la contribución diferencial de este TFG. Sé honesto: si tu propuesta también tiene algún «Parcial», reconocerlo refuerza la credibilidad del análisis. Los criterios (columnas) elegidos deberán poder respaldarse con evidencia propia en el Capítulo 8. El Capítulo 8 retomará esta tabla para contrastarla con los resultados obtenidos.]

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Identificado el hueco que dejan los trabajos existentes, el Capítulo 3 describe cómo se ha organizado el proyecto para construir la solución que lo cubre: metodología, fases, recursos y gestión de riesgos».]

Trabajo	Generación automática	Personalización	Expresividad avanzada
S.P.L.O.T. [18]	Sí	Parcial	No
FeatureIDE [19]	Sí	Parcial	No
BeTTy [20]	Sí	Sí	No
Este TFG	Sí	Sí	Sí

Tabla 2.2: Comparativa de trabajos relacionados con la generación automática de modelos de características.

3 PLANIFICACIÓN DEL PROYECTO

[Este capítulo demuestra que el TFG no es solo un producto software, sino un proyecto gestionado de principio a fin: se planifica al inicio (acta de constitución, interesados, metodología como decisión de planificación, EDT, cronograma como línea base y presupuesto como línea base de coste), se controla durante la ejecución (comunicaciones, riesgos, control de cambios, calidad) y se cierra formalmente con el análisis de desviaciones —de calendario, esfuerzo y coste— y las lecciones aprendidas. Trata el plan como un documento vivo: la estimación inicial no se borra cuando la realidad la contradice, se compara con ella.]

3.1 Acta de constitución del proyecto

[Resume en media página el «contrato» inicial del proyecto: qué se va a hacer, bajo qué supuestos y con qué restricciones. No repitas la introducción: referencia los objetivos (Sección 1.4) y el alcance (Sección 1.5) y céntrate en lo que aún no está escrito (supuestos, restricciones, criterio de cierre).]

Elemento	Descripción
Propósito	Desarrollar una herramienta que permita generar automáticamente modelos de características en formato UVL para facilitar la creación de modelos de variabilidad software, proporcionando a investigadores y desarrolladores un mecanismo configurable para definir las características, restricciones y atributos de los modelos generados.
Objetivos	Los objetivos del proyecto son los definidos en la Sección 1.4. De forma general, el proyecto busca diseñar e implementar un generador automático de modelos de características, integrarlo en UVLHub y proporcionar soporte para diferentes niveles de expresividad del lenguaje UVL.
Alcance	El proyecto comprende el diseño e implementación del generador automático de modelos de características, su integración dentro de la plataforma UVLHub y la incorporación progresiva de soporte para modelos booleanos, aritméticos y con tipos definidos por UVL. Quedan fuera del alcance la modificación del funcionamiento interno de Flamapy y el desarrollo de nuevos algoritmos de análisis o razonamiento.
Supuestos	Se asume la disponibilidad del ecosistema software necesario para el desarrollo, incluyendo acceso al código fuente de UVLHub y Flamapy, así como las herramientas necesarias para la generación y análisis de modelos de variabilidad. También se considera disponible el entorno de desarrollo necesario para implementar, probar y validar la solución.
Restricciones	El proyecto está condicionado por las tecnologías y arquitectura existentes en UVLHub y Flamapy, especialmente el lenguaje Python (su framework Flask) y el estándar UVL. Además, debe desarrollarse antes del 7 de octubre, la fecha límite para la entrega de la memoria.
Entregables principales	Los principales entregables del proyecto son el código fuente del generador automático integrado en UVLHub y la memoria del TFG.
Criterio de cierre	El proyecto se considerará finalizado cuando el generador permita crear modelos de características funcionales en los niveles de expresividad definidos, se encuentre integrado dentro de UVLHub y se hayan realizado pruebas que validen el correcto funcionamiento tanto de la generación de modelos como del front-end desarrollado para utilizarlo en UVLHub

Tabla 3.1: Acta de constitución del proyecto.

3.1.1 Interesados del proyecto

[Identifica a las personas con interés en el proyecto y qué espera cada una. En un TFG el listado es corto; lo importante es que ninguna fila quede sin expectativa concreta.]

Interesado	Rol	Interés / expectativa	Implicación
Autor del proyecto	Responsable del desarrollo	Completar el Trabajo Fin de Grado mediante el desarrollo de una solución funcional, documentada y alineada con los objetivos establecidos.	Diaria
Tutor del TFG	Supervisión y validación académica	Garantizar el correcto desarrollo del proyecto, proporcionando orientación técnica y revisando los avances y entregables realizados.	Periódica
Grupo de investigación / ecosistema UVLHub-Flamapy	Entorno receptor de la solución	Disponer de una herramienta que permita generar modelos de características de forma automática y ampliar las capacidades existentes del ecosistema de variabilidad software.	Puntual
Tribunal del TFG	Evaluación final del proyecto	Valorar la calidad de la memoria, la defensa realizada y los resultados obtenidos durante el desarrollo del trabajo.	Al cierre

Tabla 3.2: Registro de interesados del proyecto.

3.2 Metodología

3.2.1 Metodología de desarrollo

[Justifica la metodología (o metodologías) elegida(s) para el desarrollo y para la redacción de la memoria. Es habitual usar enfoques distintos para cada fase.]

[Puntos a cubrir:

- *[Metodología elegida (Waterfall, Scrum, Kanban, iterativa...) y por qué.]*

- *[Qué alternativas consideraste y por qué las descartaste (una frase por alternativa basta).]*
- *[Cómo se gestiona el control de versiones (Git, GitHub/GitLab...)].*
- *[Si se usan herramientas de seguimiento (issues, tableros, CI), nómbralas brevemente.]*

]

Para el desarrollo del proyecto se ha seguido una metodología incremental e iterativa, adaptada a la naturaleza evolutiva del generador automático de modelos de características. Debido al carácter investigador y evolutivo del proyecto, la especificación inicial se refinó progresivamente durante el desarrollo conforme se analizaban las necesidades del generador y su integración dentro del ecosistema existente. [7].

En una primera etapa se desarrolló una versión inicial del generador orientada a la creación de modelos de características booleanos, junto con su integración dentro de UVLHub mediante la interfaz gráfica proporcionada para configurar el proceso de generación. Una vez obtenida esta primera versión funcional, se realizaron ciclos sucesivos de implementación, validación y corrección que permitieron mejorar la estabilidad del sistema e incorporar nuevas funcionalidades pertinentes a los siguientes niveles de expresividad que se pretendió conseguir. Esta forma de hacerlo permitió validar cada nueva incorporación antes de continuar con las siguientes etapas y reducir el riesgo de introducir errores en funcionalidades previamente implementadas que se pudieran arrastrar a las siguientes.

Alternativas consideradas

No se optó por metodologías secuenciales como el modelo en cascada, ya que la naturaleza del proyecto requería realizar ajustes continuos conforme avanzaba el desarrollo y se identificaban nuevas necesidades relacionadas con la generación de modelos de características. Tampoco se aplicó una metodología Scrum completa, puesto que el desarrollo no se organizó mediante sprints, roles o hitos propios de este marco de trabajo.

Control de versiones

El control de versiones del proyecto se ha realizado mediante Git, utilizando GitHub como plataforma de almacenamiento y gestión del código fuente. El desarrollo se ha realizado sobre la rama principal del repositorio (`main`), manteniendo un historial de cambios organizado mediante commits atómicos.

Cada commit representa un cambio funcional o de mantenimiento claramente identificado, siguiendo el *Conventional Commits*. Esta práctica permite mantener un

historial comprensible de la evolución del proyecto y facilita la trazabilidad de las modificaciones realizadas durante el desarrollo.

Se han utilizado diferentes tipos de commits en función de la naturaleza del cambio realizado, como *feat* para la incorporación de nuevas funcionalidades, *fix* para la corrección de errores detectados durante la validación y *refactor* para modificaciones orientadas a mejorar la estructura interna del código sin alterar su comportamiento, entre otras.

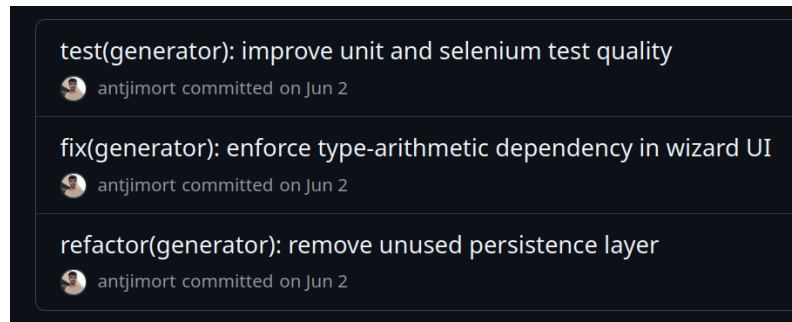


Figura 3.1: Algunos commits del proyecto. Elaboración propia.

Herramientas de seguimiento

El seguimiento del desarrollo se ha realizado principalmente mediante el propio historial de GitHub, utilizando los commits realizados como registro de la evolución del proyecto y de las decisiones técnicas tomadas durante la implementación.

Además, las reuniones periódicas con el tutor han permitido revisar el estado del desarrollo para analizar las deficiencias encontradas con el propósito de corregirlas y llevar la calidad del generador al siguiente nivel. Los detalles de estas reuniones se detallarán en el siguiente apartado.

3.2.2 Metodología de validación y evaluación

[Explica cómo se medirá que el proyecto cumple lo prometido. Normalmente combina dos vías: (a) pruebas y trazabilidad requisitos-pruebas (Capítulo 7), y (b) evaluación de resultados frente a los criterios de éxito de la Tabla 1.1, cuyo análisis se presenta en el Capítulo 8. Aquí basta con declarar el método; los datos van en sus capítulos.]

La validación del proyecto se ha realizado mediante un proceso iterativo durante el desarrollo, combinando pruebas funcionales, pruebas automatizadas y revisiones periódicas con el tutor del proyecto.

Tras la implementación de cada nueva funcionalidad, se realizaron comprobaciones manuales para verificar que el generador producía modelos de características correctos según los parámetros configurados, incluyendo los diferentes niveles de expresividad soportados por UVL, así como la correcta interacción del usuario con el front-end

desarrollado en UVLHub. Además, se ejecutaron pruebas unitarias automatizadas para detectar posibles errores o regresiones, incorporando nuevos casos de prueba únicamente cuando aportaban valor al sistema; es decir, que encontraran posibles bugs reales y relevantes.

La evaluación del cumplimiento del proyecto se ha realizado siguiendo los criterios de éxito definidos en la Tabla 1.1. Estos criterios permiten comprobar de forma objetiva aspectos como la generación automática de modelos UVL, la configuración del proceso de generación, el soporte de construcciones avanzadas del lenguaje, la integración dentro del ecosistema Flamapy y UVLHub, y la existencia de una batería de pruebas automatizadas.

Además de la validación técnica continua, se realizaron revisiones del estado del proyecto junto con el tutor. Cuando se alcanzó aproximadamente el 80 % de desarrollo, se realizó una primera revisión en la que se concluyó que la solución era funcionalmente válida, pero presentaba ciertas deficiencias técnicas que debían ser corregidas antes de considerarla finalizada. Entre ellas se identificaron problemas relacionados con la integración del generador dentro de UVLHub, la pérdida de algunas funcionalidades existentes durante determinadas modificaciones (que posteriormente se recuperaron sin problema), la reducción de la cobertura de pruebas, la presencia de elementos no deseados en el repositorio y aspectos de calidad del código (documentación y limpieza de la implementación).

Tras aplicar las correcciones necesarias, se realizó una segunda revisión con el tutor para comprobar el estado de la nueva versión. En esta ocasión únicamente se detectaron ajustes menores, que fueron solucionados posteriormente, obteniendo finalmente la validación positiva del tutor.

3.3 Estructura de descomposición del trabajo (EDT)

[Desglosa el proyecto en paquetes de trabajo con identificador jerárquico de dos niveles. La regla de oro: todo entregable debe poder marcarse como hecho/no hecho sin ambigüedad. Los IDs se reutilizan en el cronograma y en el cierre para dar trazabilidad. Con 6–10 paquetes es suficiente. Desglosa al segundo nivel solo donde aporte; los paquetes hoja deben corresponder a entregables.]

Este proyecto se podría desglosar de la siguiente manera:

ID	Paquete de trabajo	Entregable verificable
1	Gestión y planificación del proyecto	Planificación inicial del proyecto, estudio de la solución propuesta por los investigadores a partir del trabajo de referencia y seguimiento del desarrollo realizado.

ID	Paquete de trabajo	Entregable verificable
1.1	Planificación inicial y estudio de la solución	Acta de constitución, definición del alcance, planificación temporal del proyecto y análisis de la solución propuesta en el artículo de referencia.
1.2	Seguimiento y control	Reuniones de seguimiento, revisión del estado del desarrollo y gestión de los cambios realizados durante el proyecto.
2	Análisis y diseño de la solución	Diseño de la arquitectura del generador y definición de los componentes necesarios para su implementación.
2.1	Análisis de requisitos	Catálogo de requisitos funcionales y técnicos del generador automático de modelos.
2.2	Diseño del generador y su integración	Arquitectura definida para la generación de modelos, configuración del sistema e integración con UVLHub.
3	Desarrollo del generador automático	Implementación del generador capaz de producir modelos de características en formato UVL.
3.1	Implementación de niveles de expresividad	Soporte implementado para generación de modelos booleanos, aritméticos y con tipos definidos por UVL.
3.2	Desarrollo de configuración e interfaz	Sistema de configuración del generador e integración del flujo de interacción mediante el front-end de UVLHub.
4	Pruebas y validación	Suite de pruebas automatizadas y validación funcional del generador integrado.
5	Documentación y defensa	Memoria final del Trabajo Fin de Grado y material necesario para la presentación de defensa.

Tabla 3.3: Estructura de descomposición del trabajo (EDT).

3.4 Cronograma e hitos (línea base)

*[Divide el proyecto en fases claras con un hito (entregable verificable) por fase. La línea base tiene DOS dimensiones: el esfuerzo (columnas **Est.** y **Real**, en horas) y el calendario (columnas **Fin prev.** y **Fin real**, fechas de hito en formato mm/aa). Ambas se fijan al inicio y NO se reescriben cuando la realidad cambia: la estimación inicial se conserva y se compara con lo real, y los cambios significativos se registran en la Sección 3.7. La columna **EDT** indica qué paquetes de la Tabla 3.3 cubre cada*

fase, y la columna **Obj.** qué objetivo (O1–O5, Sección 1.4) ataca: un objetivo sin fase es un objetivo sin plan. Ningún paquete de primer nivel de la EDT puede quedar sin fase: el 1.2 es transversal a todas y el 5 se planifica en la tabla de la memoria.]

[Tabla de fases del desarrollo: rellena las filas con tus propias fases.]

# Fase	EDT	Obj.	Est.	Real	Fin prev.	Fin real	Hito / Entregable
1 [Fase 1]	[1.1]	[O1]	[X h]	[Y h]	[mm/aa]	[mm/aa]	[Entregable de la fase 1.]
2 [Fase 2]	[2.1–2.2]	[O2]	[X h]	[Y h]	[mm/aa]	[mm/aa]	[Entregable de la fase 2.]
3 [Fase 3]	[3]	[O3]	[X h]	[Y h]	[mm/aa]	[mm/aa]	[Entregable de la fase 3.]
4 [Fase 4]	[4]	[O4]	[X h]	[Y h]	[mm/aa]	[mm/aa]	[Entregable de la fase 4.]
Total			[X h]	[Y h]			

Tabla 3.4: Fases del desarrollo e hitos asociados (línea base de esfuerzo y calendario vs. valores reales).

Teniendo como fecha de inicio del proyecto el 12 de abril del 2025, obtenemos la siguiente tabla:

#	Fase	EDT	Obj.	Est.	Real	Fin prev.	Fin real	Hito / Entrega-ble
1	Análisis inicial y planificación del proyecto	1.1	O1	12 h	12 h	04/25	04/25	Prototipo inicial y planificación de desarrollo validada.
2	Análisis y diseño de la solución	2.1–2.2	O1–O2	25 h	25 h	04/25	04/25	Requisitos definidos, análisis de solución propuesta y prototipo inicial desarrollado para validar la viabilidad de la integración.

#	Fase	EDT	Obj.	Est.	Real	Fin prev.	Fin real	Hito / Entrega- ble
3	Desarrollo e integración del generador automático	3	O1– O4	593 h	718 h	05/26	07/26	Generador implementado e integrado dentro de UVLHub con soporte para los niveles de expresividad definidos y corrección de errores detectados durante el desarrollo.
4	Pruebas y validación de la solución	4	O5 y O6	120 h	170 h	05/26	07/26	Suite de pruebas automatizadas, validaciones funcionales y revisiones finales del sistema completadas.
Total				750 h	925 h			

Tabla 3.5: Fases del desarrollo e hitos asociados (línea base de esfuerzo y calendario vs. valores reales).

[Comenta la desviación: en qué fase se concentró, por qué se produjo y cómo se gestionó.]

Como se puede observar, la tabla presenta una desviación considerable a partir de la tercera fase. Esto se debe a las complicaciones que han ido surgiendo a lo largo de la implementación del sistema y del diseño de las pruebas, como pueden ser los errores que se han ido presentando, un enfoque impreciso en el diseño de las pruebas e inconvenientes en la gestión del código.

[Opcional: incluye un diagrama de Gantt como figura para visualizar la línea base de calendario y sus desviaciones; exporta el diagrama como imagen y descomenta el bloque anterior.]

[Tabla equivalente para la redacción de la memoria, agrupada en bloques de capítulos con hitos distintos y verificables; el desglose por capítulo individual es opcional. Todas las filas corresponden al paquete 5 de la EDT.]

# Capítulos	EDT	Est.	Real	Hito / Entregable
1 Caps. 1–3: introducción, estado del arte y planificación	5	[X h]	[Y h]	[Borrador de contexto y plan aprobado por el tutor/a.]
2 Caps. 4–7: núcleo técnico (requisitos, diseño, implementación y pruebas)	5	[X h]	[Y h]	[Núcleo técnico redactado y trazable con el repositorio.]
3 Caps. 8–9: resultados y conclusiones	5	[X h]	[Y h]	[Memoria completa lista para revisión final y depósito.]
Total		[X h]	[Y h]	

Tabla 3.6: Fases de redacción de la memoria (estimadas vs. reales).

[Tabla de resumen global: agrega las dos tablas anteriores. Los totales DEBEN coincidir con los de las tablas de origen (Tablas 3.5 y 3.6).]

# Fase	Est.	Real
1 Desarrollo (Tabla 3.5)	[X h]	[Y h]
2 Memoria (Tabla 3.6)	[X h]	[Y h]
Total	[X h]	[Y h]

Tabla 3.7: Resumen global del proyecto (estimadas vs. reales).

[Cierra la sección con una valoración global de la desviación temporal.]

3.5 Plan de comunicaciones y seguimiento

[Describe cómo se coordina el trabajo con el tutor o tutores y cómo queda constancia del avance. Tres filas bastan; lo que da valor es poder evidenciarlo después (fechas de reuniones clave, decisiones tomadas en ellas, issues cerradas).]

Actividad	Cadencia	Vía	Resultado esperado
[Reuniones con tutor/es]	[Quincenal]	[Presencial / videollamada]	[Revisión del avance y acuerdos anotados.]
[Mensajería asíncrona]	[Bajo demanda]	[Correo / chat]	[Resolución de dudas puntuales sin esperar a la reunión.]
[Registro de avance]	[Continua]	[Actas, issues, tablero]	[Trazabilidad de decisiones y tareas completadas.]

Actividad	Cadencia	Vía	Resultado esperado
-----------	----------	-----	--------------------

Tabla 3.8: Plan de comunicaciones y seguimiento del proyecto.

[Cierra con una o dos frases sobre cómo funcionó en la práctica: número aproximado de reuniones mantenidas y un ejemplo concreto de decisión tomada en una de ellas (con fecha).]

Actividad	Cadencia	Vía	Resultado esperado
Reuniones con el tutor	Según avance del proyecto	Presencial	Revisión del estado del desarrollo, validación de avances, detección de posibles deficiencias y definición de las próximas tareas a realizar.
Mensajería asíncrona	Bajo demanda	Telegram y correo electrónico	Resolución de dudas sobre el diseño y desarrollo del sistema, recepción de indicaciones adicionales, concertación de tutorías y evaluación de avances menores.
Seguimiento del desarrollo	Continua	Repositorio Git/GitHub	Trazabilidad de la evolución del proyecto mediante el historial de cambios y commits realizados durante la implementación.

Tabla 3.9: Plan de comunicaciones y seguimiento del proyecto.

Durante el desarrollo del proyecto se han mantenido diferentes reuniones presenciales con el tutor con el objetivo de revisar el estado de la solución y validar las decisiones tomadas durante la implementación. La primera reunión relevante tuvo lugar el 5 de mayo de 2026, cuando se alcanzó aproximadamente el 80 % del desarrollo, donde se concluyó que la solución era funcionalmente aceptable, aunque presentaba ciertas deficiencias técnicas que debían ser corregidas antes de considerar finalizado el proyecto. Tras aplicar las correcciones necesarias, el 29 de junio de 2026 se realizó una segunda revisión del estado del sistema, en la que se identificaron algunos aspectos que todavía requerían ajustes menores. Finalmente, tras solucionar estas últimas cuestiones y comunicar los cambios realizados al tutor, se obtuvo su validación positiva sobre el estado final de la solución el día 10 de julio del 2026.

3.6 Gestión de riesgos

[Identifica los riesgos principales que podían comprometer los objetivos o los plazos. Para cada uno indica probabilidad, impacto y exposición (combinación de ambos: Alta / Media / Baja), que ordena los riesgos por prioridad de vigilancia, y el plan de mitigación. Regla para la exposición (matriz 3×3 estándar de probabilidad por impacto): exposición = el nivel más bajo de los dos, subiendo un nivel si el impacto es Alto.]

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R1	<i>[Riesgo 1]</i>	<i>[Alta]</i>	<i>[Alto]</i>	<i>[Alta]</i>	<i>[Plan de mitigación.]</i>
R2	<i>[Riesgo 2]</i>	<i>[Media]</i>	<i>[Medio]</i>	<i>[Media]</i>	<i>[Plan de mitigación.]</i>
R3	<i>[Riesgo 3]</i>	<i>[Baja]</i>	<i>[Alto]</i>	<i>[Media]</i>	<i>[Plan de mitigación.]</i>
R4	<i>[Riesgo 4]</i>	<i>[Media]</i>	<i>[Bajo]</i>	<i>[Baja]</i>	<i>[Plan de mitigación.]</i>

Tabla 3.10: Riesgos identificados, exposición y planes de mitigación.

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R1	Complejidad de integración del generador dentro del ecosistema UVLHub-Flamapy	Alta	Alto	Alta	Realizar un estudio inicial de la arquitectura existente, validar progresivamente los cambios realizados y comprobar la compatibilidad entre los diferentes componentes del sistema.
R2	Problemas derivados de dependencias externas y configuración del entorno de ejecución	Media	Alto	Alta	Mantener actualizadas las dependencias necesarias, documentar la configuración del entorno y realizar pruebas de integración tras modificar versiones o paquetes externos.

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R3	Errores en la generación de modelos tras la incorporación de nuevas funcionalidades	Alta	Medio	Media	Realizar validaciones funcionales tras cada modificación e incorporar pruebas automatizadas que permitan detectar errores en las funcionalidades implementadas.
R4	Subestimación del esfuerzo necesario para completar la implementación y validación del sistema	Media	Alto	Media	Realizar un seguimiento periódico del progreso, priorizar las tareas críticas y reservar tiempo adicional para corrección de errores y mejora de las pruebas.

Tabla 3.11: Riesgos identificados, exposición y planes de mitigación.

3.6.1 Seguimiento de riesgos al cierre

[Un registro de riesgos solo demuestra gestión si se revisita al final. Indica, para cada riesgo, si llegó a materializarse, qué respuesta se aplicó realmente (la planificada u otra) y qué riesgo residual queda. Incluye una fila por cada riesgo de la Tabla 3.11, sin excepciones.]

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R1	<i>[Sí, en la fase 3]</i>	<i>[Qué se hizo realmente.]</i>	<i>[Qué queda pendiente.]</i>
R2	<i>[No]</i>	<i>[La mitigación preventiva funcionó / no fue necesaria.]</i>	<i>[—]</i>
R3	<i>[Parcialmente]</i>	<i>[Respuesta aplicada.]</i>	<i>[Riesgo residual.]</i>
R4	<i>[No]</i>	<i>[No fue necesaria ninguna acción.]</i>	<i>[—]</i>

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
----	------------------	--------------------	-----------------

Tabla 3.12: Seguimiento de riesgos al cierre: materialización, respuesta y residuales.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R1	Sí	El riesgo apareció principalmente durante la fase inicial de comprensión de Flamapy y durante la integración del generador dentro de UVLHub. Para solucionarlo se estudió la arquitectura de Flamapy, se realizaron modificaciones sobre la estructura del código y se aplicó una estrategia de pruebas progresivas para validar cada incorporación realizada.	El generador se encuentra completamente integrado y cumple con la funcionalidad establecida inicialmente. No obstante, como ocurre en cualquier proyecto software, siempre existe margen de mejora relacionado con la reducción de deuda técnica y la optimización de la arquitectura interna.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R2	Sí	Se produjeron problemas relacionados con las dependencias externas del proyecto, especialmente con la actualización de Flamapy, la creación y configuración de los wheels necesarios para Pyodide y los problemas encontrados para que estos paquetes fueran localizados y utilizados correctamente. La solución consistió en actualizar dependencias, crear nuevos wheels cuando fue necesario e investigar la configuración requerida para su correcta integración.	El sistema funciona correctamente con las versiones actuales de las dependencias utilizadas. Sin embargo, futuras actualizaciones de estas tecnologías podrían requerir nuevos ajustes para mantener la compatibilidad.
R3	Sí	Durante el desarrollo se detectaron en varias ocasiones problemas en la generación de modelos debido a que determinadas modificaciones no producían los resultados esperados. Para solucionarlos se comprobaba inicialmente el fallo mediante la generación de diferentes modelos, posteriormente se analizaba la causa, se corregía la implementación y se realizaban nuevas validaciones manuales, actualizando las pruebas unitarias cuando aportaban valor.	La generación de modelos ha sido validada de forma exhaustiva siguiendo los criterios establecidos. Aun así, no se descarta la aparición de nuevos casos límite ante futuras ampliaciones o nuevos escenarios de uso.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R4	Sí	La estimación inicial del esfuerzo necesario se vio superada debido a diversos factores relacionados con la calidad final del producto, como los errores detectados durante la implementación, la necesidad de mejorar el enfoque de las pruebas, los problemas de integración y las tareas de refactorización necesarias. La gestión de este riesgo se realizó priorizando las correcciones necesarias, reorganizando tareas y utilizando las revisiones con el tutor para obtener feedback sobre el estado del proyecto.	El principal riesgo residual está relacionado con la posible existencia de deuda técnica pendiente. Una mayor reducción de esta deuda mediante refactorizaciones y mejoras arquitectónicas podría facilitar futuras ampliaciones y reducir el esfuerzo necesario para incorporarlas.

Tabla 3.13: Seguimiento de riesgos al cierre: materialización, respuesta y residuales.

3.7 Control de cambios

[El plan es un documento vivo: registra los cambios sobre la línea base (alcance, plazo o enfoque) acordados durante el proyecto. Anti-burocracia: recoge solo los 3–6 cambios SIGNIFICATIVOS respecto a la línea base; no registres microajustes. Ten en cuenta que un TFG sin ningún cambio registrado resulta poco creíble. Si un cambio afectó al alcance, verifica que la Sección 1.5 quedó coherente.]

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C1	[dd/mm/aa]	[Alcance: qué se añadió o descartó.]	[Por qué fue necesario.]	[Efecto en objetivos, horas o entregables.]	[Tutor/a, reunión dd/mm/aa]

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C2	[dd/mm/aa]	[Plazo: qué hito se movió.]	[Causa.]	[Impacto.]	[Tutor/a]
C3	[dd/mm/aa]	[Enfoque: qué decisión técnica cambió.]	[Causa.]	[Impacto.]	[Tutor/a]

Tabla 3.14: Registro de control de cambios sobre la línea base.

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C1	12/05/2025	Ampliación de las opciones de configuración del generador para aumentar la flexibilidad del proceso de generación.	Necesidad de proporcionar una mayor capacidad de adaptación de los modelos generados a distintos escenarios de uso.	Incremento del alcance funcional mediante la incorporación de nuevas opciones de configuración relacionadas con la generación de operadores y atributos.	Tutor / investigadores

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C2	Durante el desarrollo	Ampliación del plazo previsto para completar la implementación y validación del generador automático.	La estimación inicial no contemplaba completamente la complejidad de la integración con el ecosistema UVLHub-Flamapy, así como los problemas surgidos durante la implementación y corrección de errores.	Desplazamiento del hito previsto de finalización del desarrollo y aumento del esfuerzo dedicado a la implementación y validación del sistema.	Tutor
C3	05/05/2026	Revisión del enfoque técnico y refactorización de la estructura interna del generador para mejorar su integración y mantenibilidad.	Durante la revisión del estado del proyecto se identificaron deficiencias técnicas relacionadas con la organización interna del código y la integración del generador dentro del ecosistema existente.	Necesidad de reorganizar componentes internos, mejorar la arquitectura de la solución y adaptar parte de la implementación existente.	Tutor, reu- nión de se- guimiento 05/05/2026

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C4	05/2026	Sustitución del front-end inicial por la propuesta proporcionada por el tutor para la integración del generador en UVLHub.	La propuesta alternativa ofrecía una mejor adaptación al flujo existente de la plataforma y una experiencia de usuario más adecuada para configurar el proceso de generación.	Modificación de la implementación del front-end y adaptación del generador al nuevo flujo de interacción, manteniendo los objetivos funcionales definidos inicialmente.	Tutor

Tabla 3.15: Registro de control de cambios sobre la línea base.

3.8 Plan de calidad

[Declara brevemente qué criterios de calidad se exigen al producto y a la memoria, y con qué mecanismo se verifica cada uno. No dupliques otros capítulos: referencia. Media página como máximo. No lo confundas con la metodología de validación (Sección 3.2.2): aquella define CÓMO se mide el cumplimiento de los objetivos; el plan de calidad fija los estándares del proceso y del producto (código, revisiones, umbrales) con los que se trabaja a diario.]

- **[Código:** estándares de estilo y linting automatizado; cómo se aplican se detalla en el Capítulo 6.]
- **[Entregables:** criterio de aceptación de cada paquete de la EDT, verificado mediante las pruebas del Capítulo 7.]
- **[Cobertura:** umbral objetivo de cobertura de pruebas (p. ej. X %), medido como se describe en la Sección 7.5.]

- *[Memoria: revisión de cada capítulo por el tutor/a y verificación de referencias cruzadas y bibliografía antes de la entrega.]*

El plan de calidad definido para el proyecto establece los criterios utilizados para garantizar la calidad tanto de la solución software desarrollada como de la documentación asociada al TFG.

- **Código:** se ha buscado mantener una implementación organizada, legible y mantenible, siguiendo una estructura coherente con la arquitectura definida para el generador. Para facilitar la comprensión del código y detectar posibles problemas de estilo, se ha utilizado una herramienta de *linting* integrada dentro del flujo de desarrollo mediante GitHub Actions, ya incorporada en la configuración de UVLHub. Además, durante el desarrollo se realizaron tareas de refactorización orientadas a mejorar la calidad interna del código y reducir posibles problemas derivados de la evolución del sistema. La trazabilidad de los cambios se ha mantenido mediante el control de versiones con Git y el uso de commits siguiendo la convención *Conventional Commits*. La siguiente imagen muestra los workflows usados:

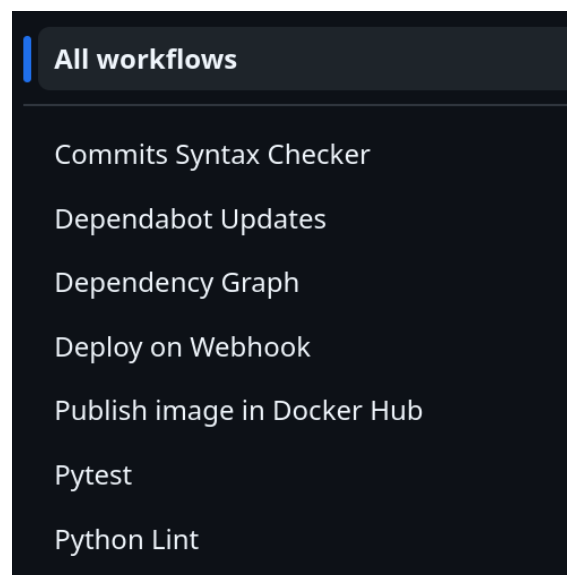


Figura 3.2: Arquitectura general de UVLHub y sus integraciones principales.
Elaboración propia.

- **Entregables:** cada bloque funcional desarrollado debía cumplir con los objetivos establecidos y aportar una funcionalidad verificable dentro del sistema. Para ello, las nuevas incorporaciones se validaban progresivamente antes de continuar con el desarrollo de nuevas capacidades, asegurando que los componentes implementados mantenían el comportamiento esperado.

- **Pruebas:** la calidad funcional del sistema se ha comprobado mediante la combinación de validaciones manuales y pruebas automatizadas. Tras la incorporación de nuevas funcionalidades se verificaba que los modelos generados cumplieran los criterios esperados y que la interacción mediante el front-end funcionaba correctamente. Además, se incorporaron pruebas unitarias adicionales cuando estas aportaban valor al detectar posibles errores relevantes en el sistema.
- **Memoria:** la calidad de la documentación se ha garantizado mediante revisiones periódicas de la estructura y contenido de la memoria, así como mediante la revisión de los avances con el tutor del proyecto. Antes de la entrega final se realizará una comprobación de la coherencia entre los diferentes capítulos, referencias cruzadas y bibliografía utilizada.

3.9 Presupuesto

[Presenta el presupuesto del proyecto. Una opción habitual es seguir el modelo TCO (Total Cost of Ownership), separando costes de capital (CapEx) y costes operacionales (OpEx). Conviene incluir, junto al coste real (a menudo 0 € en contexto académico), el valor de mercado equivalente para una valoración realista.]

CapEx — Costes de capital

[Inversiones en infraestructura física y licencias de uso prolongado.]

Recurso	Descripción	Coste real	Valor
<i>[Recurso 1]</i>	<i>[Descripción.]</i>	<i>[0,00 €]</i>	<i>[X €/año]</i>
<i>[Recurso 2]</i>	<i>[Descripción.]</i>	<i>[0,00 €]</i>	<i>[X €]</i>
Total CapEx		<i>[0,00 €]</i>	<i>[X €]</i>

Tabla 3.16: Costes de capital (CapEx).

[Justifica brevemente cómo se ha estimado el valor de mercado de cada recurso (por ejemplo, tarifa pública de un proveedor cloud o precio comercial de una licencia).]

Recurso	Descripción	Coste real	Valor
Equipo de desarrollo	Equipo personal utilizado para la implementación, pruebas y mantenimiento del sistema.	0,00 €	X €

Licencias software	Herramientas de desarrollo y tecnologías empleadas durante el proyecto, basadas principalmente en software libre.	0,00 €	X €
Total CapEx		0,00 €	X €

Tabla 3.17: Costes de capital (CapEx).

Los recursos utilizados durante el desarrollo no supusieron una inversión económica directa conocida, ya que se empleó principalmente infraestructura propia y herramientas basadas en software libre o de coste nulo. Por tanto, el coste real asociado a los costes de capital del proyecto es de 0,00 €.

OpEx — Costes operacionales

[Gastos recurrentes asociados al desarrollo y mantenimiento.]

Personal

[Indica el rol, la tarifa por hora y la justificación de esa tarifa (convenio colectivo, mercado laboral de referencia, etc.). Usa como actividades las fases de la Tabla 3.7; los totales de horas deben coincidir con ella.]

Actividad	H. estimadas	H. reales	Coste estimado (€)	Coste real (€)
<i>[Actividad 1]</i>	<i>[X]</i>	<i>[Y]</i>	<i>[0,00]</i>	<i>[0,00]</i>
<i>[Actividad 2]</i>	<i>[X]</i>	<i>[Y]</i>	<i>[0,00]</i>	<i>[0,00]</i>
<i>[Actividad 3]</i>	<i>[X]</i>	<i>[Y]</i>	<i>[0,00]</i>	<i>[0,00]</i>
Total personal	<i>[X]</i>	<i>[Y]</i>	<i>[0,00]</i>	<i>[0,00]</i>

Tabla 3.18: Desglose de costes de personal (OpEx).

El coste de personal corresponde a la dedicación asociada al contrato de apoyo técnico vinculado al proyecto TASOVA PLUS. Dicho contrato contemplaba una dedicación de 25 horas semanales y una remuneración bruta mensual de 930,60 €. Aunque el desarrollo del Trabajo Fin de Grado continuó posteriormente fuera del periodo financiado, únicamente se considera como coste real la cantidad correspondiente al periodo cubierto por dicho contrato.

Actividad	H. estimadas	H. reales	Coste estimado (€)	Coste real (€)
Análisis inicial y planificación del proyecto	12	12	111,67	59,56
Análisis y diseño de la solución	25	25	232,75	124,08
Desarrollo e integración del generador automático	593	718	5518,83	2876,50
Pruebas y validación de la solución	120	170	1117,20	662,26
Total personal	750	925	6980,45	3722,40

Tabla 3.19: Desglose de costes de personal (OpEx).

Herramientas y servicios de desarrollo

Herramienta	Licencia utilizada	Coste real	Valor de mercado
<i>[Herramienta 1]</i>	<i>[Licencia.]</i>	<i>[0 €]</i>	<i>[X €/año]</i>
<i>[Herramienta 2]</i>	<i>[Licencia.]</i>	<i>[0 €]</i>	<i>[X €/año]</i>
Total herramientas		<i>[0 €]</i>	<i>[X €/año]</i>

Tabla 3.20: Costes de herramientas y servicios de desarrollo (OpEx).

Herramienta	Licencia utilizada	Coste real	Valor de mercado
GitHub	Plataforma utilizada para almacenar el código fuente y gestionar el control de versiones.	0 €	0 €

Render	Servicio utilizado para desplegar una versión accesible de UVLHub durante las revisiones funcionales del sistema.	0 €	X € / mes
Python, Flask, Flasky y librerías asociadas	Tecnologías y dependencias utilizadas durante el desarrollo bajo licencias abiertas.	0 €	0 €
Total herramientas		0 €	X €

Tabla 3.21: Costes de herramientas y servicios de desarrollo (OpEx).

Resumen del presupuesto (TCO)

Categoría	Tipo	Coste real	Valor de mercado
Infraestructura y licencias	CapEx	[0,00 €]	[X €]
Personal	OpEx	[X €]	[X €]
Herramientas y servicios	OpEx	[0,00 €]	[X €]
TCO total		[X €]	[X €]

Tabla 3.22: Resumen del presupuesto del proyecto (TCO = CapEx + OpEx).

[Cierra la sección comentando el coste real frente al valor de mercado y la influencia del entorno académico (licencias educativas, equipos cedidos...).

Categoría	Tipo	Coste real	Valor de mercado
Infraestructura y licencias	CapEx	0,00 €	X €
Personal	OpEx	3722,40 €	X €

Herramientas y servi- cios	OpEx	0,00 €	X €
TCO total		3722,40 €	X €

Tabla 3.23: Resumen del presupuesto del proyecto (TCO = CapEx + OpEx).

El coste real del proyecto está determinado principalmente por los costes de personal asociados al contrato de apoyo técnico vinculado al proyecto TASOVA PLUS. El resto de recursos empleados no han supuesto costes adicionales debido al uso de herramientas de código abierto y servicios con planes gratuitos. Cabe destacar que, aunque el desarrollo continuó posteriormente fuera del periodo financiado, dicho esfuerzo adicional no representa un coste económico directo dentro del presupuesto del proyecto.

3.10 Cierre del proyecto y lecciones aprendidas

[Cierra formalmente el plan. Primero, verifica el proyecto contra la EDT (Tabla 3.3): todo paquete de trabajo debe estar entregado o tener un cambio registrado en la Sección 3.7 que lo descarte. Después, resume en una frase la desviación global (Tabla 3.7) y extrae 3–5 lecciones aprendidas honestas: las más valiosas son las que admiten un error y explican qué se haría distinto.]

Lección	Qué se haría distinto
<i>[P. ej., se infraestimó la fase de pruebas un 40 %.]</i>	<i>[Reservar un margen explícito para imprevistos en la estimación.]</i>
<i>[P. ej., la integración continua desde el inicio evitó regresiones tardías.]</i>	<i>[Mantenerla; automatizar también el despliegue.]</i>
<i>[P. ej., acumular la redacción de la memoria al final generó presión.]</i>	<i>[Redactar cada capítulo al cerrar su fase.]</i>

Tabla 3.24: Lecciones aprendidas al cierre del proyecto.

[Las conclusiones (Capítulo 9) retomarán esta reflexión de forma resumida: no dupliques aquí lo que dirás allí, ni allí lo que ya queda registrado aquí.]

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Con el plan establecido (qué se hará, cuándo y con qué control), el siguiente paso es precisar QUÉ debe hacer exactamente el sistema: el Capítulo 4 traduce los objetivos en un catálogo de requisitos verificables».]

Al finalizar el proyecto, todos los paquetes de trabajo definidos en la EDT han sido completados. Las fases planificadas se han ejecutado en su totalidad, sin necesidad de descartar ningún paquete de trabajo. Los cambios relevantes realizados durante el desarrollo han quedado registrados en la Sección 3.7, manteniendo la coherencia con el alcance inicial definido.

La desviación temporal global del proyecto se ha concentrado principalmente en las fases de desarrollo e integración del generador automático y pruebas y validación de la solución, debido a la complejidad añadida durante la integración con componentes externos y a la necesidad de realizar iteraciones adicionales para asegurar el correcto funcionamiento del sistema.

Lección	Qué se haría distinto
La estimación inicial del esfuerzo necesario para las fases de integración y validación fue inferior al esfuerzo realmente requerido.	Realizar una estimación más conservadora, incluyendo un margen adicional para problemas derivados de dependencias externas y correcciones durante la fase final.
El conocimiento inicial sobre algunas tecnologías utilizadas en el proyecto no era suficiente para prever completamente las dificultades de integración.	Dedicar una fase inicial más extensa al estudio de las herramientas y componentes externos antes de comenzar la implementación.
La estrategia de pruebas podría haberse definido con mayor profundidad desde las primeras etapas del desarrollo.	Diseñar desde el inicio una planificación de pruebas más completa, identificando las funcionalidades críticas y los casos de validación necesarios.
La acumulación progresiva de cambios durante el desarrollo generó cierta deuda técnica.	Realizar revisiones y refactorizaciones periódicas del código para mantener una arquitectura más mantenible durante todo el ciclo de desarrollo.

Tabla 3.25: Lecciones aprendidas al cierre del proyecto.

4 ANÁLISIS DE REQUISITOS

*[Este capítulo describe **qué** debe hacer el sistema, sin entrar todavía en cómo lo hace. Es la base contractual del TFG: si un requisito no aparece aquí, no debería darse por hecho que el sistema lo cumple.]*

4.1 Descripción del sistema

[Visión general del sistema en una página: qué es, qué resuelve y para quién.]

[Cubre:]

- *[Tipo de sistema (aplicación web, CLI, librería, servicio...)]*
- *[Qué entradas acepta y qué salidas produce.]*
- *[Subsistemas o módulos principales en los que se divide.]*
- *[Actores que interactúan con él (anónimo, autenticado, administrador...)]*

[Ejemplo de subsistemas:]

- *[**Subsistema 1**]: [responsabilidad principal.]*
- *[**Subsistema 2**]: [responsabilidad principal.]*
- *[**Subsistema 3**]: [responsabilidad principal.]*

[Ejemplo de actores:]

- *[**Actor 1**]: [rol y nivel de acceso.]*
- *[**Actor 2**]: [rol y nivel de acceso.]*

El sistema desarrollado consiste en un generador automático de modelos de características integrado dentro del ecosistema de herramientas asociado a la variabilidad software. La solución está compuesta por un plugin desarrollado como extensión de Flamapy y por su integración dentro de UVLHub, permitiendo que los usuarios puedan configurar y ejecutar el proceso de generación mediante una interfaz web.

El objetivo principal del sistema es facilitar la creación automática de modelos de características expresados en UVL con distintos niveles de complejidad y expresividad. Estos modelos permiten disponer de conjuntos de prueba más completos para la evaluación, validación y mejora de herramientas relacionadas con la Ingeniería de

Líneas de Productos Software (SPL), especialmente aquellas orientadas al análisis de variabilidad. De esta forma, el sistema aborda la necesidad de disponer de modelos con características avanzadas que permitan evaluar el comportamiento de dichas herramientas en diferentes escenarios.

La entrada principal del sistema consiste en una configuración definida por el usuario mediante los parámetros disponibles en la interfaz proporcionada por UVLHub. Estos parámetros permiten establecer las características del modelo que se desea generar, incluyendo aspectos relacionados con su estructura jerárquica, atributos y restricciones.

Como resultado, el sistema produce modelos expresados en el lenguaje UVL con los niveles de expresividad soportados. Estos modelos pueden visualizarse directamente tras finalizar el proceso de generación dentro de la plataforma UVLHub y también pueden descargarse como ficheros independientes con extensión `.uvl` para su posterior utilización en otras herramientas del ecosistema de variabilidad software.

El sistema puede dividirse principalmente en los siguientes subsistemas:

- **Subsistema de configuración del generador:** responsable de gestionar los parámetros definidos por el usuario mediante la interfaz web proporcionada en UVLHub. Este subsistema permite establecer las características del modelo que se desea generar y preparar la configuración necesaria para ejecutar el proceso desde el navegador.
- **Subsistema de generación automática:** constituye el núcleo funcional del sistema y es el encargado de producir los modelos de características a partir de la configuración proporcionada, haciendo uso de los componentes y mecanismos disponibles dentro del framework Flamapy.
- **Subsistema de integración con UVLHub:** encargado de conectar la operación generadora desarrollada con el módulo *generator* de UVLHub, permitiendo que la funcionalidad del plugin pueda ser utilizada dentro de la plataforma mediante los mecanismos definidos por esta.

Los actores principales que interactúan con el sistema son los siguientes:

- **Usuario investigador o desarrollador:** configura los parámetros del generador y obtiene modelos UVL para su utilización en procesos de evaluación, experimentación o validación de herramientas de variabilidad software. El acceso al generador no requiere autenticación previa.
- **Herramientas de análisis de variabilidad:** utilizan los modelos generados como entrada para realizar diferentes tareas relacionadas con el análisis y procesamiento de modelos de características.

4.2 Requisitos de información

[Describe las entidades de información que el sistema gestiona, con sus atributos y restricciones (claves, unicidad, tipos, opcionalidad). Suele haber una tabla por entidad.]

RI-01: *[Nombre de la entidad]*

[Breve descripción del propósito de la entidad.]

Atributo	Tipo	Descripción
<i>[atributo_1]</i>	<i>[Tipo]</i>	<i>[Descripción y restricciones.]</i>
<i>[atributo_2]</i>	<i>[Tipo]</i>	<i>[Descripción y restricciones.]</i>
<i>[atributo_3]</i>	<i>[Tipo]</i>	<i>[Descripción y restricciones.]</i>

Tabla 4.1: Atributos del requisito de información RI-01.

RI-02: *[Nombre de la entidad]*

[Breve descripción del propósito de la entidad.]

Atributo	Tipo	Descripción
<i>[atributo_1]</i>	<i>[Tipo]</i>	<i>[Descripción y restricciones.]</i>
<i>[atributo_2]</i>	<i>[Tipo]</i>	<i>[Descripción y restricciones.]</i>

Tabla 4.2: Atributos del requisito de información RI-02.

4.2.1 Requisitos de información

El sistema gestiona la información necesaria para configurar el proceso de generación automática de modelos de características. Esta información se representa mediante un conjunto de entidades de configuración que permiten definir los parámetros utilizados durante la generación.

La entidad principal del modelo conceptual es **FmgeneratorModel**, que agrupa las diferentes configuraciones específicas del generador. Cada una de estas configuraciones representa un aspecto concreto del modelo que se desea generar, como la nomenclatura, los niveles de expresividad UVL soportados, las características, la estructura jerárquica, las restricciones o los atributos.

Estas entidades son utilizadas posteriormente por la operación de generación automática para producir modelos de características expresados en UVL. La operación de generación no se considera una entidad de información, sino una funcionalidad del sistema que será descrita en los requisitos funcionales.

RI-01: FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador automático de modelos de características. Contiene los parámetros generales del proceso y las configuraciones específicas asociadas a cada aspecto del modelo generado.

Atributo	Tipo	Descripción
num_models	int	Número de modelos que se desean generar.
seed	int	Semilla utilizada para controlar la generación aleatoria.
ensure_satisfiable	bool	Indica si los modelos generados deben ser satisfacibles.
naming	NamingConfig	Configuración relacionada con la nomenclatura de los elementos generados.
levels	LevelConfig	Configuración de los niveles de expresividad soportados por UVL.
features	FeaturesConfig	Configuración relacionada con la generación de características.
hierarchy	HierarchyConfig	Configuración de la estructura jerárquica del modelo.
constraints	ConstraintsConfig	Configuración de las restricciones generadas.
attributes	AttributesConfig	Configuración de los atributos asociados a las características.

Tabla 4.3: Atributos del requisito de información RI-01.

RI-02: NamingConfig

La entidad **NamingConfig** representa la configuración utilizada para definir la nomenclatura de los elementos generados por el sistema.

Atributo	Tipo	Descripción
name_prefix	str	Prefijo utilizado para los nombres generados.

Atributo	Tipo	Descripción
include_feature_count_suffix	bool	Indica si se incluye el número de características en el nombre del modelo generado.
include_constraint_count_suffix	bool	Indica si se incluye el número de restricciones en el nombre del modelo generado.

Tabla 4.4: Atributos del requisito de información RI-02.

RI-03: LevelConfig

La entidad **LevelConfig** representa los niveles de expresividad UVL que estarán disponibles durante el proceso de generación.

Atributo	Tipo	Descripción
boolean_level	bool	Indica si se habilita la generación de modelos booleanos.
arithmetic_level	bool	Indica si se habilitan elementos aritméticos en los modelos generados.
type_level	bool	Indica si se permite el uso de tipos en características y atributos.
group_cardinality	bool	Indica si se habilitan cardinalidades de grupo.
feature_cardinality	bool	Indica si se habilitan cardinalidades individuales de características.
aggregate_functions	bool	Indica si se permite la generación de funciones agregadas.
string_constraints	bool	Indica si se habilitan restricciones sobre valores de tipo cadena.

Tabla 4.5: Atributos del requisito de información RI-03.

RI-04: FeaturesConfig

La entidad **FeaturesConfig** contiene los parámetros relacionados con la generación de características del modelo.

Atributo	Tipo	Descripción
min_features	int	Número mínimo de características que puede contener el modelo generado.
max_features	int	Número máximo de características que puede contener el modelo generado.
dist_boolean	float	Probabilidad de generación de características booleanas.
dist_integer	float	Probabilidad de generación de características enteras.
dist_real	float	Probabilidad de generación de características reales.
dist_string	float	Probabilidad de generación de características de tipo cadena.
min_feature_cardinality	int / list[int]	Valor mínimo permitido para las cardinalidades de características.
max_feature_cardinality	int / list[int]	Valor máximo permitido para las cardinalidades de características.
prob_feature_cardinality	float	Probabilidad de aplicar cardinalidades a las características.

Tabla 4.6: Atributos del requisito de información RI-04.

RI-05: HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo generado.

Atributo	Tipo	Descripción
max_tree_depth	int	Profundidad máxima permitida para la jerarquía del modelo.
dist_optional	float	Probabilidad de generar relaciones opcionales.
dist_mandatory	float	Probabilidad de generar relaciones obligatorias.
dist_alternative	float	Probabilidad de generar relaciones alternativas.

Atributo	Tipo	Descripción
dist_or	float	Probabilidad de generar relaciones OR.
group_cardinality_minint		Cardinalidad mínima para grupos de características.
group_cardinality_maxint		Cardinalidad máxima para grupos de características.
dist_group_cardinalityfloat		Probabilidad de generar relaciones con cardinalidad de grupo.

Tabla 4.7: Atributos del requisito de información RI-05.

RI-06: ConstraintsConfig

La entidad **ConstraintsConfig** contiene los parámetros utilizados para definir la generación de restricciones entre elementos del modelo.

Atributo	Tipo	Descripción
min_constraints	int	Número mínimo de restricciones generadas.
max_constraints	int	Número máximo de restricciones generadas.
extra_constraint_representativeness	float	Factor utilizado para aumentar la representatividad de las restricciones.
min_vars_per_constraint	int	Número mínimo de variables utilizadas en una restricción.
max_vars_per_constraint	int	Número máximo de variables utilizadas en una restricción.
ctc_dist_boolean	float	Probabilidad de generar restricciones booleanas.
ctc_dist_integer	float	Probabilidad de generar restricciones enteras.
ctc_dist_real	float	Probabilidad de generar restricciones reales.
ctc_dist_string	float	Probabilidad de generar restricciones sobre cadenas.
prob_not	float	Probabilidad de aplicar negación en restricciones booleanas.

Atributo	Tipo	Descripción
prob_and	float	Probabilidad de utilizar operadores AND.
prob_or_ct	float	Probabilidad de utilizar operadores OR.
prob_implication	float	Probabilidad de utilizar implicaciones.
prob_equivalence	float	Probabilidad de utilizar equivalencias.
prob_sum	float	Probabilidad de utilizar operaciones de suma.
prob_substract	float	Probabilidad de utilizar operaciones de resta.
prob_multiply	float	Probabilidad de utilizar operaciones de multiplicación.
prob_divide	float	Probabilidad de utilizar operaciones de división.
prob_equals	float	Probabilidad de utilizar comparaciones de igualdad.
prob_less	float	Probabilidad de utilizar comparaciones menor que.
prob_greater	float	Probabilidad de utilizar comparaciones mayor que.
prob_less_equals	float	Probabilidad de utilizar comparaciones menor o igual que.
prob_greater_equals	float	Probabilidad de utilizar comparaciones mayor o igual que.
prob_sum_function	float	Probabilidad de utilizar funciones agregadas de suma.
prob_avg_function	float	Probabilidad de utilizar funciones agregadas de promedio.
prob_len_function	float	Probabilidad de utilizar funciones de longitud sobre cadenas.

Tabla 4.8: Atributos del requisito de información RI-06.

RI-07: AttributesConfig

La entidad **AttributesConfig** representa la configuración relacionada con los atributos asociados a las características generadas.

Atributo	Tipo	Descripción
random_attributes	bool	Indica si los atributos deben generarse automáticamente o definirse manualmente.
min_attributes	int / None	Número mínimo de atributos generados en modo aleatorio.
max_attributes	int / None	Número máximo de atributos generados en modo aleatorio.
attributes_list	list[dict]	Lista de atributos definidos manualmente para la generación.
attribute_attach_prob	list[float]	Probabilidades de asociación de atributos a características.
attribute_in_constraint	list[bool]	Indica si determinados atributos pueden participar en restricciones.

Tabla 4.9: Atributos del requisito de información RI-07.

4.3 Requisitos funcionales

[Lista las capacidades concretas que el sistema debe ofrecer, agrupadas por bloques temáticos (gestión de usuarios, lógica de negocio principal, exploración...). Cada requisito tiene un identificador único, un nombre, una descripción y una prioridad (Alta/Media/Baja).]

ID	Nombre	Descripción	Prior.
RF-01	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe-</i> <i>rado.]</i>	<i>[Alta]</i>
RF-02	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe-</i> <i>rado.]</i>	<i>[Alta]</i>
RF-03	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe-</i> <i>rado.]</i>	<i>[Media]</i>
RF-04	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe-</i> <i>rado.]</i>	<i>[Alta]</i>

ID	Nombre	Descripción	Prior.
RF-05	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe- rado.]</i>	<i>[Alta]</i>
RF-06	<i>[Nombre del RF]</i>	<i>[Descripción del comportamiento espe- rado.]</i>	<i>[Media]</i>

Tabla 4.10: Requisitos funcionales del sistema.

El sistema debe proporcionar las funcionalidades necesarias para permitir la configuración, ejecución y obtención de modelos de características generados automáticamente. Los requisitos funcionales identificados se muestran en la Tabla 4.11.

ID	Nombre	Descripción	Prior.
RF-01	Configuración del generador	El sistema debe permitir al usuario definir los parámetros necesarios para el proceso de generación, incluyendo aspectos relacionados con las características, niveles de expresividad, estructura jerárquica, restricciones y atributos del modelo generado.	Alta
RF-02	Validación de configuración	El sistema debe comprobar que los parámetros introducidos por el usuario constituyen una configuración válida antes de iniciar el proceso de generación. Las restricciones y condiciones específicas aplicables a dicha validación se describen en la subsección 4.5.	Alta
RF-03	Generación automática de modelos UVL	El sistema debe permitir generar automáticamente modelos de características expresados en UVL a partir de la configuración proporcionada por el usuario.	Alta

ID	Nombre	Descripción	Prior.
RF-04	Soporte de niveles de expresividad UVL	El sistema debe permitir generar modelos incorporando diferentes niveles de expresividad del lenguaje UVL, incluyendo construcciones booleanas, aritméticas y de tipos cuando hayan sido seleccionadas en la configuración.	Alta
RF-05	Visualización del modelo generado	El sistema debe mostrar el modelo de características generado dentro de la plataforma UVLHub tras finalizar correctamente el proceso de generación.	Media
RF-06	Descarga del modelo generado	El sistema debe permitir descargar el modelo generado como un fichero independiente con extensión <code>.uvl</code> para su posterior utilización en otras herramientas del ecosistema de variabilidad software.	Alta

Tabla 4.11: Requisitos funcionales del sistema.

4.4 Requisitos no funcionales

[Atributos transversales del sistema: seguridad, rendimiento, mantenibilidad, calidad de código, despliegue, estándares... Son tan vinculantes como los funcionales y deben poder verificarse. Formula cada RNF con una MÉTRICA y un UMBRAL comprobables, no con adjetivos vagos: en lugar de «el sistema debe ser rápido», escribe «el 95 % de las peticiones se atiende en menos de 200 ms». El Capítulo 8 usará estos umbrales como criterios de éxito medibles, así que un RNF que no se pueda medir aquí no se podrá evaluar allí. El umbral de cada RNF cuantitativo debe coincidir con el del criterio de éxito asociado de la Tabla 1.1: un único umbral, dos vistas.]

ID	Nombre	Descripción	Prior.
RNF-01	<i>[Nombre del RNF]</i>	<i>[Atributo + métrica + umbral, p. ej. «el 95 % de las peticiones se atiende en menos de 200 ms, medido con una prueba de carga».]</i>	<i>[Alta]</i>

ID	Nombre	Descripción	Prior.
RNF-02	<i>[Nombre del RNF]</i>	<i>[Atributo de seguridad/rendimiento/- calidad con métrica, umbral y forma de medirlo.]</i>	<i>[Alta]</i>
RNF-03	<i>[Nombre del RNF]</i>	<i>[Atributo de seguridad/rendimiento/- calidad con métrica, umbral y forma de medirlo.]</i>	<i>[Media]</i>
RNF-04	<i>[Nombre del RNF]</i>	<i>[Atributo de seguridad/rendimiento/- calidad con métrica, umbral y forma de medirlo.]</i>	<i>[Alta]</i>
RNF-05	<i>[Nombre del RNF]</i>	<i>[Atributo de seguridad/rendimiento/- calidad con métrica, umbral y forma de medirlo.]</i>	<i>[Media]</i>

Tabla 4.12: Requisitos no funcionales del sistema.

ID	Nombre	Descripción	Prior.
RNF-01	Arquitectura modular y mantenibilidad	El sistema debe mantener una estructura modular basada en la arquitectura de plugins de Flamapy, siguiendo las convenciones establecidas por el ecosistema. Esta organización debe facilitar el mantenimiento del código, la incorporación de nuevas funcionalidades y la integración con otros componentes del framework.	Alta
RNF-02	Compatibilidad con modelos UVL	Los modelos generados deben cumplir la especificación UVL soportada por las herramientas del ecosistema de variabilidad software, permitiendo su posterior utilización en procesos de análisis y evaluación de modelos de características.	Alta

ID	Nombre	Descripción	Prior.
RNF-03	Ejecución eficiente del proceso de generación	El sistema debe minimizar la carga computacional sobre el servidor mediante la ejecución del proceso de generación en el entorno del cliente cuando sea posible. Para ello, se emplean tecnologías como Pyodide y WebAssembly (WASM), permitiendo ejecutar el código del generador directamente en el navegador y reduciendo los tiempos de respuesta asociados a la comunicación y procesamiento remoto.	Alta
RNF-04	Usabilidad de la interfaz de configuración	La interfaz proporcionada para configurar el generador debe permitir al usuario seleccionar y comprender los parámetros disponibles de forma clara, facilitando la creación de configuraciones válidas y reduciendo posibles errores durante el proceso de configuración.	Media
RNF-05	Calidad del software	El código desarrollado debe cumplir los mecanismos de control de calidad establecidos en el proyecto, incluyendo las comprobaciones automáticas mediante integración continua y las herramientas de análisis estático utilizadas para mejorar la legibilidad y mantenibilidad del código.	Alta

Tabla 4.13: Requisitos no funcionales del sistema.

4.5 Reglas de negocio

[Las reglas de negocio (RN) son afirmaciones sobre cómo se comporta el dominio que el sistema debe respetar, independientemente de la tecnología o de la interfaz. No son requisitos funcionales (qué hace el sistema) ni requisitos no funcionales (atributos

transversales): son restricciones lógicas del problema. Son las que se rompen últimas si cambian: no porque hayamos refactorizado, sino porque el negocio ha cambiado.]

ID	Nombre	Descripción
RN-01	<i>[Nombre de la regla]</i>	<i>[Enunciado preciso de la regla y, si procede, la condición que la activa o la fuente que la justifica (normativa, decisión de producto, restricción del dominio).]</i>
RN-02	<i>[Nombre de la regla]</i>	<i>[Enunciado preciso.]</i>
RN-03	<i>[Nombre de la regla]</i>	<i>[Enunciado preciso.]</i>

Tabla 4.14: Reglas de negocio del dominio.

ID	Nombre	Descripción
RN-01	Validación de parámetros generales	Los parámetros generales del generador deben cumplir las restricciones mínimas necesarias para permitir la ejecución del proceso de generación. El número de modelos a generar (<code>num_models</code>) debe encontrarse comprendido entre 1 y 1000. Además, la semilla utilizada para controlar la aleatoriedad del proceso (<code>seed</code>) debe ser un valor positivo.

ID	Nombre	Descripción
RN-02	Validación de niveles de expresividad	Los niveles de expresividad del generador deben respetar la jerarquía establecida entre niveles principales (<i>major levels</i>) y niveles secundarios (<i>minor levels</i>). El nivel booleano (<code>boolean_level</code>) constituye el nivel base del generador y debe permanecer siempre habilitado. La activación de un nivel principal implica la disponibilidad de los niveles secundarios asociados. Por otro lado, un nivel secundario únicamente puede activarse cuando el nivel principal necesario para soportar su funcionalidad se encuentra habilitado. En concreto, los niveles deben cumplir las siguientes dependencias: ■ El nivel aritmético (<code>arithmetic_level</code>) permite la generación de expresiones numéricas y es necesario para habilitar los niveles secundarios relacionados con cardinalidades, funciones agregadas y operaciones aritméticas. ■ El nivel de tipos (<code>type_level</code>) requiere la activación previa del nivel aritmético. ■ Los niveles secundarios no pueden permanecer activos si el nivel del que dependen está deshabilitado.
RN-03	Validación de configuración del árbol de características	La configuración del árbol de características generado debe cumplir las restricciones necesarias para garantizar una estructura válida del modelo. Estas validaciones afectan al número de características, la estructura jerárquica y las distribuciones probabilísticas asociadas.

ID	Nombre	Descripción
		Las restricciones establecidas son las siguientes: ▪ Los parámetros <code>min_features</code> y <code>max_features</code> deben ser mayores o iguales que uno, y el número mínimo de características no puede superar al número máximo configurado. ▪ La profundidad máxima del árbol (<code>max_tree_depth</code>) debe encontrarse comprendida entre uno y el número máximo de características configurado (<code>max_features</code>). ▪ Las probabilidades asociadas a las relaciones jerárquicas deben encontrarse entre 0 y 1 y formar una distribución cuya suma sea exactamente uno. ▪ Cuando se habilite la cardinalidad de características (<code>feature_cardinality</code>), los parámetros asociados deben cumplir las siguientes restricciones: • Los valores <code>min_feature_cardinality</code> y <code>max_feature_cardinality</code> deben ser mayores o iguales que uno, y el valor mínimo no puede superar al máximo. • La probabilidad de asignación de cardinalidad (<code>prob_feature_cardinality</code>) debe encontrarse dentro del rango comprendido entre 0 y 1.

ID	Nombre	Descripción
		■ Cuando se habilite la cardinalidad de grupos (<code>group_cardinality</code>), los parámetros asociados deben cumplir las siguientes restricciones: • Los valores <code>group_cardinality_min</code> y <code>group_cardinality_max</code> deben ser mayores o iguales que uno, y el valor mínimo no puede superar al máximo. • Ambos valores no pueden superar el número máximo de características configurado (<code>max_features</code>). ■ La distribución de tipos de características únicamente debe estar disponible cuando el nivel de tipos (<code>type_level</code>) esté habilitado y sus probabilidades deben sumar exactamente uno.

ID	Nombre	Descripción
RN-04	Validación de configuración de restricciones	La configuración de restricciones entre características y atributos debe cumplir las siguientes reglas: ■ Los límites del número de restricciones (<code>min_constraints</code> y <code>max_constraints</code>) deben ser mayores o iguales que uno y el mínimo no puede superar al máximo. ■ El número mínimo de variables utilizadas en una restricción (<code>min_vars_per_constraint</code>) debe ser menor o igual que el número máximo permitido (<code>max_vars_per_constraint</code>), siendo ambos valores mayores o iguales que uno. ■ El parámetro <code>extra_constraint_representativeness</code> debe ser mayor o igual que uno. ■ Las probabilidades utilizadas en la generación de restricciones deben encontrarse dentro del rango comprendido entre 0 y 1. ■ La distribución asociada a los tipos de restricciones (<code>ctc_dist_boolean</code>, <code>ctc_dist_integer</code>, <code>ctc_dist_real</code> y <code>ctc_dist_string</code>) debe sumar exactamente uno cuando los niveles de expresividad correspondientes estén habilitados. ■ La distribución de operadores booleanos (<code>prob_and</code>, <code>prob_or_ct</code>, <code>prob_implication</code> y <code>prob_equivalence</code>) debe sumar exactamente uno. La probabilidad asociada al operador NOT (<code>prob_not</code>) se considera de forma independiente y debe encontrarse dentro del rango comprendido entre 0 y 1.

ID	Nombre	Descripción
		■ Cuando el nivel aritmético esté habilitado, la distribución de operaciones aritméticas básicas (<code>prob_sum</code>, <code>prob_subtract</code>, <code>prob_multiply</code> y <code>prob_divide</code>) debe sumar exactamente uno. ■ Las probabilidades asociadas a las funciones agregadas (<code>prob_sum_function</code> y <code>prob_avg_function</code>) se consideran de forma independiente y su suma debe ser menor o igual que uno. ■ Cuando el nivel aritmético esté habilitado, la distribución de operadores de comparación (<code>prob_equals</code>, <code>prob_less</code>, <code>prob_greater</code>, <code>prob_less_equals</code> y <code>prob_greater_equals</code>) debe sumar exactamente uno.

ID	Nombre	Descripción
RN-05	Validación de configuración de atributos	La configuración de atributos debe cumplir las restricciones asociadas al modo de generación seleccionado: ■ En modo de generación automática (<code>random_attributes</code>), los límites <code>min_attributes</code> y <code>max_attributes</code> deben estar definidos, deben ser mayores o iguales que cero y el mínimo no puede superar al máximo. ■ Cuando se utiliza la generación automática de atributos, la distribución de tipos de atributos (<code>attribute_dist_boolean</code>, <code>attribute_dist_integer</code>, <code>attribute_dist_real</code> y <code>attribute_dist_string</code>) debe encontrarse dentro del rango comprendido entre 0 y 1 y su suma debe ser exactamente uno. ■ En modo de definición manual, cada atributo debe cumplir las restricciones asociadas a su tipo: • Los atributos booleanos deben definir al menos un valor posible. • Los atributos de tipo entero, real o cadena deben establecer un valor mínimo y máximo válidos, siendo el mínimo menor o igual que el máximo. ■ En modo manual, la probabilidad de asociación del atributo (<code>attribute_attach_prob</code>) debe encontrarse dentro del rango comprendido entre 0 y 1.

Tabla 4.15: Reglas de negocio del dominio.

4.6 Restricciones del proyecto

[Limitaciones externas que acotan el espacio de soluciones del proyecto. A diferencia de los RNF, no son atributos del sistema construido sino condicionantes del proceso de construcción.]

ID	Tipo	Descripción
RES-01	<i>[Tecnológica]</i>	<i>[p. ej. uso obligatorio de un lenguaje, framework o plataforma concretos.]</i>
RES-02	<i>[Legal / regulatoria]</i>	<i>[p. ej. cumplimiento de RGPD, normativa sectorial, accesibilidad WCAG.]</i>
RES-03	<i>[Organizativa]</i>	<i>[p. ej. uso de la infraestructura del grupo de investigación, integración con sistemas internos existentes.]</i>
RES-04	<i>[Económica / temporal]</i>	<i>[p. ej. presupuesto cero, plazo fijado por el calendario académico.]</i>

Tabla 4.16: Restricciones del proyecto.

ID	Tipo	Descripción
RES-01	Tecnológica	El desarrollo debe integrarse dentro del ecosistema existente de herramientas de variabilidad software, utilizando como base el framework Flamapy y la plataforma UVLHub. Esta condición limita la arquitectura de la solución, ya que el generador debe seguir los mecanismos de extensión definidos por Flamapy, adaptarse a la estructura proporcionada por UVLHub y utilizar las librerías propias del framework y componentes externos necesarios para la generación y representación de modelos de características.
RES-02	Tecnológica / interoperabilidad	Los modelos generados deben utilizar el lenguaje UVL como formato de salida, debido a que constituye el estándar empleado dentro del ecosistema de herramientas con el que se integra el proyecto.

ID	Tipo	Descripción
RES-03	Organizativa	El desarrollo debe realizarse dentro del contexto del grupo de investigación y de los proyectos software existentes asociados, respetando los procedimientos, recursos y estructuras de trabajo establecidos.
RES-04	Temporal	El proyecto está condicionado por la planificación académica del TFG, estableciendo un periodo limitado para realizar las fases de análisis, diseño, implementación, validación y documentación de la solución.

Tabla 4.17: Restricciones del proyecto.

4.7 Casos de uso

4.7.1 Diagrama general de casos de uso

*[Resumen narrativo del diagrama: qué actores hay, qué casos de uso conectan con cada actor y qué relaciones **include** o **extend** existen entre casos de uso.]*

[(Espacio reservado para el diagrama de casos de uso del sistema, no incluido en esta plantilla.)]

4.7.2 Diagrama general de casos de uso

El diagrama de casos de uso representa las principales interacciones entre los actores externos y el sistema generador de modelos UVL. El usuario investigador o desarrollador es el encargado de configurar los parámetros del generador y, una vez finalizado el proceso, puede visualizar los modelos generados dentro de la plataforma o descargarlos para su posterior utilización. El sistema se encarga de validar la configuración proporcionada y ejecutar el proceso de generación automática de modelos de características UVL. Los modelos descargados pueden ser utilizados posteriormente por herramientas externas de análisis de variabilidad para realizar procesos de evaluación y análisis sobre los modelos generados.

La Figura 4.1 muestra el diagrama general de casos de uso del sistema.

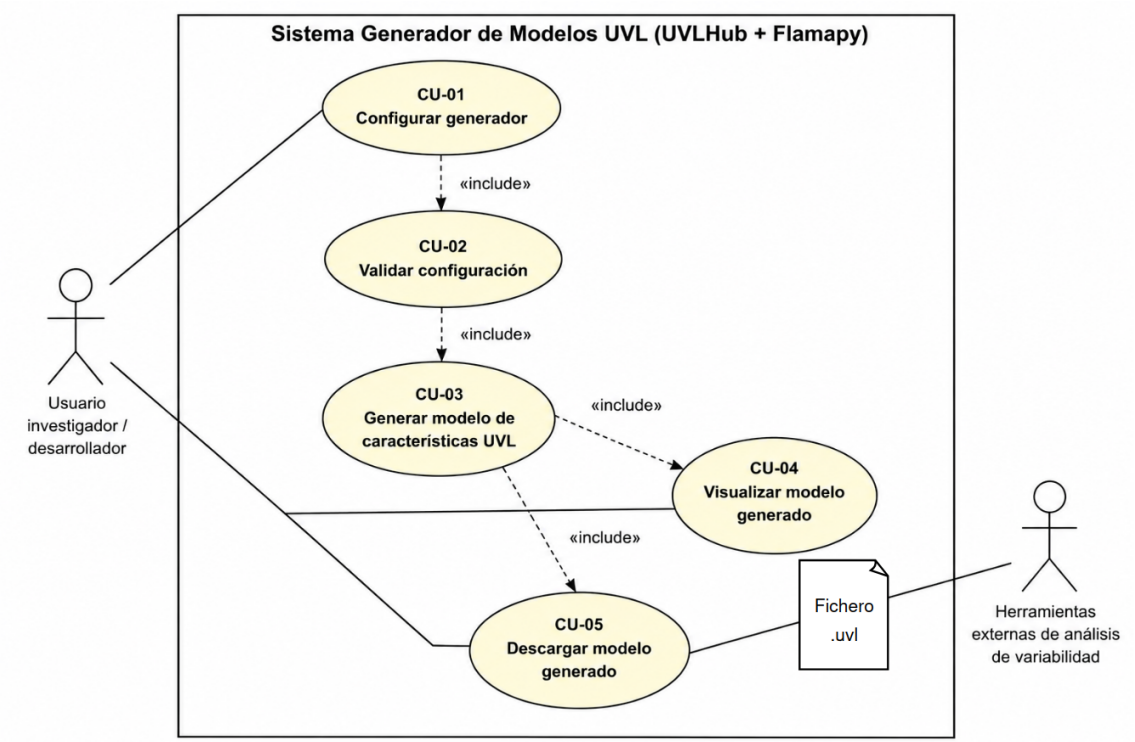


Figura 4.1: Diagrama general de casos de uso del sistema. Elaboración propia

4.7.3 Especificación de casos de uso principales

[Especifica con detalle los casos de uso más relevantes. Cada uno usa una ficha con los campos: actores, descripción, precondiciones, flujo principal, flujos alternativos y postcondiciones. Repite la ficha tantas veces como casos de uso quieras especificar.]

CU-01 — [Nombre del caso de uso]	
Actores	[Actor/es implicado/s.]
Descripción	[Resumen del objetivo del caso de uso.]
Precondiciones	[Estado previo necesario para iniciar el caso de uso.]
Flujo principal	[1. Paso 1. 2. Paso 2. 3. Paso 3. 4. Paso 4. 5. Paso 5.]
Flujo alternativo	[Xa. Variante o error y respuesta del sistema.]
Postcondiciones	[Estado del sistema tras la ejecución correcta.]

Tabla 4.18: Especificación del caso de uso CU-01.

CU-01 — Configurar generador	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario definir la configuración necesaria para el proceso de generación automática de modelos de características UVL. El usuario puede seleccionar los niveles de expresividad y establecer los parámetros relacionados con características, estructura jerárquica, restricciones y atributos.
Precondiciones	El usuario debe acceder a la interfaz de configuración del generador disponible dentro de UVLHub.
Flujo principal	1. El usuario accede al formulario de configuración del generador. 2. El usuario establece los parámetros asociados a la generación del modelo en cada <i>step</i>. 3. El sistema registra la configuración proporcionada.
Flujo alternativo	Si los valores introducidos no cumplen las restricciones establecidas, el sistema informa al usuario de los errores detectados durante la introducción de dichos valores o antes de pasar al siguiente <i>step</i> .
Postcondiciones	Si la configuración de cada <i>step</i> es válida, se procede en el último <i>step</i> a la generación de los modelos.

Tabla 4.19: Especificación del caso de uso CU-01.

CU-02 — Validar configuración	
Actores	Sistema generador de modelos UVL.
Descripción	Permite comprobar que la configuración proporcionada por el usuario cumple las reglas de negocio definidas antes de ejecutar el proceso de generación automática.
Precondiciones	Debe existir una configuración del generador proporcionada por el usuario.
Flujo principal	1. El sistema recibe la configuración definida por el usuario y analiza sus parámetros. 3. El sistema comprueba las restricciones asociadas a características, jerarquía, restricciones y atributos en cada <i>step</i>, respectivamente. 4. El sistema valida las distribuciones probabilísticas y dependencias entre parámetros.
Flujo alternativo	Si alguna regla de negocio no se cumple, el sistema devuelve los errores detectados e impide continuar con la generación hasta corregir la configuración.
Postcondiciones	Si la configuración de cada <i>step</i> es válida, se le permite al usuario pasar al siguiente <i>step</i> .

Tabla 4.20: Especificación del caso de uso CU-02.

CU-03 — Generar modelo de características UVL	
Actores	Sistema generador de modelos UVL.
Descripción	Permite generar automáticamente un modelo de características expresado en UVL a partir de una configuración previamente validada.
Precondiciones	Las dependencias necesarias deben ser cargadas correctamente al acceder al primer <i>step</i> . Además, la configuración proporcionada por el usuario debe ser validada en cada <i>step</i> .
Flujo principal	1. Cuando el usuario llega al último <i>step</i>, el sistema inicializa el proceso de generación utilizando la configuración proporcionada. 2. El sistema genera, para cada modelo y de forma aleatoria, la estructura jerárquica del modelo de características, incorporando los niveles de expresividad, atributos y restricciones pertinentes, cumpliendo con la configuración seleccionada por el usuario. 3. El sistema genera la representación final de los modelos en formato UVL y los almacena temporalmente para su visualización o descarga.
Flujo alternativo	Si se produce un error durante el proceso de generación, el sistema informa del fallo y no genera los modelos solicitados.
Postcondiciones	Existe un/unos modelo/s de características UVL generado/s correctamente disponible/s para su posterior visualización o descarga.

Tabla 4.21: Especificación del caso de uso CU-03.

CU-04 — Visualizar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario consultar visualmente el modelo de características generado dentro de la plataforma UVLHub.
Precondiciones	Debe existir al menos un modelo generado correctamente por el sistema.
Flujo principal	1. El sistema carga la representación del modelo generado. 2. El sistema muestra el modelo dentro de la interfaz de UVLHub al hacer clic en el botón correspondiente. 3. El usuario inspecciona la estructura y elementos generados.
Flujo alternativo	Si no se ha podido realizar la generación previa a la visualización, el sistema lanza un error informativo.
Postcondiciones	El usuario puede consultar el modelo generado dentro de la plataforma.

Tabla 4.22: Especificación del caso de uso CU-04.

CU-05 — Descargar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite descargar en formato zip los modelos generados como ficheros independientes en formato UVL para su posterior utilización dentro o fuera de la plataforma.
Precondiciones	Debe existir un modelo de características generado correctamente.
Flujo principal	1. El usuario solicita la descarga del modelo generado haciendo clic en el botón correspondiente. 2. El sistema prepara el zip con los ficheros UVL generados. El usuario puede ver el archivo en la bandeja de descargas de su navegador.
Flujo alternativo	Si el modelo no se encuentra disponible o se produce un error durante la generación del fichero, el sistema informa al usuario y cancela la descarga.
Postcondiciones	El modelo UVL queda disponible como fichero independiente para su almacenamiento o análisis mediante herramientas externas.

Tabla 4.23: Especificación del caso de uso CU-05.

4.8 Matriz de trazabilidad

[La matriz de trazabilidad cruza los requisitos funcionales con los casos de uso que los implementan y, opcionalmente, con las pruebas que los verifican. Demuestra que ningún requisito queda huérfano y que cada caso de uso responde a uno o más requisitos identificados.]

[Marca con × la celda en la que el caso de uso (o la prueba) cubre el requisito de la fila. Una fila vacía es un requisito sin cobertura; una columna vacía es un caso de uso sin requisito que lo justifique. Ambos son señales de alerta.]

4.8.1 Requisitos funcionales ↔ casos de uso

[Ajusta el número de columnas al número real de casos de uso especificados (la tabla de ejemplo muestra seis columnas, pero solo se han incluido tres fichas de ejemplo).]

	CU-01	CU-02	CU-03	CU-04	CU-05	CU-06
RF-01	[×]					
RF-02		[×]				
RF-03			[×]			
RF-04				[×]		
RF-05					[×]	
RF-06						[×]

Tabla 4.24: Matriz de trazabilidad entre requisitos funcionales y casos de uso.

	CU-01	CU-02	CU-03	CU-04	CU-05
RF-01	×				
RF-02		×			
RF-03			×		
RF-04			×		
RF-05				×	
RF-06					×

Tabla 4.25: Matriz de trazabilidad entre requisitos funcionales y casos de uso.

4.8.2 Requisitos ↔ pruebas

[Cruce de los requisitos con la suite de pruebas (Capítulo 7). Cada requisito debería tener al menos una prueba que lo cubra. Indica el identificador o nombre del test correspondiente.]

Requisito	Tipo de prueba	Test / archivo
RF-01	[Unitaria / Integración / E2E]	[test_XXX::test_caso]
RF-02	[Unitaria / Integración / E2E]	[test_XXX::test_caso]
RF-03	[Unitaria / Integración / E2E]	[test_XXX::test_caso]
RNF-01	[Tipo]	[Test / verificación.]
RNF-02	[Tipo]	[Test / verificación.]

Tabla 4.26: Matriz de trazabilidad entre requisitos y pruebas.

Requisito	Tipo de prueba	Test / archivo
RF-01	Integración / E2E	<code>test_full_happy_path_produces_valid_params</code> (<code>test_wizard_flow.py</code>)
RF-02	Unitaria	<code>test_step1_rejects_batch_size_that_could_</code> <code>_generation,</code> <code>test_step2_rejects_minor_levels_without_required_maj</code> <code>test_step3_rejects_invalid_relation_distribution</code> (<code>test_validators.py</code>)
RF-03	E2E	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>)
RF-04	Integración	<code>test_everything_on_every_family_represented</code> (<code>test_wizard_outputs.py</code>)
RF-05	E2E	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>)
RF-06	Integración	<code>test_filename_suffixes_applied_to_generated_files</code> (<code>test_wizard_outputs.py</code>)
RNF-01	Integración / Compatibilidad	Verificación mediante la ejecución del sistema dentro del ecosistema UVLHub y la integración con Flamapy. Los tests <code>test_full_happy_path_produces_valid_params</code> y los tests de generación de modelos comprueban que el generador puede utilizarse desde la plataforma existente.
RNF-02	Integración / Validación de formato	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>) verifica que el sistema genera modelos correctamente en formato UVL y permite su posterior visualización.
RNF-03	Verificación estructural	La integración dentro del repositorio y la adaptación a la arquitectura existente se comprueban mediante la ejecución completa de la suite de pruebas del módulo generador, incluyendo los tests de flujo del wizard (<code>test_wizard_flow.py</code>).
RNF-04	Verificación documental	El cumplimiento de la planificación temporal se valida mediante el seguimiento del desarrollo, las revisiones periódicas y la documentación generada durante el proyecto.

Requisito	Tipo de prueba	Test / archivo
-----------	----------------	----------------

Tabla 4.27: Matriz de trazabilidad entre requisitos funcionales y pruebas.

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Con los requisitos especificados y trazados, queda fijado qué debe hacer el sistema. El Capítulo 5 aborda cómo satisfacerlos: la arquitectura, los modelos y las decisiones de diseño que dan forma a la solución».]

Con los requisitos funcionales, no funcionales, reglas de negocio y restricciones del proyecto definidos y trazados, queda establecido el comportamiento esperado del sistema y las condiciones que debe cumplir la solución desarrollada. El siguiente capítulo aborda cómo satisfacer estos requisitos mediante la definición de la arquitectura del sistema, los componentes principales y las decisiones de diseño adoptadas durante el desarrollo.

5 DISEÑO DE LA SOLUCIÓN

*[Este capítulo describe **cómo** se ha diseñado el sistema para cumplir los requisitos del capítulo anterior. Se centra en la arquitectura, los modelos y los flujos de datos, sin entrar todavía en el código concreto (eso es la implementación).]*

5.1 Modelo conceptual del dominio

[Si el TFG se apoya en un modelo formal del dominio (modelo de features, ontología, modelo de datos, gramática, modelo matemático...), descríbelo aquí. Si no aplica, sustituye esta sección por la que mejor encaje con el trabajo.]

[Cubre:]

- *[Para qué sirve el modelo y qué representa.]*
- *[Dónde se almacena (archivo, formato, lenguaje).]*
- *[Cómo se relaciona con el resto del sistema (validación, generación, persistencia).]*

El modelo conceptual utilizado para el desarrollo de la solución representa la estructura estándar mediante la cual se implementa un plugin dentro del ecosistema Flamapy. En este caso, dicho modelo ha sido utilizado para representar un generador automático de modelos de características expresados en UVL, definiendo las entidades necesarias para configurar y ejecutar el proceso de generación. La solución se ha desarrollado siguiendo el paradigma de programación orientada a objetos de Python, donde cada entidad del modelo se implementa mediante clases organizadas en archivos con extensión `.py`. Esta estructura permite separar las responsabilidades de los diferentes componentes del plugin, diferenciando la configuración del generador de la operación encargada de producir los modelos de características.

La relación del modelo conceptual con el resto del sistema se establece mediante su integración con UVLHub. La clase `GenerateFeatureModel` constituye el punto principal de entrada para la operación de generación y es invocada desde el módulo *generator* de UVLHub cuando el usuario solicita la creación de nuevos modelos de características. Antes de realizar dicha llamada, la plataforma UVLHub se encarga de validar la configuración proporcionada por el usuario, actuando como una barrera que evita que la herramienta generadora reciba parámetros inválidos o incoherentes. En cuanto a la persistencia, el sistema no requiere un mecanismo de almacenamiento permanente, ya que está diseñado para permitir al usuario visualizar

o descargar los modelos generados inmediatamente después de cada ejecución y comenzar posteriormente una nueva generación con una configuración diferente.

5.1.1 Estructura del modelo

[Describe la jerarquía o estructura principal del modelo. Si hay un árbol, descríbelo de raíz a hojas. Si hay entidades, descríbelas con sus relaciones.]

Modelo conceptual del dominio – Generador de modelos de características UVL

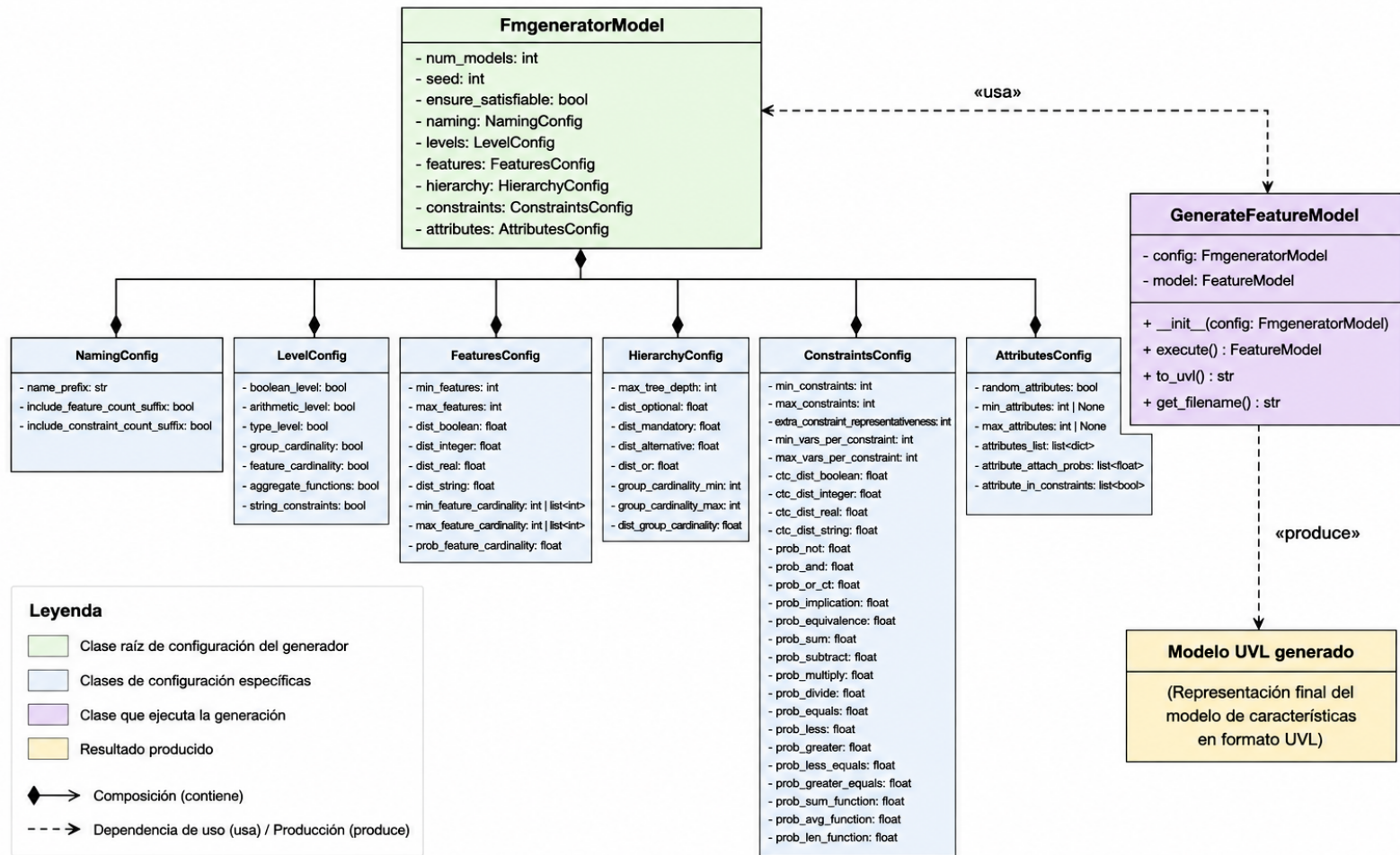


Figura 5.1: Modelado conceptual del sistema. Elaboración propia

El modelo conceptual representa la estructura interna del plugin desarrollado siguiendo el modelo de extensión definido por Flamapy. En este caso, dicho modelo ha sido adaptado para representar un generador automático de modelos de características expresados en UVL, separando la configuración del generador, la operación encargada de ejecutar la generación y el resultado producido.

Las entidades principales que componen el modelo son las siguientes:

FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador de modelos de características. Actúa como entidad raíz del modelo conceptual y agrupa las diferentes configuraciones necesarias para definir las características del modelo que se desea generar, incluyendo aspectos relacionados con la nomenclatura, los niveles de expresividad UVL, la estructura jerárquica, las restricciones y los atributos. Esta entidad constituye la representación completa del estado del generador antes de iniciar el proceso de generación.

NamingConfig

La entidad **NamingConfig** representa la configuración relacionada con la generación de nombres de los elementos creados durante el proceso de generación. Su finalidad es establecer las reglas utilizadas para definir la nomenclatura de los modelos y sus elementos asociados.

LevelConfig

La entidad **LevelConfig** representa la configuración de los niveles de expresividad disponibles en el lenguaje UVL durante la generación. Esta entidad permite definir las capacidades del lenguaje que estarán habilitadas en los modelos generados, estableciendo las construcciones que podrán aparecer en el resultado final.

FeaturesConfig

La entidad **FeaturesConfig** representa la configuración asociada a la generación de características del modelo. Su responsabilidad es definir las condiciones que controlan la creación de los elementos principales que forman parte de la estructura del modelo de características.

HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo de características generado. Esta entidad define los parámetros necesarios para construir el árbol de características y establecer las relaciones jerárquicas entre sus elementos.

ConstraintsConfig

La entidad **ConstraintsConfig** representa la configuración utilizada para la generación de restricciones entre los elementos del modelo. Su objetivo es definir

cómo se crean las relaciones adicionales entre características y atributos, permitiendo generar modelos con diferentes niveles de complejidad y expresividad.

AttributesConfig

La entidad **AttributesConfig** representa la configuración asociada a la generación de atributos vinculados a las características del modelo. Esta entidad permite establecer cómo se incorporan atributos al modelo generado, tanto mediante generación automática como mediante configuraciones definidas manualmente.

GenerateFeatureModel

La entidad **GenerateFeatureModel** representa la operación encargada de ejecutar el proceso de generación automática de modelos de características. Esta clase constituye el punto de entrada principal del generador y utiliza la configuración definida en **FmgeneratorModel** para producir modelos de características expresados en UVL. Dentro de la arquitectura del sistema, esta operación es invocada desde el módulo *generator* de UVLHub para iniciar el proceso de generación.

Modelo UVL generado

La entidad **Modelo UVL generado** representa el resultado producido tras la ejecución del proceso de generación. No constituye una entidad persistente dentro del modelo conceptual del generador, sino la representación final del modelo de características expresado en lenguaje UVL, preparado para su posterior utilización dentro del ecosistema de herramientas de variabilidad software.

5.1.2 Restricciones del dominio

[Restricciones que el modelo no captura por estructura y que se expresan como reglas adicionales (rangos, dependencias, exclusiones, fórmulas...). Justifica brevemente cada una desde el punto de vista del dominio.]

- **[Restricción 1]:** *[descripción y justificación.]*
- **[Restricción 2]:** *[descripción y justificación.]*
- **[Restricción 3]:** *[descripción y justificación.]*
- **Dependencia entre niveles de expresividad UVL:** El nivel booleano no puede no existir porque no tiene sentido debido a ser el nivel más básico de los modelos, al igual que no puede no existir el Type Level sin el Arithmetic Level, ya que para crear *constraints* con los tipos proporcionados por este nivel hace

falta el Arithmetic Level para procesarlos. A este respecto, hablamos tanto de existencia (a nivel conceptual) como también se comentó anteriormente de activación (a nivel técnico y funcional)

- **Validez estructural de los modelos de características generados:** A nivel conceptual, no se puede determinar como tal la coherencia o validez de los modelos generados. Para poder determinarla, ya existen los motores de razonamiento actuales, como SAT, para modelos únicamente con Boolean Level. Es objetivo de este proyecto proporcionar modelos con mayor expresividad (es decir, que incluyan Arithmetic y Type level) para poder diseñar motores de razonamiento que permitan evaluar la validez de modelos con estos niveles de expresividad.
- **Reproducibilidad mediante el uso de semillas:** La semilla de aleatoriedad la debe elegir el usuario. Esto se debe a que si se deja que el sistema la elija, debido a que los sistemas en los que trabajamos hoy día son deterministas, en algún momento acabarán usando una misma semilla de nuevo debido a la pseudoaleatoriedad [21]. Aunque no sea un problema mayor, se ha tomado esta decisión para enmendar esta restricción, que si se considera desde otro enfoque, puede ser que sea más técnica que conceptual, pero igualmente implica añadir un atributo *seed* a la clase *FmgeneratorModel*.

5.1.3 Espacio de soluciones / configuraciones

[Si tiene sentido, calcula o estima el tamaño del espacio de soluciones válidas (combinatoria de opciones, rango continuo de parámetros, etc.). Es útil para ilustrar la magnitud del problema.]

El espacio de configuraciones del generador está determinado por los valores que pueden tomar los diferentes parámetros configurables de la clase *FmgeneratorModel*. Para calcular el número de configuraciones posibles sería necesario revisar los límites entre los cuales puede variar cada parámetro. Debido a que algunos de ellos no presentan un límite superior establecido, como ocurre con el número de características, además de que existen parámetros numéricos reales, el espacio de configuraciones posibles es ilimitado.

Asimismo, al tratarse de un generador basado en mecanismos aleatorios y teniendo en cuenta que la semilla utilizada para controlar dicha aleatoriedad puede tomar un valor positivo cualquiera, el espacio de soluciones generado también es ilimitado, lo cual permite obtener modelos de características con estructuras y niveles de expresividad completamente variados.

5.2 Arquitectura del sistema

5.2.1 Visión general

[Describe la arquitectura de alto nivel: tipo de aplicación (monolito, microservicios, librería, CLI...), patrón principal (capas, hexagonal, MVC...) y las tecnologías centrales.]

[Lista de capas habituales:]

- *[Capa de presentación: interfaz visible al usuario.]*
- *[Capa de API / controladores: coordinación de peticiones y respuestas.]*
- *[Capa de dominio / lógica: reglas de negocio, validación, generación.]*
- *[Capa de persistencia: acceso a base de datos o almacenamiento.]*
- *[Capa de infraestructura: contenedores, despliegue, CI/CD.]*

[Si usas un modelo de documentación arquitectónica concreto (C4, 4+1, ATAM...), nómbralo aquí y comenta qué niveles incluye.]

[(Espacio reservado para los diagramas de contexto y de contenedores, no incluidos en esta plantilla.)]

La arquitectura diseñada organiza los diferentes componentes del sistema en capas lógicas, separando las responsabilidades asociadas a la interacción con el usuario, la gestión de la configuración, la generación de modelos y la infraestructura necesaria para su ejecución.

La **capa de presentación** está formada por la interfaz web proporcionada por UVLHub, mediante la cual el usuario configura los diferentes parámetros del generador. Esta interfaz sigue un diseño basado en un flujo tipo *wizard*, permitiendo organizar la configuración del modelo en diferentes pasos y facilitar la comprensión del proceso por parte del usuario.

La **capa de API/controladores** está formada principalmente por las *routes* del módulo *generator* de UVLHub. Estas funciones se encargan de gestionar la comunicación entre los diferentes pasos del flujo de configuración, enviando y recibiendo los parámetros introducidos por el usuario hasta completar la configuración necesaria para ejecutar el proceso de generación.

La **capa de dominio/lógica** está formada por la lógica interna del plugin generador desarrollado siguiendo la arquitectura de extensiones de Flamapy, junto con los servicios encargados de gestionar la configuración dentro de UVLHub. Por un lado, el plugin contiene el modelo de configuración y la lógica necesaria para producir modelos de características expresados en UVL. Por otro lado, los servicios

se encargan de reconstruir y validar los parámetros recibidos entre los diferentes pasos del generador mediante las *routes*, garantizando que la información utilizada durante la generación sea coherente.

La **capa de infraestructura** está formada por los elementos necesarios para la ejecución y despliegue de la aplicación. Para el desarrollo local se ha utilizado Docker junto con sus correspondientes contenedores y configuraciones, permitiendo construir y levantar el entorno completo de la aplicación de forma reproducible. Además, se ha utilizado Render como plataforma de despliegue principalmente para facilitar al tutor la revisión del funcionamiento de la aplicación durante el desarrollo del proyecto. Asimismo, esta capa incluye el entorno de ejecución basado en WebAssembly mediante Pyodide, utilizado para ejecutar el generador directamente en el navegador del usuario y cargar las dependencias necesarias para su funcionamiento.

No existe una **capa de persistencia** en la arquitectura diseñada, debido a que los modelos generados no se almacenan de forma permanente. El sistema está orientado a que el usuario pueda visualizar o descargar los modelos generados tras cada ejecución y comenzar posteriormente una nueva generación con una configuración diferente.

5.2.2 Diagrama de componentes

[Describe cómo se distribuyen los módulos del sistema entre las capas y cómo se comunican entre sí. Indica responsabilidades por módulo.]

[(Espacio reservado para el diagrama de capas / componentes.)]

El diagrama de componentes mostrado en la Figura ?? representa la distribución de los principales módulos del sistema y la comunicación existente entre ellos.

6. [Paso 6.]

[Comenta también qué ocurre en caso de error (el flujo se interrumpe, se devuelve un código de error, se notifica al usuario...)].

1. **Inicio de la configuración:** El usuario accede al generador mediante la interfaz basada en *wizard* proporcionada por UVLHub y comienza a introducir los parámetros necesarios para definir las características del modelo que desea generar.

2. **Comunicación entre los diferentes pasos del *wizard*:** Las *routes* del módulo *generator* reciben los parámetros introducidos por el usuario en cada paso del flujo de configuración y se encargan de enviarlos al siguiente paso junto con la información necesaria para continuar con la configuración del generador.

En este punto ya existen validaciones implementadas en el propio frontend que se encargan de realizar comprobaciones básicas, por ejemplo que la suma de las distribuciones sea exactamente 1, entre otras.

3. **Reconstrucción y validación de la configuración:** Tras cada *step*, los servicios internos de UVLHub reconstruyen la configuración completa del generador a partir de los parámetros recibidos y realizan las validaciones necesarias antes de pasar al siguiente *step*.

En caso de que alguno de los parámetros proporcionados no cumpla las reglas de negocio establecidas, el sistema impide al usuario pasar al siguiente *step* hasta que el usuario corrija los parámetros inválidos, indicados mediante mensajes de error.

4. **Ejecución del generador:** Si la configuración es válida, en el sexto y último *step* se instancia la operación *GenerateFeatureModel* perteneciente al plugin y se ejecuta su método *execute()*, utilizando la configuración almacenada en la clase *FmgeneratorModel*.

Si se produce algún error durante el proceso de generación, la ejecución se detiene y se informa al usuario de que no ha sido posible completar la generación del modelo.

5. **Carga de dependencias y ejecución mediante Pyodide:** La generación se realiza dentro del entorno proporcionado por Pyodide y WebAssembly en el navegador del usuario. Para ello, previamente se cargan las dependencias necesarias del generador, permitiendo ejecutar la lógica del plugin directamente en el cliente.

6. **Visualización y descarga del resultado:** Una vez generado correctamente el modelo UVL, este se devuelve a la interfaz de UVLHub, permitiendo al usuario visualizarlo en la plataforma o descargarlo para su posterior utilización.

5.3 Patrones de diseño aplicados

[Enumera los patrones de diseño (creacionales, estructurales, de comportamiento o arquitectónicos) que se han aplicado en la solución. Para cada uno indica qué problema resuelve, dónde se aplica en el sistema y por qué se eligió frente a alternativas.]

[Ejemplos habituales en TFGs de Ingeniería del Software:]

- **[Patrón 1 (p. ej. MVC, Repositorio, Singleton, Factory...)]:** *[problema que resuelve, módulo o capa donde se aplica, alternativas descartadas.]*
- **[Patrón 2]:** *[descripción y justificación.]*
- **[Patrón 3]:** *[descripción y justificación.]*

[Si la arquitectura sigue un patrón global (capas, hexagonal, MVC, microservicios...), descríbelo aquí o referencia la sección de arquitectura donde se haya tratado.]

- **Arquitectura por capas (*Layered Architecture*):** La solución sigue una organización basada en capas lógicas, separando las responsabilidades asociadas a la interfaz de usuario, comunicación con el sistema, lógica de generación e infraestructura. Este patrón permite reducir el acoplamiento entre componentes y facilitar la evolución del sistema, permitiendo modificar una capa sin afectar al resto siempre que se mantengan sus interfaces de comunicación.
- **Arquitectura basada en plugins (*Plugin Architecture*):** El generador desarrollado sigue el mecanismo de extensiones proporcionado por Flamapy, encapsulando la lógica de generación dentro de un componente independiente. Esta decisión permite integrar nuevas funcionalidades dentro del framework sin modificar su núcleo, favoreciendo la reutilización y el mantenimiento del sistema.
- **Patrón de interacción *Wizard*:** La interfaz de configuración del generador utiliza un patrón de interacción basado en *wizard*, mediante el cual la introducción de parámetros se divide en diferentes pasos independientes. Este patrón permite organizar una gran cantidad de opciones de configuración de forma progresiva, facilitando la comprensión del usuario y permitiendo realizar validaciones durante el proceso de configuración. Además, se incorporó un panel

de progreso que permite consultar en todo momento el estado actual de la configuración realizada.

5.4 Diseño de la interfaz de usuario

[Si el sistema tiene interfaz gráfica (web, móvil, escritorio), dedica una sección al diseño de la experiencia de usuario. No basta con decir “se ha hecho bonito”; conviene documentar las decisiones tomadas.]

5.4.1 Mapa de navegación

[Diagrama o lista jerárquica de las pantallas/vistas del sistema y los caminos posibles entre ellas. Sirve para que el lector se haga una idea de la extensión funcional sin recorrerse el sistema en vivo.]

[Listado de vistas:]

- *[Vista 1 — propósito y enlaces salientes.]*
- *[Vista 2 — propósito y enlaces salientes.]*
- *[Vista 3 — propósito y enlaces salientes.]*

El mapa de navegación del generador sigue una estructura secuencial basada en un flujo tipo *wizard*, donde la configuración del modelo de características se divide en diferentes etapas independientes. Esta organización permite separar los diferentes grupos de parámetros configurables según su funcionalidad, facilitando al usuario la definición progresiva de la configuración necesaria para realizar la generación del modelo.

Las vistas principales del sistema son las siguientes:

Step 1: Batch

Esta vista permite establecer los parámetros generales del proceso de generación, incluyendo el número de modelos a generar, la semilla utilizada para controlar la aleatoriedad y la nomenclatura de los modelos generados.

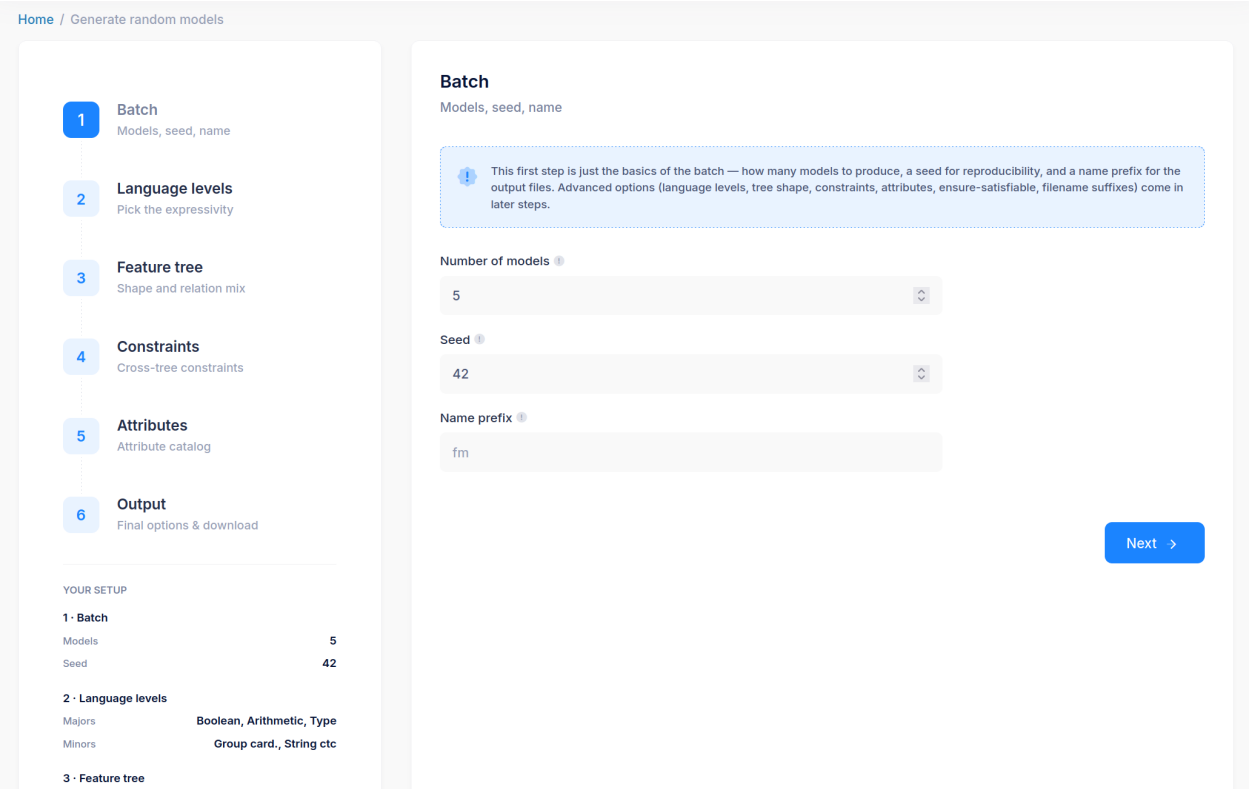


Figura 5.3: Interfaz del Step 1: Batch. Elaboración propia

Step 2: Language levels

Esta vista permite seleccionar los niveles de expresividad UVL que estarán disponibles durante la generación, incluyendo la activación de los diferentes niveles principales y secundarios soportados.

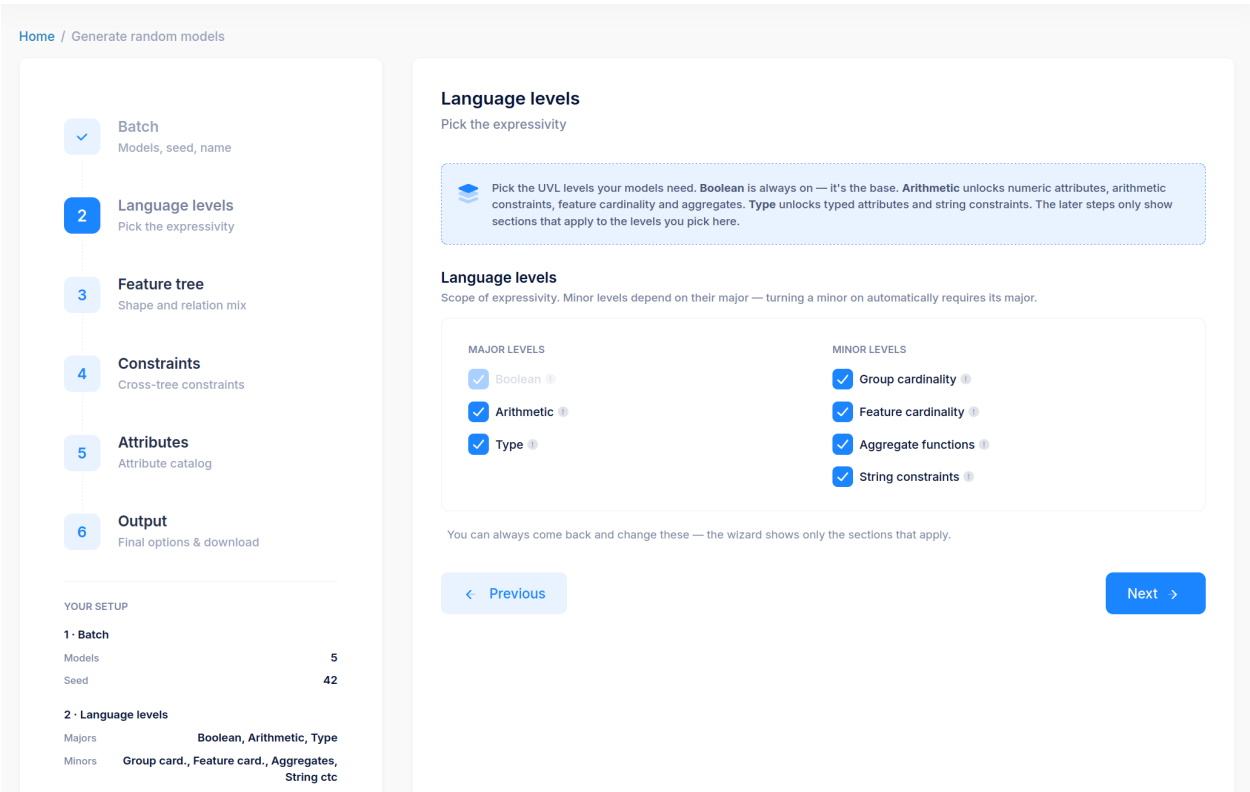


Figura 5.4: Interfaz del Step 2: Language levels. Elaboración propia

Step 3: Feature tree

Esta vista permite configurar los parámetros relacionados con la generación de la estructura del árbol de características, incluyendo el número de características, la profundidad máxima y la distribución de relaciones jerárquicas.

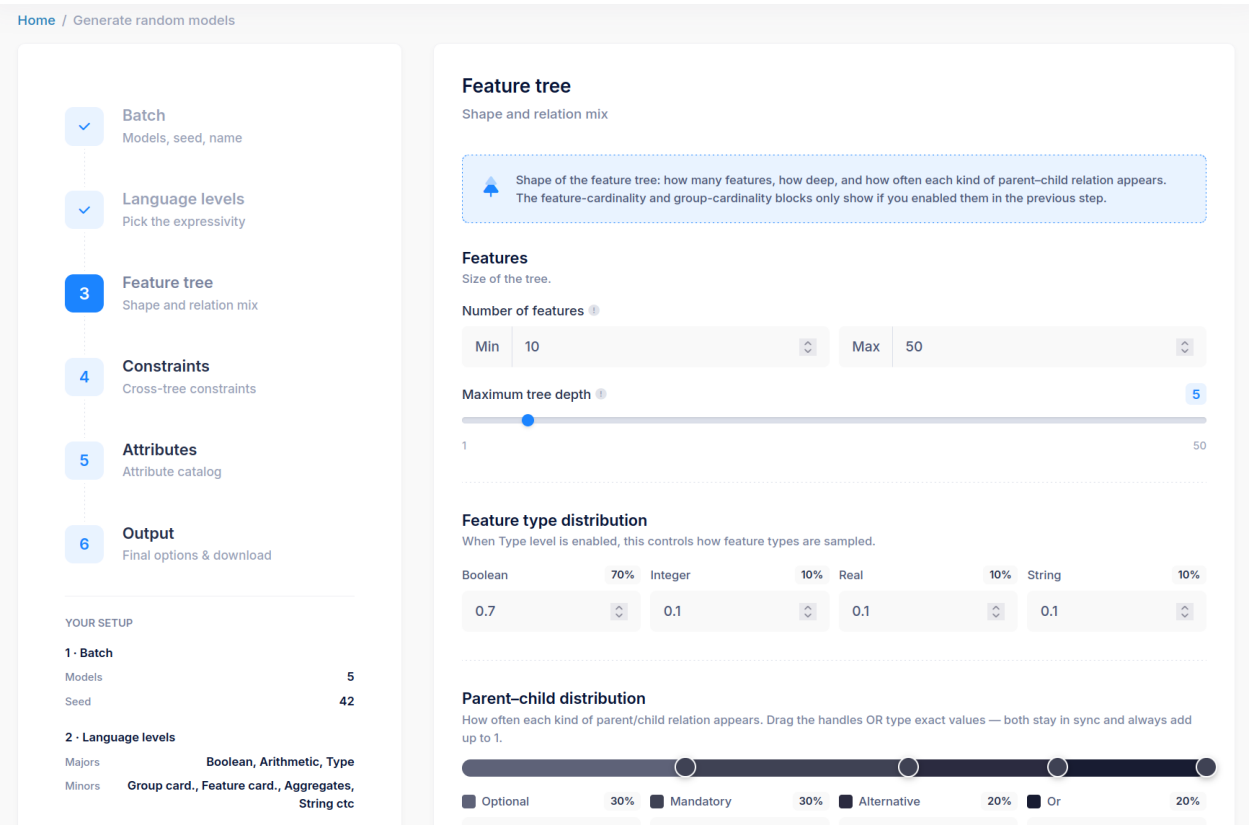


Figura 5.5: Interfaz del Step 3: Feature tree. Elaboración propia

1 · Batch

Models5

Seed42

2 · Language levels

MajorsBoolean, Arithmetic, Type

MinorsGroup card., Feature card., Aggregates, String etc

3 · Feature tree

Features10 – 50

Depth ≤5

Rel mixopt 30% · mand 30% · alt 20% · or 20%

Group card.[1..6]

4 · Constraints

Count1 – 10

Vars/ctc1 – 10

Type mixB 100%

Parent-child distribution

How often each kind of parent/child relation appears. Drag the handles OR type exact values — both stay in sync and always add up to 1.

Optional30%

Mandatory30%

Alternative20%

Or20%

0.3000

0.3000

0.2000

0.2000

Group cardinality0%

0.0000

Feature cardinality ⓘ

Attach probability ⓘ

0.0

Min instances

2

Max instances

5

Min must be ≤ Max.

Group cardinality ⓘ

Children selectable per group ⓘ

Min1

Max6

< Previous

Next >

Figura 5.6: Interfaz del Step 3: Feature tree. Elaboración propia

Step 4: Constraints

Esta vista permite definir los parámetros asociados a la generación de restricciones entre características y atributos, incluyendo la distribución de los diferentes tipos de restricciones y operadores disponibles.

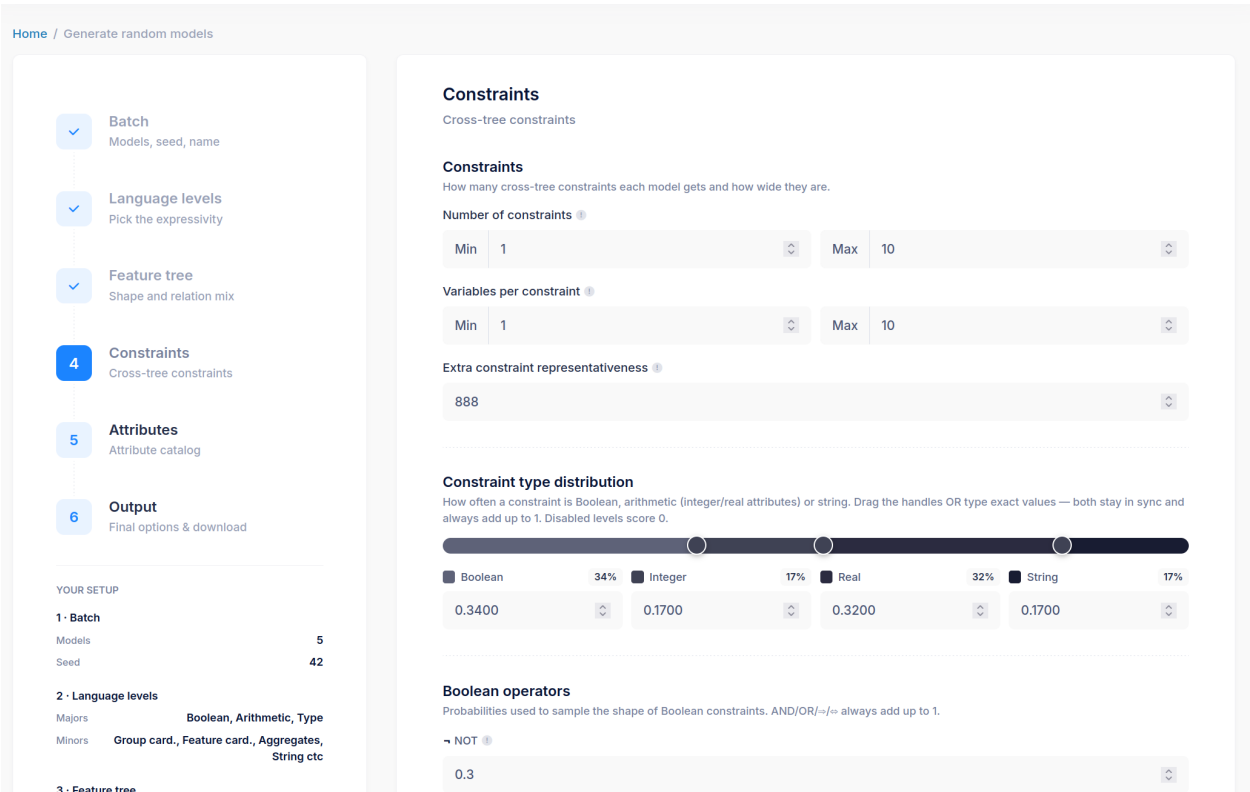


Figura 5.7: Interfaz del Step 4: Constraints. Elaboración propia

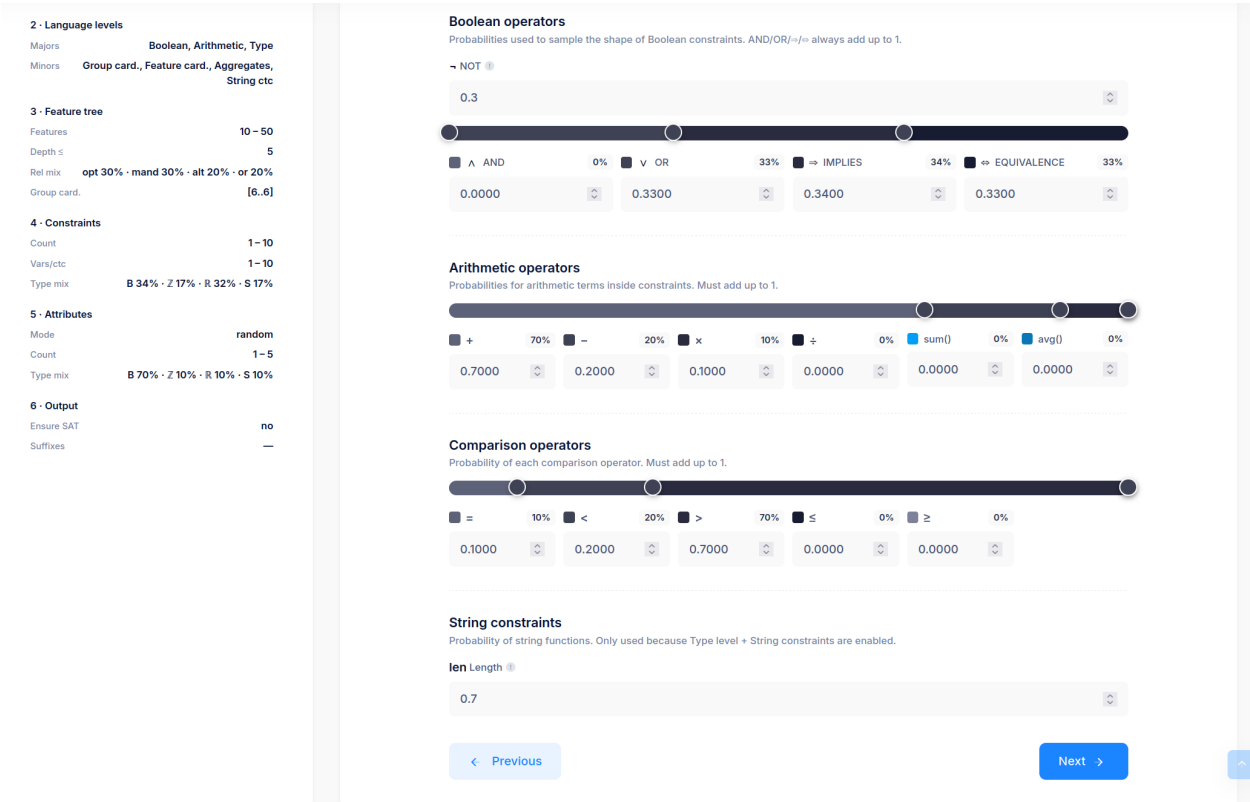


Figura 5.8: Interfaz del Step 4: Constraints. Elaboración propia

Step 5: Attributes

Esta vista permite configurar los parámetros relacionados con la generación de atributos asociados a las características, incluyendo la posibilidad de generar atributos aleatorios mediante una distribución de tipos configurable.

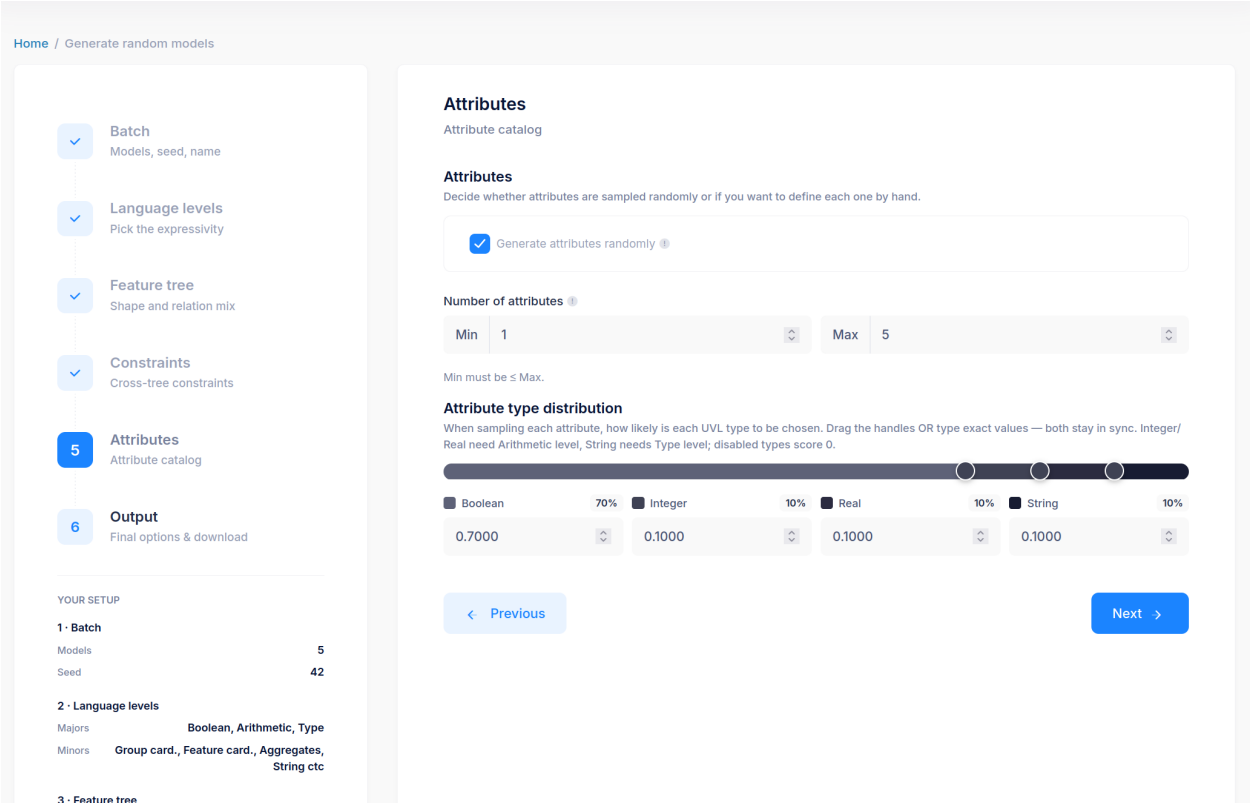


Figura 5.9: Interfaz del Step 5: Attributes. Elaboración propia

Home / Generate random models

Attributes
Attribute catalog

Attributes
Decide whether attributes are sampled randomly or if you want to define each one by hand.

☐ Generate attributes randomly ⓘ

There are 1 attribute card(s) with missing required fields. Complete them before continuing.

+ Add attribute

Attribute #1 Remove

Name: e.g. Seats Type: Integer

Min: Max:

Attach probability:

☒ Use in constraints

< Previous Next >

YOUR SETUP

1 - Batch

Models: 5
Seed: 42

2 - Language levels

Majors: Boolean, Arithmetic, Type
Minors: Group card., Feature card., Aggregates, String etc

3 - Feature tree

Features: 10 - 50

Figura 5.10: Interfaz del Step 5: Attributes. Elaboración propia

Step 6: Output

Esta vista permite ejecutar el proceso de generación del modelo UVL utilizando la configuración proporcionada en los pasos anteriores. Además, permite configurar opciones adicionales como la satisfacibilidad (únicamente para nivel booleano) y los sufijos utilizados en los archivos generados.

Una vez finalizada la generación, el usuario puede visualizar el modelo obtenido dentro de la plataforma o descargarlo para su posterior utilización.

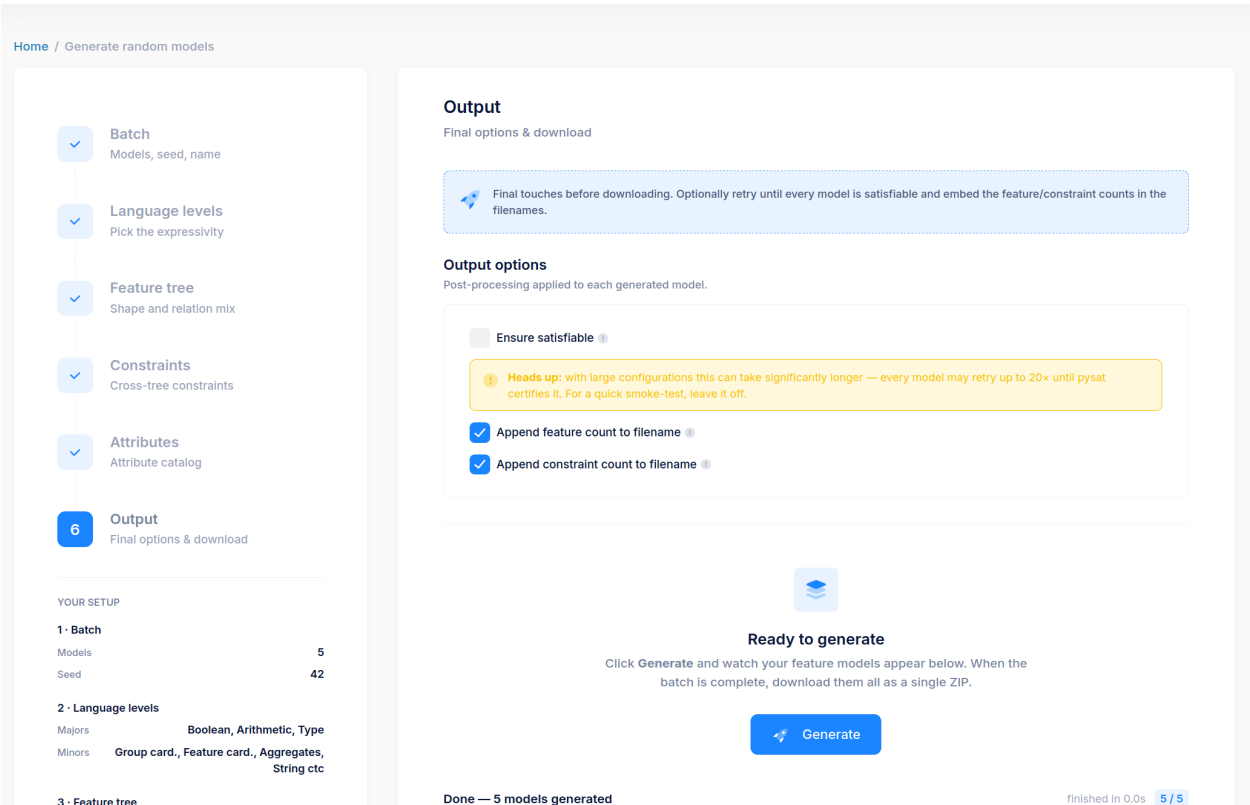


Figura 5.11: Interfaz del Step 6: Output. Elaboración propia

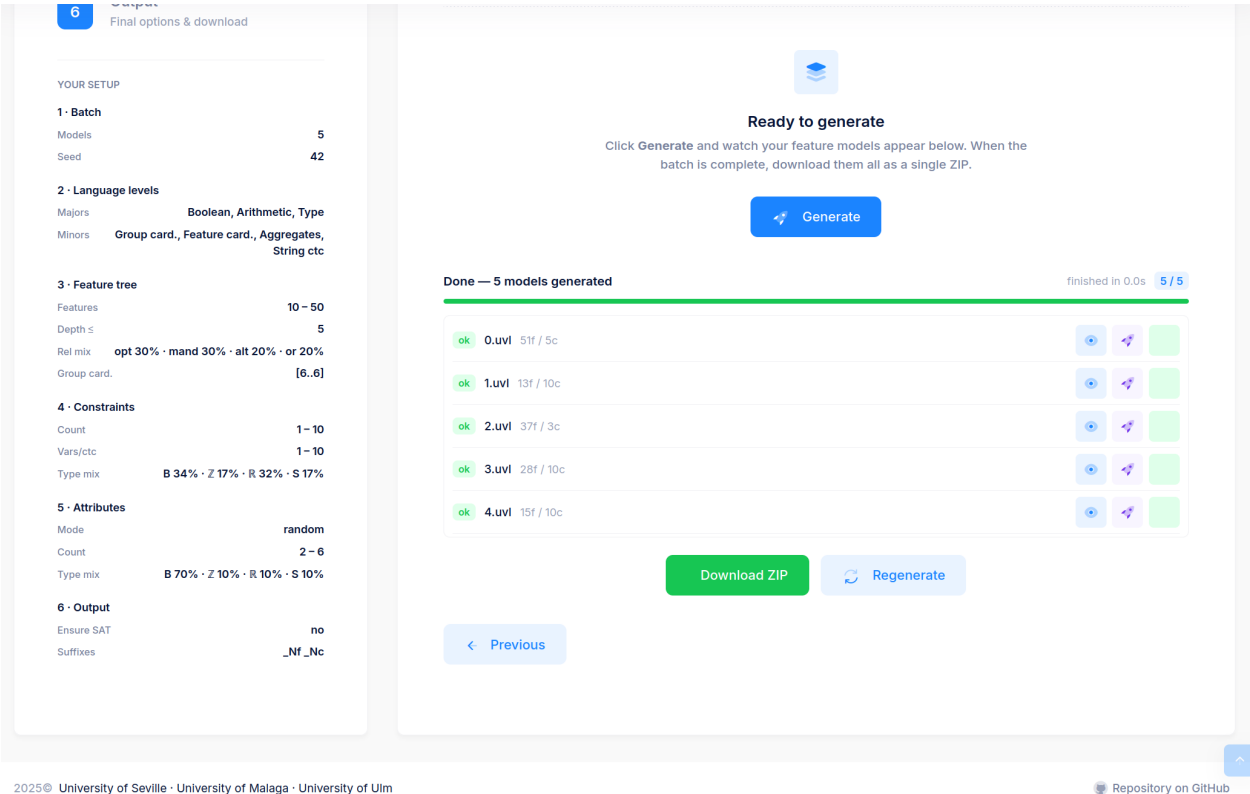


Figura 5.12: Interfaz del Step 6: Output. Elaboración propia

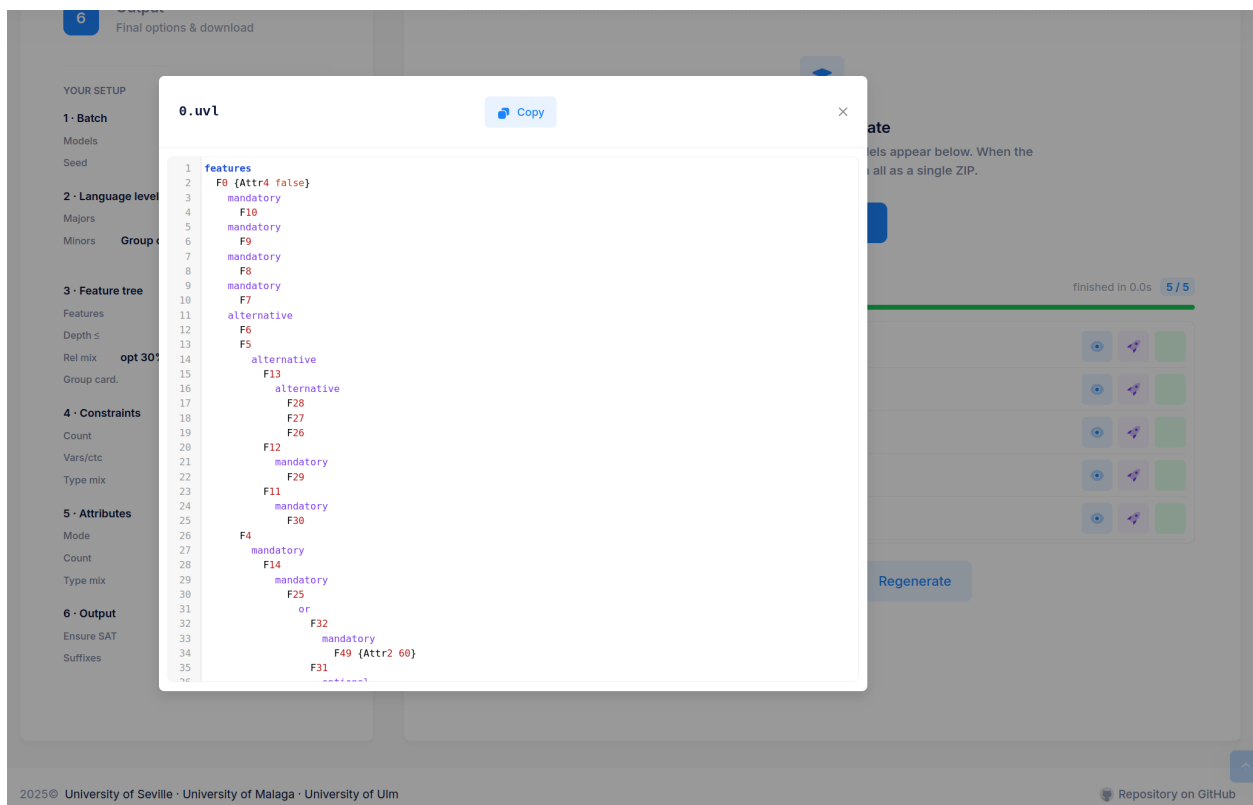


Figura 5.13: Interfaz del Step 6: Output. Elaboración propia

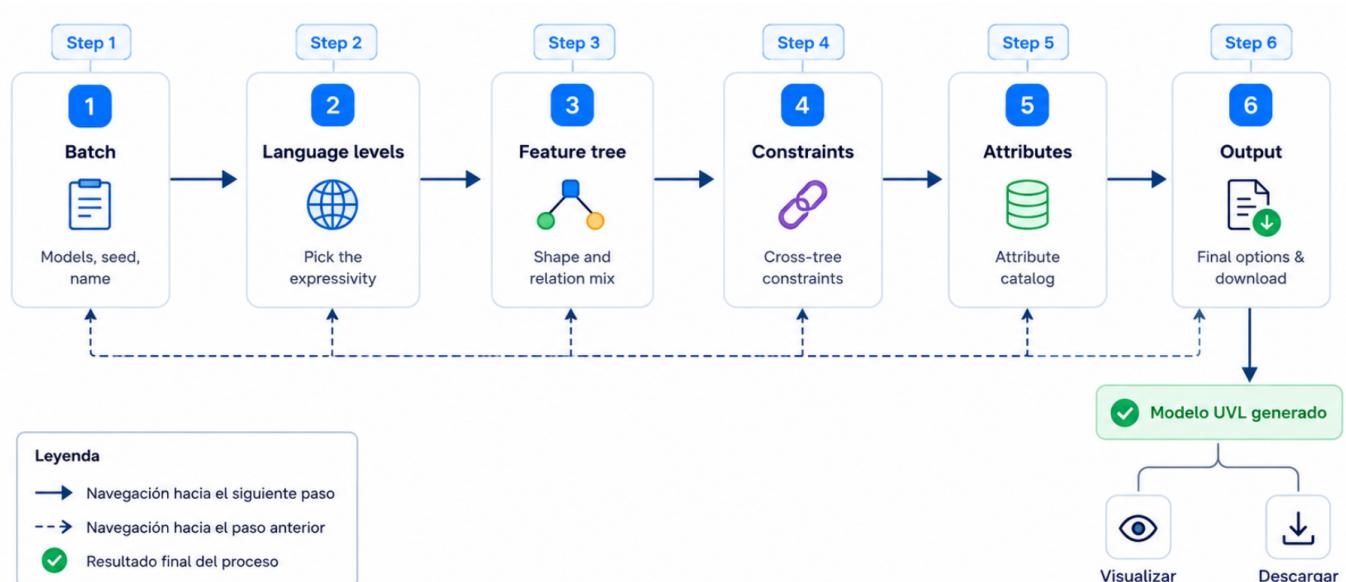


Figura 5.14: Mapa de navegación del generador UVL. Elaboración propia

5.4.2 Wireframes y mockups

[Bocetos de baja fidelidad (wireframes) o prototipos visuales (mockups) de las pantallas principales. Conviene incluir al menos las dos o tres más representativas: la pantalla principal, la pantalla del flujo principal y una pantalla de gestión/listado.]

[(Espacio reservado para los wireframes/mockups, no incluidos en esta plantilla.)]

Durante la fase inicial del desarrollo se realizó un prototipo de la interfaz con el objetivo de cubrir los requisitos iniciales identificados para el sistema. Los *wireframes* mostrados a continuación corresponden a dicho prototipo inicial y no representan completamente la versión final de la interfaz, ya que durante el desarrollo se fueron incorporando diferentes mejoras y modificaciones orientadas a mejorar la usabilidad y adaptar el sistema a las nuevas necesidades surgidas durante el desarrollo.

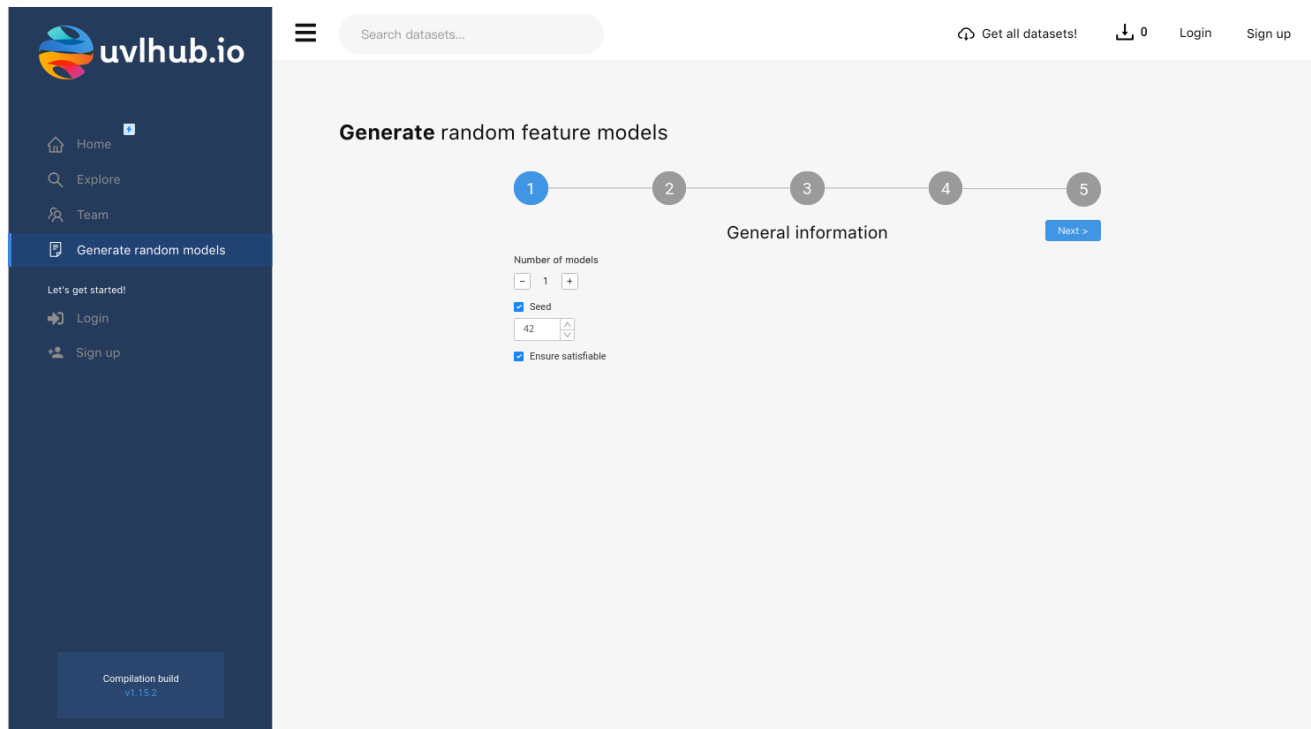


Figura 5.15: Wireframe inicial del Step 1 del generador. Elaboración propia

The image shows a web application interface for generating random feature models. The page is titled "Generate random feature models" and is part of a 5-step process, with step 2 currently active. The interface includes a sidebar with navigation links (Home, Explore, Team, Generate random models) and a main content area with various configuration options.

Navigation and Header:

- uvlhub.io logo
- Search datasets...
- Get all datasets!
- 0 datasets
- Login
- Sign up

Process Flow:

- 1
- 2 (Active)
- 3
- 4
- 5

Configuration Options:

- Features:**
 - Number of features: Select one (dropdown)
 - Boolean: 0.7
 - Integer: 0.2
 - Real: 0.7
 - String: 0.0
- Language level:** Boolean level is selected by default.
 - Arithmetic level (selected)
 - Type level
- Note:** Arithmetic level includes Boolean level, and Type level include all of them.
- Feature cardinality (only if Arithmetic level is selected):**
 - Min: Select one (dropdown)
 - Min: 1, Max: 3
 - Max: Select one (dropdown)
 - Min: 1, Max: 3
 - Attach probability: 0.05
- Maximum tree depth:** 5
- Groups:**
 - Optional: 0.7
 - Mandatory: 0.2
 - Alternative: 0.7
 - Or: 0.0

Figura 5.16: Wireframe inicial del Step 2 del generador. Elaboración propia

The wireframe shows the 'Generate random feature models' interface at Step 3, 'Constraints'. The left sidebar contains the 'uvlhub.io' logo and navigation links: Home, Explore, Team, and 'Generate random models' (highlighted). Below these are links for 'Let's get started!', 'Login', and 'Sign up'. The main content area has a progress bar with five steps; Step 3 is active. The 'Constraints' section includes the following fields:

- Number of models:** A dropdown menu set to 'Select one', with 'Min: 1' and 'Max: 4'.
- Extra constraint representativeness:** A text input field with the value '0.7'.
- Variables per constraint:** A dropdown menu set to 'Select one', with 'Min: 1' and 'Max: 3'.
- Distribution:** Four input fields for 'Boolean: 0.7', 'Integer: 0.2', 'Real: 0.7', and 'String: 0.0'.
- Boolean level:** A row of logical operators: '!', '&', '|', '=>', and '<=>', each followed by a text input field.
- Arithmetic level constraints (only if Arithmetic level is selected):** A row of arithmetic operators: '+', '-', '*', '/', '=', '<', '>', '<=', '>=', 'sum()', and 'avg()', each followed by text input fields.
- Type level constraints (only if Type level is selected):** A single input field for 'len()' with the value '0.7'.

At the bottom left, a box indicates 'Compilation build v1.15.2'. The top right of the main area has links for 'Get all datasets!', a download icon with '0', 'Login', and 'Sign up'.

Figura 5.17: Wireframe inicial del Step 3 del generador. Elaboración propia

The wireframe shows the 'Generate random feature models' interface at Step 4, 'Attributes'. The layout is consistent with Step 3. The progress bar shows Step 4 as the active step. The 'Attributes' section includes the following fields:

- Add / Delete last one:** Two buttons at the top of the attribute list.
- Attribute 1:**
 - Name:** Text input field with 'Seats'.
 - Type:** Dropdown menu set to 'Integer'.
 - Value:** Dropdown menu set to 'Select one'.
 - Min:** '1' and **Max:** '3'.
 - Attach probability:** Text input field with '0.7'.
 - Use in constraints:** A checked checkbox.
- Attribute 2:**
 - Name:** Text input field with 'License Plate'.
 - Type:** Dropdown menu set to 'String'.
 - Length:** Dropdown menu set to 'Select one'.
 - Min:** '1' and **Max:** '3'.
 - Attach probability:** Text input field with '0.7'.

The 'Finish' button is visible at the end of the progress bar. The top right navigation links remain the same.

Figura 5.18: Wireframe inicial del Step 4 del generador. Elaboración propia

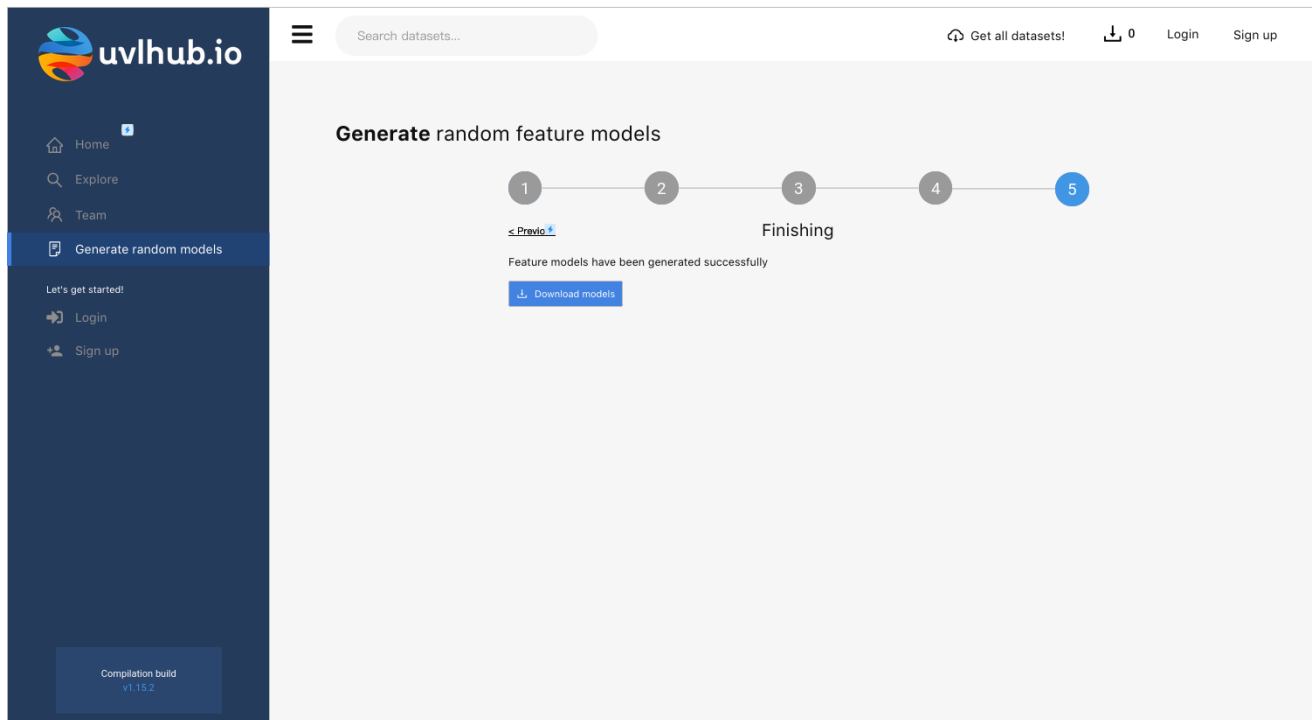


Figura 5.19: Wireframe inicial del Step 5 del generador. Elaboración propia

5.4.3 Decisiones de diseño visual

[Comenta brevemente las decisiones globales de la interfaz:]

- **[Paleta de colores]:** *[principales, secundarios, accesibilidad de contraste.]*
- **[Tipografía]:** *[familias usadas y por qué.]*
- **[Sistema de diseño / framework CSS]:** *[si se ha usado uno (Bootstrap, Tailwind, Material...) o se han creado componentes propios.]*
- **[Responsive / accesibilidad]:** *[breakpoints, criterios de accesibilidad seguidos (contraste, navegación por teclado, ARIA...)]*
- **Paleta de colores:** La interfaz mantiene la identidad visual proporcionada por UVLHub, utilizando principalmente tonos azules, blancos y grises para los diferentes elementos de la interfaz. Además, se utilizan colores específicos para representar diferentes estados del sistema, empleando rojo para mensajes de error, amarillo anaranjado para avisos y verde para operaciones completadas correctamente.
- **Tipografía:** Se mantiene la tipografía utilizada por UVLHub, basada en las familias *Inter*, *sans-serif* y *Helvetica*, utilizadas como estándares dentro de la plataforma, conservando la coherencia visual con el resto de la aplicación y proporciona una lectura clara de los elementos de la interfaz.

- **Organización de la interfaz y flujo de configuración:** Debido a la cantidad de parámetros necesarios para configurar el generador, se decidió utilizar un diseño basado en un flujo tipo *wizard*, separando la configuración en diferentes pasos independientes. Esta decisión permite reducir la complejidad percibida por el usuario y mejorar la comprensión del proceso de configuración.

Además, se incorporó un panel lateral de progreso que permite consultar en todo momento el estado actual del proceso, los pasos completados y un resumen de la configuración seleccionada sin necesidad de volver atrás durante el flujo.

- **Componentes visuales y sistema de diseño:** La interfaz utiliza los componentes proporcionados por el sistema de diseño utilizado en UVLHub, basado principalmente en Bootstrap. Además, se desarrollaron componentes específicos para el generador, como el selector de distribuciones probabilísticas utilizado para configurar la generación de modelos, permitiendo adaptar la interfaz a las necesidades particulares del sistema.



Figura 5.20: Componente selector de distribuciones. Elaboración propia

- **Ayudas visuales y gestión de la información:** Para facilitar la configuración de un sistema con una gran cantidad de parámetros, se decidió mostrar únicamente las opciones relacionadas con los niveles de expresividad seleccionados por el usuario. También se incorporaron textos explicativos y *tooltips* en determinados parámetros con el objetivo de proporcionar contexto adicional sobre su funcionamiento.
- **Responsive y adaptación visual:** Aunque la aplicación está orientada principalmente a su uso desde equipos de escritorio, la interfaz se ha diseñado utilizando una estructura adaptable que permite su correcta visualización en diferentes tamaños de pantalla.

5.5 Esquema de base de datos

5.5.1 Modelo entidad-relación

[Describe las entidades persistidas, sus atributos y las relaciones entre ellas. Suele bastar con un par de tablas y una representación simbólica de las relaciones.]

[(Espacio reservado para la representación simbólica de las relaciones, p. ej. una relación de uno a muchos: `entidad_A` $1 \rightarrow N$ `entidad_B`. Descomenta y adapta el bloque comentado del fuente.)]

[

Campo	Tipo de dato	Restricciones
<code>[campo_1]</code>	<code>[Tipo]</code>	<code>[PK / NOT NULL / ...]</code>
<code>[campo_2]</code>	<code>[Tipo]</code>	<code>[...]</code>
<code>[campo_3]</code>	<code>[Tipo]</code>	<code>[...]</code>
<code>[campo_4]</code>	<code>[Tipo]</code>	<code>[...]</code>

Tabla 5.1: Esquema de la tabla `[nombre_tabla_1]`.

]

[

Campo	Tipo de dato	Restricciones
<code>[campo_1]</code>	<code>[Tipo]</code>	<code>[PK / FK / ...]</code>
<code>[campo_2]</code>	<code>[Tipo]</code>	<code>[...]</code>
<code>[campo_3]</code>	<code>[Tipo]</code>	<code>[...]</code>

Tabla 5.2: Esquema de la tabla `[nombre_tabla_2]`.

]

[Comenta cómo se garantiza el aislamiento entre usuarios (filtros por `user_id`, comprobaciones de propiedad, etc.) si aplica.]

En el sistema diseñado no existe un esquema de base de datos asociado al generador, debido a que no se realiza persistencia de los modelos generados ni de las configuraciones utilizadas durante el proceso de generación. Esta decisión de diseño se debe a que la herramienta está orientada a permitir al usuario configurar el generador, obtener los modelos de características resultantes y comenzar inmediatamente después una nueva generación con una configuración diferente.

Los parámetros introducidos durante el flujo de configuración se mantienen únicamente durante la ejecución actual del generador, siendo utilizados para crear

la instancia de configuración de la clase *FmgeneratorModel*. Una vez finalizada la generación, el resultado puede ser visualizado o descargado por el usuario, sin necesidad de almacenarse de forma permanente dentro del sistema.

5.5.2 Migraciones / versionado del esquema

[Si el esquema se versiona (Alembic, Flyway, Liquibase...), explica brevemente la herramienta usada, dónde se almacenan las migraciones y qué comandos se utilizan en desarrollo y en CI.]

Al no haber persistencia de datos, no existen migraciones de ningún tipo.

5.6 Flujo de datos principal

[Describe el flujo de transformación de datos desde la entrada del usuario hasta el artefacto final. Por ejemplo: **Entrada** → **Representación intermedia** → **Artefacto final**.]

El flujo de datos del sistema comienza con la configuración proporcionada por el usuario mediante la interfaz basada en *wizard*. Esta información es enviada entre los diferentes pasos mediante las *routes* del módulo *generator* de UVLHub, utilizando la sesión de Flask para mantener el estado de la configuración durante el proceso, la cual tiene forma de diccionario. Los servicios internos se encargan de reconstruir y validar dicha configuración antes de ejecutar la generación.

Una vez validada la configuración, esta se transforma en una instancia de la clase *FmgeneratorModel*, que es utilizada por la operación *GenerateFeatureModel* para ejecutar el proceso de generación mediante su método *execute()*. Como resultado, se obtienen los modelos de características en formato UVL, que son devueltos al usuario para su visualización o descarga.



Figura 5.21: Flujo de datos principal. Elaboración propia

5.6.1 Estructura de la representación intermedia

[Si el sistema produce una representación intermedia (JSON canónico, IR, AST, fichero de configuración...), describe sus campos principales y referencia el apéndice donde se incluya un ejemplo completo.]

La representación intermedia utilizada durante el flujo de configuración está basada en la *session* de Flask, la cual permite mantener el estado de los parámetros introducidos por el usuario entre los diferentes pasos del *wizard*, almacenando la información de la configuración en forma de diccionario.

Cada *route* del módulo *generator* actualiza la *session* con los nuevos parámetros introducidos en el paso correspondiente, permitiendo que la configuración completa del generador esté disponible antes de ejecutar el proceso de generación. Posteriormente, los servicios internos de UVLHub utilizan esta información para reconstruir la configuración y crear la instancia de la clase `FmgeneratorModel`, que será utilizada por el plugin generador.

```

435 def load_step_state(step: int, defaults: dict):
436     wizard = session.get("wizard", {})
437     saved = wizard.get(str(step), {})
438
439     out = defaults.copy()
440     out.update(saved)
441
442     return out
443
579
580 if has_saved:
581     defaults.update(
582         {
583             "prob_plus": params_dict.get("PROB_SUM", step4_ui_defaults["prob_plus"]),
584             "prob_minus": params_dict.get("PROB_SUBSTRACT", step4_ui_defaults["prob_minus"]),
585             "prob_times": params_dict.get("PROB_MULTIPLY", step4_ui_defaults["prob_times"]),
586             "prob_div": params_dict.get("PROB_DIVIDE", step4_ui_defaults["prob_div"]),
587             "prob_sum": params_dict.get("PROB_SUM_FUNCTION", step4_ui_defaults["prob_sum"]),
588             "prob_avg": params_dict.get("PROB_AVG_FUNCTION", step4_ui_defaults["prob_avg"]),
589             "prob_eq": params_dict.get("PROB_EQUALS", step4_ui_defaults["prob_eq"]),
590             "prob_lt": params_dict.get("PROB_LESS", step4_ui_defaults["prob_lt"]),
591             "prob_gt": params_dict.get("PROB_GREATER", step4_ui_defaults["prob_gt"]),
592             "prob_leq": params_dict.get("PROB_LESS_EQUALS", step4_ui_defaults["prob_leq"]),
593             "prob_geq": params_dict.get("PROB_GREATER_EQUALS", step4_ui_defaults["prob_geq"]),
594             "prob_len": (
595                 params_dict.get("PROB_LEN_FUNCTION", step4_ui_defaults["prob_len"])
596                 if params_dict.get("TYPE_LEVEL") and params_dict.get("STRING_CONSTRAINTS")
597                 else 0.0
598             ),
599         }
600     )
601 else:
602     defaults.update(step4_ui_defaults)
603     if not (params_dict.get("TYPE_LEVEL") and params_dict.get("STRING_CONSTRAINTS")):
604         defaults["prob_len"] = 0.0
605
606 values = load_step_state(4, defaults)

```

Figura 5.22: Ejemplo de uso de la *session*. Elaboración propia

Esta representación no se mantiene de forma permanente, ya que únicamente se utiliza durante la ejecución del flujo de configuración y generación del modelo. Una vez finalizado el proceso, el usuario puede visualizar o descargar los modelos generados, sin necesidad de almacenar la configuración utilizada.

5.6.2 Mapeo entre niveles

[Si el flujo implica transformaciones explícitas (símbolos a valores numéricos, alias a constantes, claves a entidades...), describe los mapeos con ejemplos concretos.]

- $[símbolo_a \rightarrow valor_a]$
- $[símbolo_b \rightarrow valor_b]$

El mapeo principal de datos del sistema se realiza entre la representación intermedia almacenada en la *session* de Flask y el modelo interno de configuración utilizado por el plugin generador de Flamapy. Los parámetros almacenados en forma de diccionario se transforman en atributos de la clase *FmgeneratorModel* y sus clases de configuración asociadas, permitiendo utilizar una representación orientada a objetos durante el proceso de generación.

Algunos ejemplos de este mapeo son:

- *NUM_MODELS* → *num_models*: número de modelos de características que serán generados.
- *TYPE_LEVEL* → *levels.type*: activación de las construcciones asociadas al nivel de tipos del lenguaje UVL.
- *MIN_FEATURES* / *MAX_FEATURES* → *features.min_features* / *features.max_features*: rango utilizado para determinar el número de características del modelo generado.

Una vez construida la instancia de *FmgeneratorModel*, esta es utilizada por la operación *GenerateFeatureModel* para producir el modelo de características final expresado en formato UVL.

5.6.3 Trazabilidad e identificadores

[Si el sistema necesita relacionar artefactos generados con su configuración de origen, explica el esquema de identificadores (UUID, hash, slug...) y dónde se persisten.]

En el sistema no se mantiene trazabilidad persistente entre los modelos generados y la configuración utilizada para generarlos. La información de la configuración únicamente se mantiene durante la ejecución mediante la *session* de Flask.

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Definidos la arquitectura, los modelos y los flujos de datos, el diseño queda cerrado sobre el papel. El Capítulo 6 descende al código: las tecnologías elegidas y cómo cada módulo materializa las decisiones tomadas aquí».]

Con la arquitectura del sistema, los modelos conceptuales, los flujos de datos y las principales decisiones de diseño definidos, queda establecida la solución desde un punto de vista estructural y conceptual. El siguiente capítulo aborda la implementación de dicha solución, detallando las tecnologías utilizadas y cómo los diferentes componentes definidos en este diseño han sido desarrollados para obtener el sistema final.

6 IMPLEMENTACIÓN

*[Este capítulo describe **cómo** se ha materializado el diseño en código: qué tecnologías se han usado, qué módulos se han implementado y qué decisiones concretas se han tomado en cada uno.]*

6.1 Análisis de tecnologías

[Justifica la elección de las tecnologías principales del proyecto. Si has comparado varias opciones, hazlo explícito (es más convincente que afirmar la elegida sin alternativas).]

[Necesidad técnica 1]

[Plantea el problema concreto que motivó la elección (p. ej., procesar archivos en formato X, validar reglas, automatizar Y...) y qué características debe tener la tecnología elegida.]

[Necesidad técnica 2]

[Otra necesidad que orienta una decisión tecnológica.]

[Necesidad técnica 3]

[Otra necesidad que orienta una decisión tecnológica (p. ej., experiencia de usuario, despliegue, rendimiento, mantenibilidad).]

La implementación del sistema requiere la utilización de tecnologías compatibles con la arquitectura existente de UVLHub y con el ecosistema Flamapy. Por ello, la selección de tecnologías se ha realizado teniendo en cuenta principalmente la integración con la plataforma existente, la facilidad de desarrollo y la necesidad de disponer de un entorno reproducible para la ejecución y despliegue del sistema.

Necesidad técnica 1: Integración con la plataforma existente

El generador debe integrarse dentro de UVLHub, una plataforma web existente desarrollada utilizando Flask. Por este motivo, la tecnología utilizada debe permitir mantener la arquitectura actual basada en rutas, servicios y plantillas Jinja2, evitando introducir cambios estructurales innecesarios en la aplicación.

Necesidad técnica 2: Desarrollo del plugin generador

La lógica de generación debe implementarse siguiendo la arquitectura de extensiones proporcionada por Flamapy. Para ello, la tecnología utilizada debe permitir desarrollar un plugin independiente que encapsule la configuración del generador y la operación encargada de producir modelos de características expresados en UVL.

Necesidad técnica 3: Ejecución y despliegue reproducible

El sistema debe disponer de un entorno de ejecución reproducible tanto para el desarrollo local como para facilitar las revisiones del proyecto. Por ello, es necesario utilizar tecnologías que permitan contenerizar la aplicación, gestionar sus dependencias y proporcionar una versión accesible del sistema desplegada en un entorno remoto.

Comparativa de tecnologías del backend

- *[Tecnología A]: [descripción y ventajas/inconvenientes.]*
- *[Tecnología B]: [descripción y ventajas/inconvenientes.]*
- *[Tecnología C]: [descripción y ventajas/inconvenientes.]*
- **Flask:** Framework utilizado por UVLHub sobre el que se ha integrado el generador. Su principal ventaja es la compatibilidad directa con la arquitectura existente, permitiendo reutilizar las rutas, servicios y mecanismos de configuración ya implementados.
- **Django:** Framework web completo con numerosas funcionalidades integradas, como ORM o gestión de usuarios. Sin embargo, su utilización habría requerido adaptar la arquitectura existente de UVLHub, introduciendo una complejidad innecesaria.
- **FastAPI:** Framework orientado principalmente al desarrollo de APIs de alto rendimiento. Aunque ofrece ventajas para servicios desacoplados, no resulta adecuado en este caso al no encajar con la arquitectura basada en Flask utilizada por UVLHub.

Comparativa de tecnologías del frontend

- *[Tecnología A]: [descripción y ventajas/inconvenientes.]*
- *[Tecnología B]: [descripción y ventajas/inconvenientes.]*
- *[Tecnología C]: [descripción y ventajas/inconvenientes.]*

- **Jinja2, JavaScript y Bootstrap:** Tecnologías utilizadas por UVLHub para la construcción de la interfaz web. En particular, Jinja2 constituye el motor de plantillas integrado con Flask, permitiendo generar las vistas de la aplicación utilizando una arquitectura compatible con el backend existente. Esta opción permite incorporar el flujo tipo *wizard* desarrollado para el generador manteniendo la coherencia visual y estructural con el resto de la plataforma.
- **React:** Biblioteca ampliamente utilizada para el desarrollo de interfaces dinámicas basadas en componentes reutilizables. Sin embargo, su utilización habría requerido introducir una arquitectura frontend independiente de la empleada actualmente por UVLHub, aumentando la complejidad de integración del generador.
- **Vue.js:** Framework orientado al desarrollo de interfaces reactivas mediante componentes. Aunque ofrece una solución ligera para construir interfaces modernas, no fue seleccionado debido a que no forma parte de la arquitectura frontend existente en UVLHub.

Análisis de la base de datos

1. *[Opción A]: [justificación.]*
2. *[Opción B]: [justificación.]*

El sistema desarrollado no requiere una base de datos propia, ya que la naturaleza del generador no implica el almacenamiento permanente de configuraciones ni de modelos generados. La información necesaria durante el flujo de configuración se mantiene temporalmente mediante la *session* de Flask, permitiendo ejecutar el proceso completo sin introducir una capa de persistencia adicional, lo que hace innecesario realizar aquí una comparación.

Plataformas de despliegue

- *[Plataforma A]: [descripción y casuística.]*
- *[Plataforma B]: [descripción y casuística.]*
- *[Plataforma C]: [descripción y casuística.]*
- **Render:** Plataforma utilizada para desplegar una versión accesible de UVLHub durante las revisiones del proyecto. Su elección estuvo motivada tanto por la facilidad de configuración e integración con el repositorio del proyecto como por la experiencia previa personal con esta plataforma, evitando la necesidad

de invertir tiempo adicional en el aprendizaje de una nueva tecnología de despliegue.

- **Servicios cloud tradicionales (AWS, Azure, Google Cloud):** Ofrecen una mayor capacidad de configuración y escalabilidad, aunque requieren una gestión más compleja de infraestructura. Debido a que el objetivo del despliegue era facilitar la revisión funcional del sistema y no proporcionar un servicio preparado para producción, no resultaban necesarios para este proyecto.
- **Heroku:** Plataforma orientada al despliegue sencillo de aplicaciones web con una filosofía similar a Render. Aunque habría permitido obtener una solución equivalente, no fue seleccionada al disponer ya de experiencia previa y una configuración establecida con Render.

Tecnologías finales seleccionadas

- **Desarrollo:** *[tecnologías elegidas para backend / frontend.]*
- **Base de datos:** *[motor elegido.]*
- **Contenedor:** *[tecnología de contenerización (Docker, Podman...)]*
- **Despliegue:** *[plataforma final.]*
- **Desarrollo backend:** Python, Flask y Flamapy, utilizando la arquitectura de plugins proporcionada por este último.
- **Desarrollo frontend:** Plantillas Jinja2, JavaScript y Bootstrap, manteniendo la arquitectura visual existente de UVLHub.
- **Persistencia:** No aplica, al no requerirse almacenamiento permanente de configuraciones ni modelos generados.
- **Contenerización:** Docker, utilizado para construir y ejecutar el entorno completo de desarrollo local mediante contenedores.
- **Despliegue:** Render, utilizado para proporcionar una versión accesible del sistema durante las revisiones del proyecto.

6.2 Tecnologías utilizadas

[Tras la justificación, describe brevemente cómo se usa cada tecnología en el proyecto. Una tecnología por párrafo, indicando módulo o archivo en que aparece.]

[Sugerencia de bloques:]

- *[Backend (framework, gestión de dependencias, configuración).]*
- *[Autenticación (estrategia, almacenamiento de credenciales, gestión de sesión o token).]*
- *[Persistencia (ORM, motor, migraciones).]*
- *[Lógica de dominio (módulos en los que se divide).]*
- *[Generación de artefactos / motor de plantillas (si aplica).]*
- *[Capa de presentación (HTML/CSS/JS o framework).]*
- *[Infraestructura de desarrollo (Docker, Compose, Make...).]*
- *[Suite de pruebas (framework, marcadores, comandos).]*
- *[Automatización CI/CD (proveedor y workflows principales).]*

Tras la selección de tecnologías realizada en el apartado anterior, esta sección describe cómo se han utilizado durante la implementación del sistema, indicando los principales componentes en los que aparecen.

Backend y lógica del servidor

La implementación del backend de UVLHub está desarrollada utilizando Python y Flask. Dentro del módulo *generator* se encuentran las rutas encargadas de gestionar el flujo de configuración del generador, implementadas en el archivo `app/modules/generator/routes.py`, así como los servicios internos definidos en `app/modules/generator/services.py`, responsables de reconstruir y validar la configuración introducida por el usuario.

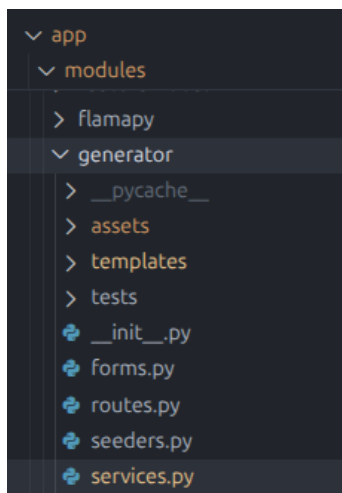


Figura 6.1: Estructura del módulo generator de UVLHub. Elaboración propia

Además, Flask proporciona el mecanismo de *session*, utilizado durante el flujo de configuración para mantener temporalmente el estado de los parámetros del generador entre los diferentes pasos del *wizard*. Esta información es posteriormente utilizada para construir la configuración necesaria antes de iniciar el proceso de generación.

Plugin generador basado en Flamapy

La lógica principal de generación se ha implementado como un plugin compatible con la arquitectura de extensiones proporcionada por Flamapy. Este componente se encuentra dentro del directorio `fm_generator` del proyecto UVLHub y contiene el modelo de configuración del generador mediante la clase *FmgeneratorModel*, junto con las diferentes clases encargadas de representar los parámetros específicos de generación.

La ejecución del proceso se encapsula en la operación *GenerateFeatureModel*, ubicada dentro del módulo de operaciones del plugin, que utiliza la configuración definida en *FmgeneratorModel* para producir modelos de características expresados en formato UVL mediante su método *execute()*.

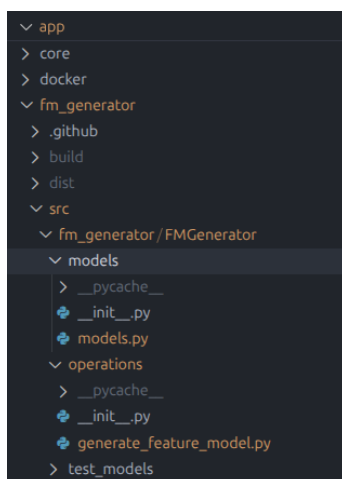


Figura 6.2: Estructura del plugin generador basado en Flamapy. Elaboración propia

Capa de presentación

La interfaz web del generador mantiene la arquitectura frontend existente en UVLHub, utilizando plantillas Jinja2 integradas con Flask para la generación de las vistas, junto con JavaScript y Bootstrap para gestionar la interacción y los componentes visuales de la aplicación. Las diferentes pantallas que forman el flujo tipo *wizard* se encuentran definidas como plantillas HTML dentro del directorio `app/modules/generator/templates/generator`.

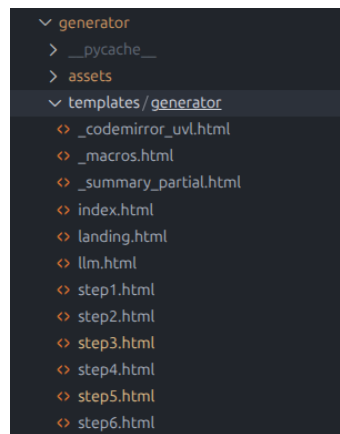


Figura 6.3: Plantillas HTML utilizadas para los pasos del wizard. Elaboración propia

Ejecución mediante Pyodide y WebAssembly

Para permitir la ejecución del generador directamente en el navegador del usuario se ha utilizado Pyodide, un entorno basado en WebAssembly que permite ejecutar código Python en el cliente. Los scripts encargados de configurar Pyodide y cargar las dependencias necesarias para la generación se encuentran en el directorio `app/modules/generator/assets/js`, donde también se incluyen los paquetes *wheel* necesarios para la ejecución del plugin generador.

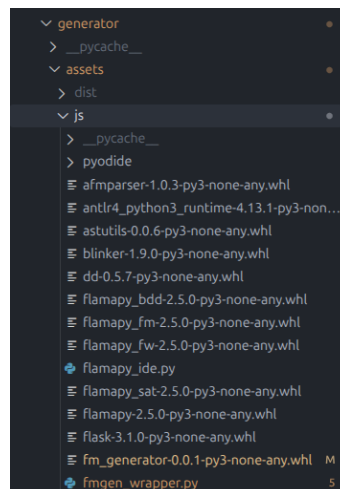


Figura 6.4: Scripts y dependencias utilizadas para la ejecución con Pyodide. Elaboración propia

Infraestructura de desarrollo

Durante el desarrollo local se ha utilizado Docker junto con Docker Compose para definir y ejecutar el entorno completo necesario para UVLHub. Los ficheros de configuración asociados se encuentran en el directorio `docker`, donde se incluyen las definiciones necesarias tanto para los entornos de desarrollo como para los entornos de producción.

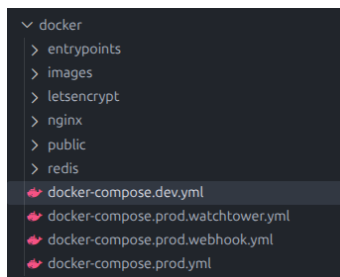


Figura 6.5: Configuración de Docker en UVLHub. Elaboración propia

Aunque el entorno contiene diferentes servicios auxiliares necesarios para el funcionamiento global de UVLHub, el único servicio con una relación directa con el generador es Redis. Este servicio se utiliza durante el despliegue de revisiones para almacenar temporalmente la información asociada a la *session* de Flask, permitiendo mantener la configuración actual del usuario durante el flujo de generación.

Suite de pruebas

Las pruebas del sistema se ejecutan mediante *Rosemary*, una CLI (*Command Line Interface*) incluida dentro del entorno de UVLHub [22].

La suite de pruebas incluye comprobaciones sobre diferentes componentes del sistema, incluyendo pruebas de flujo del *wizard*, validación de parámetros y pruebas de interfaz mediante Selenium, permitiendo verificar el correcto funcionamiento de la interacción del usuario con la aplicación.

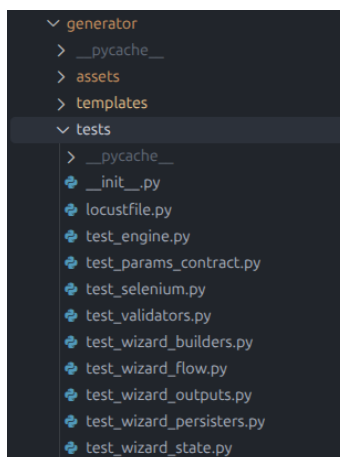


Figura 6.6: Suite de pruebas del módulo generator. Elaboración propia

Automatización CI/CD

Para automatizar las tareas de integración y despliegue se ha utilizado GitHub Actions. Los flujos definidos permiten realizar comprobaciones técnicas previas sobre el código desarrollado antes de completar el despliegue de la aplicación.

Además, se ha configurado la automatización del despliegue, permitiendo actualizar la versión desplegada del sistema de forma automática tras superar los controles establecidos.

6.3 Módulo de validación / lógica principal

[Describe el módulo central de la lógica de negocio. Suele ser el que aplica las reglas del dominio (validador, motor de reglas, planificador...)].

[Cubre:]

- *[Función o clase principal y su contrato (entrada/salida)].*
- *[Estructura del resultado (campos, tipos, semántica)].*
- *[Cómo lo consumen otros módulos (API, generadores...)].*

El módulo de validación se estructura en dos niveles diferenciados. Por un lado, el servicio `GeneratorWizardService` del módulo `generator` realiza una primera validación durante el proceso de configuración, comprobando que los valores introducidos por el usuario cumplen las restricciones básicas necesarias para continuar el flujo.

Por otro lado, el modelo de dominio `FmgeneratorModel` incorpora la validación semántica de la configuración completa del generador, comprobando la coherencia entre sus diferentes parámetros antes de iniciar la generación.

La generación de modelos de características se encapsula en la operación `GenerateFeatureModel`, que recibe una instancia validada de `FmgeneratorModel` y devuelve uno o varios objetos `FeatureModel` compatibles con el metamodelo de FlamaPy. Antes de ejecutar la generación, el método `execute()` realiza una última validación mediante:

```
1 self.model.validate()
```

garantizando que únicamente configuraciones consistentes alcanzan la fase de generación.

6.3.1 Validación sintáctica / comprobaciones básicas

La validación sintáctica se encuentra implementada principalmente en el fichero `services.py`, mediante un conjunto de funciones asociadas a cada paso del asistente de configuración:

```
1 validate_step1_form()  
2 validate_step2_form()  
3 validate_step3_form()  
4 validate_step4_form()
```

```

5 validate_step5_form()
6 validate_step6_form()

```

Estas funciones comprueban que los valores introducidos por el usuario cumplen las restricciones básicas necesarias antes de realizar la validación semántica del modelo de dominio.

Validación de parámetros generales

Durante el primer paso del asistente se validan los parámetros básicos de generación. El número de modelos y la semilla deben cumplir:

$$1 \leq NUM_MODELS \leq 1000$$

$$SEED \in \mathbb{Z}^+$$

Validación de rangos numéricos

Los parámetros asociados a cantidades, límites y tamaños deben mantenerse dentro de rangos válidos. De forma general:

$$VALUE \geq 1$$

Además, todos los pares de límites mínimo-máximo cumplen:

$$MIN_VALUE \leq MAX_VALUE$$

Esta restricción se aplica a parámetros como número de features, número de restricciones, número de atributos o rangos definidos para atributos manuales.

Validación de distribuciones probabilísticas

Las configuraciones probabilísticas utilizadas durante la generación deben representar distribuciones válidas. Para cualquier conjunto de probabilidades asociado a una decisión aleatoria:

$$\forall p_i \in P, \quad 0 \leq p_i \leq 1$$

$$\sum_{p_i \in P} p_i = 1$$

Estas comprobaciones se aplican a las distribuciones de relaciones entre features, operadores booleanos, operaciones aritméticas, tipos de restricciones y tipos de atributos.

Validación de dependencias básicas

Las opciones del asistente presentan dependencias entre niveles de generación que deben cumplirse antes de continuar con la configuración:

$$FEATURE_CARDINALITY \Rightarrow ARITHMETIC_LEVEL$$

$$AGGREGATE_FUNCTIONS \Rightarrow ARITHMETIC_LEVEL$$

$$STRING_CONSTRAINTS \Rightarrow TYPE_LEVEL$$

Validación de atributos manuales

En el modo de definición manual, cada atributo debe cumplir las restricciones básicas de configuración:

$$name(Attribute) \neq \emptyset$$

$$0 \leq attach_probability(Attribute) \leq 1$$

Para atributos con dominio definido mediante rangos:

$$MIN_ATTRIBUTE_VALUE \leq MAX_ATTRIBUTE_VALUE$$

Además, la posibilidad de utilizar un atributo dentro de una restricción depende de los niveles activos del generador:

$$UseInConstraints(Attribute_{Integer|Real}) \Rightarrow Arithmetic_Level$$

$$UseInConstraints(Attribute_{String}) \Rightarrow Type_Level \wedge String_Constraints$$

De esta forma, la validación evita configuraciones incompatibles antes de que sean procesadas por la lógica de generación.

6.3.2 Validación semántica / restricciones de dominio

La validación semántica se encuentra encapsulada en el modelo `FmgeneratorModel`, definido dentro del plugin `fm_generator`. Este modelo agrupa la configuración completa del generador mediante diferentes bloques especializados:

Configuración	Responsabilidad
<code>NamingConfig</code>	Nombres y opciones de salida
<code>LevelConfig</code>	Niveles de expresividad UVL
<code>FeaturesConfig</code>	Parámetros de features
<code>HierarchyConfig</code>	Estructura jerárquica
<code>ConstraintsConfig</code>	Restricciones generadas
<code>AttributesConfig</code>	Gestión de atributos

Tabla 6.1: Bloques de configuración de `FmgeneratorModel`

La validación completa se ejecuta mediante el método:

```
1 FmgeneratorModel.validate()
```

que comprueba la coherencia global de la configuración antes de iniciar la generación.

Coherencia de niveles UVL

Los niveles de expresividad presentan dependencias jerárquicas entre ellos. Las reglas aplicadas son:

$$BOOLEAN_LEVEL = True$$

$$TYPE_LEVEL \Rightarrow ARITHMETIC_LEVEL$$

$$FEATURE_CARDINALITY \Rightarrow ARITHMETIC_LEVEL$$

$$AGGREGATE_FUNCTIONS \Rightarrow ARITHMETIC_LEVEL$$

$$STRING_CONSTRAINTS \Rightarrow TYPE_LEVEL$$

Validación de la jerarquía de features

La estructura jerárquica generada debe ser compatible con el número máximo de features:

$$1 \leq MAX_TREE_DEPTH \leq MAX_FEATURES$$

Además, la selección probabilística de relaciones entre features requiere una distribución normalizada:

$$P_{optional} + P_{mandatory} + P_{alternative} + P_{or} + P_{group} = 1$$

Validación de restricciones

Las restricciones generadas dependen de los niveles activos. La distribución de tipos de restricciones se valida mediante:

$$\sum_i P(ConstraintType_i) = 1$$

donde los tipos disponibles quedan determinados por:

$$P(Integer) > 0 \Rightarrow ARITHMETIC_LEVEL$$

$$P(Real) > 0 \Rightarrow ARITHMETIC_LEVEL$$

$$P(String) > 0 \Rightarrow TYPE_LEVEL \wedge STRING_CONSTRAINTS$$

Las distribuciones de operadores deben estar normalizadas. Para operadores booleanos:

$$P(AND) + P(OR) + P(Implication) + P(Equivalence) = 1$$

Cuando está activo el nivel aritmético:

$$P(+) + P(-) + P(*) + P(/) = 1$$

$$P(=) + P(<) + P(>) + P(\leq) + P(\geq) = 1$$

Validación de atributos

La configuración de atributos diferencia entre generación aleatoria y definición manual.

Para atributos aleatorios:

$$1 \leq MIN_ATTRIBUTES \leq MAX_ATTRIBUTES$$

y la distribución de tipos debe cumplir:

$$P(Boolean) + P(Integer) + P(Real) + P(String) = 1$$

Para atributos definidos manualmente:

$$MIN_ATTRIBUTES = NULL$$

$$MAX_ATTRIBUTES = NULL$$

Además, la utilización de atributos en restricciones depende de los niveles disponibles:

$$UseInConstraints(Attribute_{Integer|Real}) \Rightarrow Arithmetic_Level$$

$$UseInConstraints(Attribute_{String}) \Rightarrow Type_Level \wedge String_Constraints$$

6.3.2.1. Consumo por otros módulos

Una vez validada la configuración, la instancia de `FmgeneratorModel` es utilizada por la operación `GenerateFeatureModel`, encargada de construir el modelo final compatible con FlamaPy.

Esta operación utiliza dicha configuración para generar:

- La jerarquía de features.
- Los tipos y cardinalidades.
- Los atributos asociados.
- Las restricciones UVL.

La corrección de estas reglas se verifica mediante la suite de pruebas ubicada en:

```
1 tests/test_validators.py
```

donde se incluyen casos que comprueban el rechazo de configuraciones inconsistentes, como distribuciones no normalizadas, dependencias incumplidas o rangos inválidos.

6.4 Módulo de generación de artefactos

[Describe el componente que produce el artefacto final (documento JSON, código fuente, fichero CAD, informe PDF...) a partir de la representación intermedia.]

La generación se encuentra encapsulada en la operación **GenerateFeatureModel**, perteneciente al plugin **fm_generator**. Esta operación recibe como entrada una instancia validada de **FmgeneratorModel**, que contiene todos los parámetros necesarios para controlar la generación, y produce como resultado un objeto **FeatureModel**.

El flujo principal de generación se resume en:

$$FmgeneratorModel \xrightarrow{\text{GenerateFeatureModel}} FeatureModel$$

El modelo resultante contiene la estructura jerárquica de features, sus relaciones, atributos asociados y restricciones expresadas mediante el formato interno de FlamaPy, permitiendo posteriormente su transformación a formatos compatibles como UVL.

La ejecución principal se realiza mediante el método:

```
1 GenerateFeatureModel.execute()
```

que valida nuevamente la configuración recibida y coordina las diferentes fases necesarias para construir el modelo final en formato UVL.

6.4.1 Cálculos o transformaciones específicas

[Si la generación implica cálculos no triviales (geometría, fórmulas analíticas, transformaciones de coordenadas...), descríbelos con sus fórmulas o pseudocódigo.]

La operación **GenerateFeatureModel** constituye el núcleo del proceso de generación. Su funcionamiento se divide en varias fases encargadas de transformar la configuración definida en **FmgeneratorModel** en un objeto **FeatureModel**. Cada fase está encapsulada mediante métodos independientes que separan la generación de la jerarquía, atributos, restricciones y configuración específica de UVL.

El algoritmo principal de la operación se muestra en el siguiente pseudocódigo:

```
1 FUNCION GenerateFeatureModel.execute:
2     INICIO
3         VALIDAR configuracion del generador
4         INICIALIZAR semilla aleatoria
5         feature_model, features =
6             GENERAR jerarquia de features
7         ASIGNAR atributos a las features generadas
8         GENERAR restricciones del modelo
9         GENERAR directivas UVL necesarias
```

```

10      DEVOLVER feature_model
11      FIN

```

Código 6.1: Esquema en pseudocódigo de la operación `GenerateFeatureModel.execute`.

Inicialización de la generación

Antes de comenzar la generación se ejecuta el método `_seed_generation()`, encargado de configurar la semilla utilizada por el generador aleatorio.

```

1 FUNCION _seed_generation(index, attempt):
2     INICIO
3         CALCULAR seed =
4             modelo.seed + index + (attempt * SAT_SEED_STRIDE)
5         CONFIGURAR generador aleatorio con seed
6     FIN

```

Código 6.2: Esquema en pseudocódigo del método `_seed_generation()`.

Generación de la jerarquía de features

El método `_generate_hierarchy()` construye la estructura principal del modelo. Inicialmente crea la feature raíz y determina el número total de features que serán generadas.

```

1 FUNCION _generate_hierarchy:
2     INICIO
3         CREAR feature raíz F0
4         DETERMINAR numero_features aleatorio
5         CREAR lista de features
6         PARA cada feature:
7             ASIGNAR tipo booleano o tipado UVL
8         PARA cada nivel de profundidad:
9             SELECCIONAR features del nivel actual
10            SELECCIONAR tipo de relacion padre-hijo
11            CREAR relaciones entre features
12            ASIGNAR cardinalidades
13        DEVOLVER FeatureModel y features generadas
14    FIN

```

Código 6.3: Esquema en pseudocódigo del método `_generate_hierarchy()`.

La creación de relaciones depende de las distribuciones configuradas por el usuario, pudiendo generar relaciones obligatorias, opcionales, alternativas, disyuntivas o grupos con cardinalidad.

Asignación de atributos

El método `_assign_attributes()` determina el modo de generación de atributos seleccionado.

```
1 FUNCION _assign_attributes(features):
2     INICIO
3         SI modo_aleatorio:
4             GENERAR atributos aleatorios
5         EN OTRO CASO:
6             ASIGNAR atributos definidos manualmente
7         ASOCIAR cada atributo con su feature correspondiente
8     FIN
```

Código 6.4: Esquema en pseudocódigo del método `_assign_attributes()`.

En el modo aleatorio, el tipo y dominio de cada atributo se obtiene mediante las distribuciones configuradas. En el modo manual, se utilizan las definiciones proporcionadas por el usuario durante la configuración del generador.

Generación de restricciones

La generación de restricciones se realiza mediante el método `_add_constraints()`, encargado de construir expresiones compatibles con el metamodelo de FlamaPy.

```
1 FUNCION _add_constraints(feature_model, features):
2     INICIO
3         CREAR conjuntos de features y atributos disponibles
4         DETERMINAR numero de restricciones
5         REPETIR para cada restriccion:
6             SELECCIONAR variables compatibles
7             DETERMINAR tipo de expresion:
8                 - booleana
9                 - aritmetica
10                - cadenas
11            CONSTRUIR arbol sintactico AST
12            ANADIR restriccion al FeatureModel
13     FIN
```

Código 6.5: Esquema en pseudocódigo del método `_add_constraints()`.

Cada restricción se representa mediante un árbol sintáctico abstracto (AST) que permite expresar operaciones lógicas, aritméticas y funciones específicas de UVL.

Generación de directivas UVL

Finalmente, el método `_build_uvl_includes()` analiza los niveles activados y genera las directivas necesarias para representar correctamente el modelo en UVL.

```

1 FUNCION _build_uvl_includes:
2     INICIO
3         CREAR lista de directivas
4         SI group_cardinality activo:
5             ANADIR Boolean.group-cardinality
6         SI feature_cardinality activo:
7             ANADIR Arithmetic.feature-cardinality
8         SI aggregate_functions activo:
9             ANADIR Arithmetic.aggregate-function
10        SI string_constraints activo:
11            ANADIR Type.string-constraints
12        DEVOLVER lista de directivas
13    FIN

```

Código 6.6: Esquema en pseudocódigo del método `_build_uvl_includes()`.

6.4.2 Síntesis del artefacto final

[Describe el proceso de generación final paso a paso, separando preparación de datos, renderizado de plantillas y operaciones específicas de la herramienta destino.]

Preparación de la configuración

Los parámetros definidos mediante el asistente son transformados en una instancia de `FmgeneratorModel`. Antes de iniciar la generación, dicha configuración es validada para garantizar su coherencia.

Generación del modelo intermedio

La operación `GenerateFeatureModel` utiliza la configuración validada para construir un objeto `FeatureModel` que contiene la jerarquía de features, atributos y restricciones generadas.

$$FmgeneratorModel \rightarrow FeatureModel$$

Transformación a formato UVL

El modelo generado es transformado mediante las operaciones proporcionadas por FlamaPy a una representación textual compatible con UVL.

$$FeatureModel \rightarrow UVL$$

Generación del artefacto final

Finalmente, la representación UVL obtenida se almacena como fichero de salida, constituyendo el artefacto final generado por el sistema y listo para ser utilizado por herramientas compatibles con el lenguaje UVL.

6.4.3 Operaciones complementarias

[Si tras la generación hay operaciones complementarias (ensamblado, postprocesado, decoración con metadatos...), descríbelas aquí.]

6.5 API REST / capa de servicios

[Describe la API expuesta por el sistema: agrupa los endpoints por bloque temático y, para cada endpoint, indica método, ruta, autenticación requerida y semántica básica.]

6.5.1 Endpoints implementados

[Bloques sugeridos:]

[Páginas / vistas:]

- *[GET /ruta_publica]: [descripción.]*
- *[GET /ruta_protegida]: [descripción.]*

[Endpoints de lógica principal:]

- *[POST /api/...]: [descripción y respuestas (200, 4xx, 5xx).]*
- *[GET /api/...]: [descripción.]*

[Endpoints de autenticación:]

- *[POST /api/auth/register]: [descripción.]*
- *[POST /api/auth/login]: [descripción.]*
- *[POST /api/auth/logout]: [descripción.]*

[Endpoints de gestión de recursos del usuario:]

- *[POST /api/...]: [descripción.]*
- *[GET /api/...]: [descripción.]*

Endpoints de entrada al generador

- **GET /generator**: inicializa el asistente de generación, reiniciando el estado almacenado en sesión y mostrando la página inicial.
- **GET /generator/random**: redirige al primer paso del flujo de generación aleatoria.
- **GET /generator/llm**: muestra la interfaz correspondiente al modo de generación basado en modelos de lenguaje.

Endpoints del asistente de configuración

El flujo principal de generación aleatoria se divide en seis etapas. Cada paso admite peticiones GET para mostrar el formulario correspondiente y POST para procesar los valores introducidos por el usuario.

- **GET/POST /generator/random/step1**: configuración inicial del generador, incluyendo número de modelos y parámetros generales.
- **GET/POST /generator/random/step2**: configuración de los niveles de expresividad UVL disponibles.
- **GET/POST /generator/random/step3**: definición de la estructura jerárquica del árbol de features.
- **GET/POST /generator/random/step4**: configuración de restricciones y operadores utilizados durante la generación.
- **GET/POST /generator/random/step5**: configuración de atributos asociados a las features.
- **GET/POST /generator/random/step6**: configuración final, generación del modelo y gestión de la salida.

Endpoints auxiliares

- **GET /generator/random/params-json**: devuelve la configuración actual del asistente en formato JSON.
- **POST /generator/random/summary-refresh/<step>**: actualiza dinámicamente el resumen de parámetros mostrado durante la configuración.

6.5.2 Descarga / exportación de artefactos

[Si el sistema permite descargar artefactos generados, describe los escenarios (descarga inmediata tras generación, descarga desde el historial de configuraciones guardadas...) y la estrategia de regeneración bajo demanda frente a almacenamiento permanente.]

Una vez finalizada la generación, el modelo de características obtenido se exporta en un formato compatible con UVL para su descarga en ZIP y posterior uso. El sistema no almacena permanentemente los modelos generados, sino que permite reproducir la generación a partir de la configuración utilizada y la semilla aleatoria definida.

6.6 Integración continua

[Describe la automatización de calidad y despliegue del proyecto. Si usas un proveedor concreto (GitHub Actions, GitLab CI, Jenkins...), nómbralo y comenta los workflows configurados.]

[Workflows típicos:]

[

- *[Validación de commits]: [convención de mensajes, autores...]*
- *[Linting / análisis estático]: [herramienta y comandos.]*
- *[Pruebas]: [ejecución de la suite en cada push y PR.]*
- *[Cobertura]: [umbral mínimo y políticas de fallo.]*
- *[Release]: [condiciones para crear releases.]*
- *[Despliegue]: [construcción y publicación de imagen / artefacto.]*

]

[Si dispones de comandos locales equivalentes a los workflows remotos (Make, scripts...), enuméralos para que el lector pueda reproducir las comprobaciones.]

- **Validación de commits:** comprobación automática de los cambios introducidos mediante GitHub Actions, verificando que las modificaciones cumplen las condiciones establecidas antes de su integración.
- **Linting y análisis estático:** ejecución de herramientas de análisis del código para detectar errores de estilo, problemas potenciales y mantener una calidad homogénea.

- **Pruebas:** ejecución automática de la suite de tests del proyecto en los cambios realizados, asegurando que las nuevas modificaciones no afectan a la funcionalidad existente.
- **Despliegue:** automatización de la actualización del entorno de ejecución tras superar correctamente las comprobaciones anteriores.

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Con el sistema implementado y la integración continua en marcha, falta demostrar que se comporta como exigen los requisitos del Capítulo 4. El Capítulo 7 presenta la estrategia de pruebas con la que se verifica».]

Con la implementación del generador, la integración de sus componentes y la automatización de los procesos de validación y despliegue, queda establecido el funcionamiento completo del sistema desarrollado. El siguiente capítulo presenta la estrategia de pruebas aplicada, orientada a verificar que la solución cumple con los requisitos definidos y que los distintos módulos funcionan correctamente.

7 PRUEBAS Y VALIDACIÓN

[Este capítulo describe cómo se ha verificado que el sistema cumple los requisitos. Las secciones siguen el orden clásico de la pirámide de pruebas: de la base (muchas pruebas, rápidas y baratas) a la cumbre (pocas, lentas y caras). En la base están las pruebas unitarias, en el medio las de integración y en la cima las de extremo a extremo / aceptación. Delimita el alcance desde el principio: este capítulo responde a la pregunta de la verificación («¿hemos construido bien el sistema?»); la evaluación de los resultados frente a los objetivos del trabajo y su discusión se realiza en el Capítulo 8.]

7.1 Estrategia de pruebas

[Resumen del enfoque adoptado: niveles de pruebas usados, framework elegido y entorno en el que se ejecutan. Adapta los niveles al tipo de TFG (software, hardware, modelo de IA, estudio empírico...).]

[Niveles de la pirámide aplicados en este TFG (incluye los que apliquen):]

- **[Pruebas unitarias]:** *[base de la pirámide. Verifican unidades aisladas (funciones, clases, módulos). Son las más numerosas y rápidas; deben ser deterministas y no depender de servicios externos.]*
- **[Pruebas de integración]:** *[nivel intermedio. Comprueban que varios componentes funcionan juntos correctamente, incluyendo contratos con servicios externos (BD, APIs internas...)].*
- **[Pruebas de extremo a extremo / sistema]:** *[cumbre de la pirámide. Validan flujos completos desde la perspectiva del usuario o cliente. Son las menos numerosas porque son las más lentas y frágiles.]*
- **[Pruebas de aceptación (opcional)] :** *[cotejan el sistema frente a los requisitos del Capítulo 4.]*

[Comenta brevemente el entorno de pruebas: si las dependencias externas son reales, simuladas o efímeras (en memoria, contenedores desechables, mocks...) y cómo se garantiza el aislamiento entre tests.]

La estrategia de pruebas aplicada tiene como objetivo verificar el correcto funcionamiento del generador integrado en UVLHub, validando tanto los componentes individuales como el flujo completo de configuración y generación.

Las pruebas desarrolladas se organizan en los siguientes niveles:

- **Pruebas unitarias:** enfocadas en validar componentes aislados del sistema, incluyendo las reglas de validación, la construcción de configuraciones y la lógica interna del generador.
- **Pruebas de integración:** comprueban la interacción entre los distintos componentes del módulo **generator**, incluyendo la gestión del estado del , la persistencia de parámetros y la generación de salidas.
- **Pruebas de sistema:** verifican el funcionamiento completo del flujo desde la perspectiva del usuario, incluyendo la interacción con el asistente de configuración mediante pruebas automatizadas.
- **Pruebas manuales:** se han realizado comprobaciones manuales del flujo completo de generación, verificando el comportamiento de la aplicación durante el desarrollo.

La ejecución de las pruebas automatizadas se realiza mediante la suite de tests del proyecto, utilizando el entorno definido mediante Docker para garantizar la disponibilidad de las dependencias necesarias y la reproducibilidad de los resultados.

7.2 Pruebas unitarias

[Describe cómo se prueban los componentes centrales del sistema en aislamiento (la lógica de negocio, el algoritmo principal o el módulo más crítico del TFG). Indica qué grupos de casos cubre, no la lista exhaustiva.]

[Grupos de casos típicos:]

- *[Caso nominal]: [entrada válida que debe producir el resultado esperado.]*
- *[Entradas inválidas]: [datos mal formados, fuera de rango o con campos ausentes.]*
- *[Casos límite]: [valores en la frontera del dominio admisible.]*
- *[Reglas de negocio]: [combinaciones que activan restricciones específicas del dominio.]*

[Si la lógica del módulo no se reduce a un booleano, comenta también qué información complementaria se valida (mensajes, códigos de error, listas de campos afectados...)]

Las pruebas unitarias se han centrado en validar los componentes internos del generador y la lógica asociada al asistente de configuración. Los principales grupos de casos cubiertos son:

- **Generador de modelos de características:** se valida la operación `GenerateFeatureModel.execute`, comprobando que los parámetros del objeto `FmgeneratorModel` se aplican correctamente durante la generación. Se verifican aspectos como el número de modelos generados, la reproducibilidad mediante semillas, los límites de características y profundidad del árbol, así como la generación de relaciones jerárquicas, atributos y restricciones según los niveles UVL habilitados.
- **Generación de restricciones y atributos:** se comprueba que la activación de los diferentes niveles del generador produce únicamente los elementos permitidos. Se validan restricciones booleanas, aritméticas y de tipo string, además de operadores agregados (`sum`, `avg`), funciones sobre cadenas y distribución de tipos de atributos. También se verifica que los atributos configurados para no participar en restricciones no aparecen en las expresiones generadas.
- **Validación del modelo de parámetros:** se prueban las comprobaciones realizadas por `FmgeneratorModel`, incluyendo la validación de distribuciones de probabilidad, rangos de generación y coherencia entre niveles activados. Además, se comprueba la correcta conversión desde el diccionario plano de parámetros utilizado por UVLHub hacia la estructura interna del generador.
- **Validadores del asistente de configuración:** se prueban los métodos de validación de cada paso del wizard (`validate_step1_form` a `validate_step5_form`), comprobando casos como límites inválidos de modelos o características, distribuciones incorrectas, configuraciones incompatibles entre niveles y restricciones imposibles de generar.
- **Gestión del estado y construcción de parámetros:** se valida la persistencia temporal de la configuración almacenada en la sesión de Flask mediante las funciones de gestión del estado del wizard. También se comprueba la normalización de distribuciones y la correcta propagación de los valores entre pasos mediante los constructores de parámetros.

7.3 Pruebas de integración

[Describe las pruebas que ejercitan varios componentes a la vez. Conviene separarlas según los componentes que combinan.]

- *[Pruebas sobre el contrato con la base de datos / capa de persistencia (ORM, esquemas, transacciones).]*

- *[Pruebas sobre la capa de servicios o API (códigos de respuesta, contratos de entrada/salida, autenticación, control de acceso entre usuarios).]*
- *[Pruebas sobre integraciones con servicios externos (proveedores de identidad, almacenamiento, APIs de terceros...)].*

[Documenta también las pruebas de casos negativos (valores fuera de rango, restricciones violadas, datos incompletos...)].

Las pruebas de integración se han centrado en verificar la comunicación entre los diferentes módulos que intervienen en el proceso de generación, desde la configuración realizada mediante el wizard de UVLHub hasta la obtención del modelo UVL generado.

Los principales casos cubiertos son:

- **Integración del wizard con la capa de servicios:** se prueba el recorrido completo del asistente mediante las rutas `/generator/random/step1` a `/generator/random/step6`, verificando la interacción entre los controladores Flask y los métodos de `GeneratorWizardService` encargados de procesar cada etapa.
- **Integración de la configuración persistida con el generador:** se comprueba que la configuración almacenada durante el flujo del wizard se recupera correctamente mediante el endpoint `/generator/random/params-json` y puede ser transformada en una instancia válida de `FmgeneratorModel`.
- **Integración del modelo de parámetros con la generación UVL:** se valida que una configuración completa procedente del wizard es utilizada correctamente por `GenerateFeatureModel`, generando artefactos UVL que reflejan las opciones seleccionadas, como niveles UVL, cardinalidades, tipos de atributos, restricciones y opciones de nomenclatura.
- **Integración del flujo de navegación y estado del asistente:** se verifican los escenarios de navegación hacia atrás entre pasos, comprobando que los valores almacenados en la sesión Flask permanecen consistentes y que una configuración parcialmente modificada no provoca estados inválidos durante la generación.

7.4 Pruebas de extremo a extremo

[Describe la validación del sistema desde la perspectiva del usuario final. No tiene por qué ejecutarse en un navegador o entorno gráfico real: muchas veces basta con automatizar el flujo a través de la interfaz pública del sistema (HTTP, CLI, etc.).]

Estas pruebas son las menos numerosas pero las que dan más confianza en que el sistema funciona como un todo.]

[Plantilla de pasos del flujo principal validado:]

[

1. [Paso 1.]

2. [Paso 2.]

3. [Paso 3.]

4. [Paso 4.]

5. [Paso 5.]

]

[Casos complementarios habituales:]

[

■ *[Estado inicial vacío.]*

■ *[Repetición del flujo con varias instancias y comprobación del orden.]*

■ *[Acceso no autorizado o sesión inválida.]*

■ *[Recuperación tras un error controlado.]*

]

Las pruebas de extremo a extremo se han implementado mediante Selenium WebDriver, validando el comportamiento del sistema desde la interfaz web hasta la ejecución del proceso de generación en el navegador.

El flujo principal validado es el siguiente:

1. **Carga del generador:** se verifica la correcta inicialización de la interfaz mediante el acceso a la ruta `/generator/random/step1` y la disponibilidad de los elementos necesarios para comenzar la configuración.
2. **Ejecución del proceso de generación:** se automatiza la interacción con el wizard completo, comprobando la transición entre las diferentes vistas y la correcta respuesta de la interfaz ante las acciones del usuario.
3. **Inicialización del entorno Pyodide:** se valida la carga del runtime JavaScript necesario para ejecutar el generador en el navegador antes de iniciar la generación.

4. **Generación y visualización del modelo UVL:** se comprueba la ejecución de la generación desde la interfaz, verificando el estado final del proceso, la aparición del resultado generado y la visualización del contenido UVL obtenido.

Además, se han probado escenarios adicionales relacionados con la interacción del usuario:

- Navegación entre pasos mediante los controles de la interfaz, verificando que el comportamiento visual coincide con el estado esperado.
- Activación y desactivación de opciones dependientes dentro del formulario, como niveles aritméticos, cardinalidades y opciones avanzadas.
- Ejecución del generador con configuraciones completas desde la interfaz, comprobando la disponibilidad de las acciones finales sobre el modelo generado.

7.5 Cobertura y métricas de calidad

[Indica las métricas que has usado para evaluar la calidad de la suite y, si procede, los umbrales mínimos exigidos en CI.]

[Métricas habituales:]

- *[Cobertura de líneas / ramas.]*
- *[Número de pruebas y desglose por nivel de la pirámide.]*
- *[Resultado del análisis estático (linter, tipado).]*

[Comenta también qué se excluye de la medición (ficheros generados, plantillas, código de migración, dependencias externas...)].

[Resume la suite completa en la Tabla 7.1: el tribunal debe poder ver de un vistazo cuántas pruebas hay, de qué tipo y con qué resultado.]

Nivel	N.º de pruebas	Resultado	Cobertura
Unitarias	[N]	[N/N pasan]	[X %]
Integración	[N]	[N/N pasan]	[X %]
Extremo a extremo	[N]	[N/N pasan]	[—]
Total	[N]	[N/N pasan]	[X %]

Tabla 7.1: Resumen de la suite de pruebas por nivel de la pirámide.

[No te limites a enumerar cifras: interprétalas. Una cobertura del X % no dice nada por sí sola. Indica qué módulos quedan menos cubiertos y por qué es asumible (código

trivial, generado, de difícil automatización...), y qué te dice la distribución de pruebas por nivel sobre la fiabilidad de la suite (¿la base de la pirámide es realmente la más ancha?).]

[Cierra el eslabón con los objetivos: indica qué criterios de la Tabla 1.1 quedan verificados por esta suite, apoyándote en la matriz de la Tabla 4.26.]

En este trabajo no se ha utilizado una métrica de cobertura de código basada en líneas o ramas ejecutadas. Durante el desarrollo se ha priorizado la realización de pruebas que aportasen una mayor evidencia sobre el correcto funcionamiento del sistema, incluyendo comprobaciones automáticas sobre los comportamientos críticos y pruebas manuales de los flujos principales de la aplicación, frente al cálculo de una métrica de cobertura cuya obtención no aportaba información adicional suficiente en este contexto.

La suite de pruebas permite verificar principalmente los siguientes aspectos:

- La generación correcta de modelos UVL a partir de configuraciones diferentes, comprobando que los niveles de expresividad seleccionados se reflejan en los modelos obtenidos.
- La correcta propagación de los parámetros definidos durante el flujo de configuración hasta el motor de generación, incluyendo validaciones, normalización de valores y restricciones entre opciones incompatibles.
- La generación de construcciones avanzadas del lenguaje UVL, como atributos, restricciones booleanas, expresiones aritméticas, funciones de agregación y restricciones sobre cadenas.
- La integración del generador dentro de UVLHub, verificando el flujo completo desde la configuración realizada por el usuario hasta la obtención del modelo generado.

Los resultados obtenidos permiten verificar los criterios de éxito asociados a los objetivos definidos en la Tabla 1.1. En particular, las pruebas realizadas proporcionan evidencia de cumplimiento para la generación de modelos UVL con diferentes niveles de expresividad (O1), la configuración del proceso de generación mediante diferentes parámetros (O2), la integración de la solución dentro del ecosistema UVLHub y Flamapy (O3) y la existencia de una batería automatizada de validación del funcionamiento del generador (O5).

Además, la correspondencia entre requisitos y pruebas queda recogida en la matriz de trazabilidad de la Tabla 4.27, donde se relacionan los principales requisitos funcionales y no funcionales con los casos de prueba encargados de verificar su cumplimiento.

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Verificado que el sistema está

bien construido y que cumple los requisitos especificados, queda la pregunta de fondo: ¿resuelve el problema planteado y en qué medida? El Capítulo 8 evalúa los resultados obtenidos frente a los criterios de éxito y los discute críticamente».]

La estrategia de pruebas desarrollada permite verificar el correcto funcionamiento del sistema y comprobar el cumplimiento de los principales requisitos definidos. Una vez validada la implementación, el siguiente capítulo analiza los resultados obtenidos y evalúa en qué medida la solución desarrollada alcanza los objetivos planteados inicialmente.

8 RESULTADOS Y DISCUSIÓN

[Mientras el Capítulo 7 verificaba que el sistema está bien construido, este capítulo evalúa si resuelve el problema planteado en el Capítulo 1 y en qué medida: presenta los resultados obtenidos, los contrasta con los criterios de éxito fijados, los compara con los trabajos previos y los interpreta críticamente. Es el capítulo donde el tribunal busca la evidencia de lo conseguido: cada afirmación debe apoyarse en una tabla o una figura.]

8.1 Diseño de la evaluación

[Antes de mostrar números, define qué se mide y cómo. Retoma la Tabla 1.1 del Capítulo 1: sus criterios de éxito son el guion de esta evaluación, y cada uno debe quedar respondido en la Sección 8.2. Describe:]

- **[Entorno de ejecución]:** *[hardware (CPU, RAM...) y software (sistema operativo, versiones relevantes) sobre los que se han obtenido las medidas, para que sean reproducibles.]*
- **[Datos o casos de estudio]:** *[origen, tamaño y representatividad de los datos, escenarios o usuarios empleados en la evaluación.]*
- **[Repeticiones]:** *[si hay medidas de rendimiento, indica cuántas veces se repite cada medición y por qué: una sola ejecución está sujeta a variabilidad (carga del sistema, caché, red...) y no es concluyente.]*
- **[Agregación]:** *[cómo se resumen los datos: media y desviación típica, mediana, percentiles (p. ej. p95 para tiempos de respuesta)... y por qué ese estadístico es el adecuado. Toda media debe ir acompañada de su desviación típica o de un intervalo de confianza; y si se comparan alternativas, conviene apoyar la comparación en un test estadístico sencillo (t-test o Wilcoxon).]*

[Rigor: distingue siempre lo que se ha medido de lo que es una impresión subjetiva. «La interfaz es intuitiva» no es un resultado; «8 de 10 usuarios completaron la tarea sin ayuda en menos de 2 minutos» sí lo es.]

La evaluación se centra en comprobar la capacidad del generador para producir modelos UVL con diferentes niveles de expresividad, la influencia de los parámetros de configuración sobre los modelos obtenidos, la incorporación de construcciones avanzadas del lenguaje, la integración dentro del ecosistema UVLHub y Flamapy, y

la disponibilidad de una batería de pruebas automatizadas que permita validar su funcionamiento.

Durante el proceso de desarrollo se han utilizado diferentes configuraciones del generador y se han analizado los modelos UVL resultantes, comprobando que las características seleccionadas durante la configuración se reflejan correctamente en los artefactos generados. Estas comprobaciones se han complementado con la ejecución de la suite de pruebas automatizadas y con validaciones manuales de los principales flujos de uso de la aplicación.

La evaluación se ha realizado sobre el siguiente entorno:

- **Sistema operativo:** Ubuntu 24.04.4 LTS.
- **Hardware:** Slimbook Executive con procesador Intel® Core™ i7-13700H (20 núcleos) y 32 GB de memoria RAM.
- **Navegador:** Firefox 148.0.2 (64 bits).
- **Lenguaje de desarrollo:** Python 3.13.11.
- **Contenedores:** Docker 29.7.2.

Adicionalmente, se realizará una evaluación del rendimiento del generador mediante la medición del tiempo de inicialización del entorno y del tiempo empleado en la generación de modelos una vez cargadas las dependencias. Para ello, se utilizará una configuración con un volumen elevado de modelos generados, con el objetivo de obtener una estimación representativa del comportamiento del sistema en escenarios de mayor carga.

8.2 Resultados

[Organiza esta sección con una subsección por objetivo (o por criterio de éxito), en el mismo orden en que se enunciaron en el Capítulo 1. En cada subsección sigue el patrón: (1) presenta el resultado en una tabla o figura, (2) referénciala desde el texto y (3) interprétala frente al umbral fijado. Recuerda el principio: toda tabla o figura se referencia desde el texto y se interpreta — un resultado sin interpretación no es un resultado. Si el criterio de un objetivo ya quedó verificado en un capítulo anterior (columna Cap. de la Tabla 1.1), su subsección se limita a recapitular esa evidencia con \ref y a dar el veredicto frente al criterio, sin repetir datos. Las dos subsecciones siguientes modelan ambos patrones: O1 ejemplifica la recapitulación y O2 la medición completa.]

8.2.1 Resultados sobre el objetivo O1

[Patrón RECAPITULACIÓN, para objetivos cuyo criterio ya quedó verificado en un capítulo anterior según la columna Cap. de la Tabla 1.1. Siguiendo el ejemplo conductor, el criterio de O1 (cobertura de todos los RF de prioridad alta) quedó verificado en los Capítulos 4 y 7: recapitula esa evidencia con \ref —la matriz de trazabilidad de la Tabla 4.26 muestra que cada RF tiene al menos una prueba que lo cubre, y la Tabla 7.1 acredita que la suite completa pasa— y cierra con el veredicto frente al criterio («el criterio de éxito asociado al objetivo O1 se considera cumplido»), sin repetir datos ni añadir tablas nuevas.]

El objetivo O1 consistía en desarrollar un generador automático capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, incluyendo construcciones avanzadas del lenguaje que permitan ampliar la variedad de modelos disponibles para el análisis de variabilidad.

Para verificar este objetivo se han generado modelos utilizando distintas configuraciones del generador, comprobando que los parámetros seleccionados se reflejan correctamente en los modelos UVL obtenidos y que estos mantienen una estructura válida conforme al lenguaje.

En las siguientes imágenes se puede observar que efectivamente se ha alcanzado el objetivo definido. En la primera se puede ver la configuración usada para generar los modelos con expresividad avanzada, donde se han activado todos los *major levels* junto con todos sus *minor levels*

YOUR SETUP	
1 · Batch	
Models	2
Seed	4266
2 · Language levels	
Majors	Boolean, Arithmetic, Type
Minors	Group card., Feature card., Aggregates, String ctc
3 · Feature tree	
Features	12 – 14
Depth ≤	6
Rel mix	opt 21% · mand 22% · alt 18% · or 26% · group 13%
Feat card.	50% [..]
Group card.	[3..6]
4 · Constraints	
Count	10 – 12
Vars/ctc	2 – 10
Type mix	B 31% · Z 24% · R 17% · S 28%
5 · Attributes	
Mode	random
Count	3 – 6
6 · Output	
Ensure SAT	no
Suffixes	—

Figura 8.1: Configuración usada para verificar O1. Elaboración propia

Las dos siguientes imágenes muestran algunos de los modelos en cuestión, donde se puede comprobar que existen características, atributos y constraints de varios niveles de expresividad.

```

1 include
2   Boolean.group-cardinality
3   Arithmetic.*
4   Type.string-constraints
5 features
6   F0
7   or
8     Integer F2 cardinality [4..4] {Attr5 true}
9   F1
10  optional
11    F4 cardinality [5..5]
12    optional
13      F5 cardinality [3..4]
14      mandatory
15        F8 {Attr1 true}
16  optional
17    F3 {Attr3 false}
18    mandatory
19      F6
20      optional
21        F7 cardinality [2..3]
22        alternative
23          F10 {Attr2 false}
24          mandatory
25            F12 {Attr4 67}
26          mandatory
27            F11 {Attr0 72.37}
28          Real F9 cardinality [4..4]
29          mandatory
30            F13
31 constraints
32   (F12.Attr4 - F2) > F2
33   (F9 < F2) & F12
34   (F2 > F9) <=> F4
35   (((F2 < F2) <=> (F12.Attr4 > F2)) | F13
36   (((F2 - F2) + F9) > F9) | F3.Attr3
37   ((F12.Attr4 > F12.Attr4) <=> (((F11.Attr0 + F9) + F2) > F2)) => ! F8
38   (((! F13 & ! F4) <=> ! F7) | ((F12 | ! F12) <=> F8)) & (F7 => ! F5)
39   (((F9 > F2) & (F9 > F9)) | (F2 > (F2 + F11.Attr0))) <=> (F11.Attr0 > F11.Attr0)

```

Figura 8.2: Ejemplo 1 para verificar O1. Elaboración propia

```

1  include
2    Boolean.group-cardinality
3    Arithmetic.*
4    Type.string-constraints
5  features
6    F0
7      mandatory
8        F2 cardinality [4..4] {Attr2 false}
9      optional
10       F3
11         or
12         F6
13         F5 {Attr0 32.67, Attr1 false}
14         alternative
15           F8
16             mandatory
17               F9
18                 [3]
19                 F13 {Attr3 'Ahj6GYVR5NJTHGLj4brjN9Rh96lWhnjGXBN06DAjLyK0a1cR18'}
20                 F12 cardinality [4..4]
21                 F11 cardinality [2..4]
22             F7 cardinality [5..5]
23             mandatory
24               F10
25         mandatory
26           F1 cardinality [3..3]
27         mandatory
28           F4
29  constraints
30    ((F5.Attr0 > len(F13.Attr3)) & ((F9 <=> F9) <=> F5)
31    ((F8 <=> (F5.Attr0 > F5.Attr0)) & ((F1 <=> F4) => ! F3)) | ! F13
32    ((! F1 => F4) <=> F1) & (len(F13.Attr3) > len(F13.Attr3))
33    ((F2 <=> F9) | (F5.Attr0 > len(F13.Attr3))) => (F13.Attr3 == F13.Attr3)
34    (((len(F13.Attr3) + len(F13.Attr3)) == len(F13.Attr3)) <=> ! F12) | (! F9 & F10)) <=> F8 & ((F5 => ! F12)
35    ((F5.Attr0 > F5.Attr0) <=> (len(F13.Attr3) > len(F13.Attr3))) <=> F2.Attr2
36    ((F5.Attr0 < F5.Attr0) => (F13.Attr3 == F13.Attr3)) & ((! F2.Attr2 & ! F3) <=> F3)
37    (len(F13.Attr3) > len(F13.Attr3)) <=> (F5.Attr0 > len(F13.Attr3))
38    (((F5.Attr0 + F5.Attr0) * F5.Attr0) > len(F13.Attr3)) & (len(F13.Attr3) > len(F13.Attr3))
39    (((F5.Attr0 > F5.Attr0) & (len(F13.Attr3) > (len(F13.Attr3) - F5.Attr0 - len(F13.Attr3)))) & ((F3 | ! F8) &

```

Figura 8.3: Ejemplo 2 para verificar O1. Elaboración propia

Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O1.

8.2.2 Resultados sobre el objetivo O2

El objetivo O2 consistía en desarrollar un sistema de configuración flexible que permitiera modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a diferentes escenarios.

Para verificar este objetivo se ha analizado el flujo de configuración del generador, comprobando que los distintos parámetros disponibles se encuentran organizados en diferentes etapas según su funcionalidad. La estructura del asistente de configuración y las diferentes vistas que permiten definir estos parámetros se muestran en la Sección 5.4.1.

Además, se ha comprobado durante el desarrollo que las modificaciones realizadas sobre estos parámetros son aplicadas correctamente durante el proceso de generación, lo cual a su vez puede comprobarse en el 8.2.1.

Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O2.

Resultados sobre el objetivo O3

El objetivo O3 consistía en integrar la solución desarrollada dentro del ecosistema Flamapy y permitir su utilización desde UVLHub como herramienta de generación de modelos de características.

La prueba de que se ha alcanzado este objetivo es la propia integración del generador como un plugin de Flamapy, cuya estructura propia puede comprobarse observando la imagen correspondiente en la Sección 6.2 (**Plugin generador basado en Flamapy**), así como su correcta utilización desde el flujo proporcionado por UVLHub.

Además, se ha verificado que el módulo desarrollado mantiene la compatibilidad con las dependencias y componentes existentes del ecosistema, permitiendo su ejecución dentro del entorno habitual de la aplicación.

Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O3.

Resultados sobre el objetivo O4

El objetivo O4 consistía en evaluar el comportamiento temporal del generador en escenarios de generación con un volumen elevado de modelos de características.

Para realizar esta evaluación se ha utilizado una configuración con 1000 modelos a generar, incorporando diferentes niveles de expresividad del lenguaje UVL. La configuración empleada no representa un caso extremo en cuanto al tamaño individual de los modelos, ya que se han utilizado rangos moderados de características y restricciones, permitiendo obtener una estimación realista del coste de generación en un escenario habitual con un volumen elevado de modelos.

La configuración utilizada durante la evaluación se muestra en la Figura 8.4.

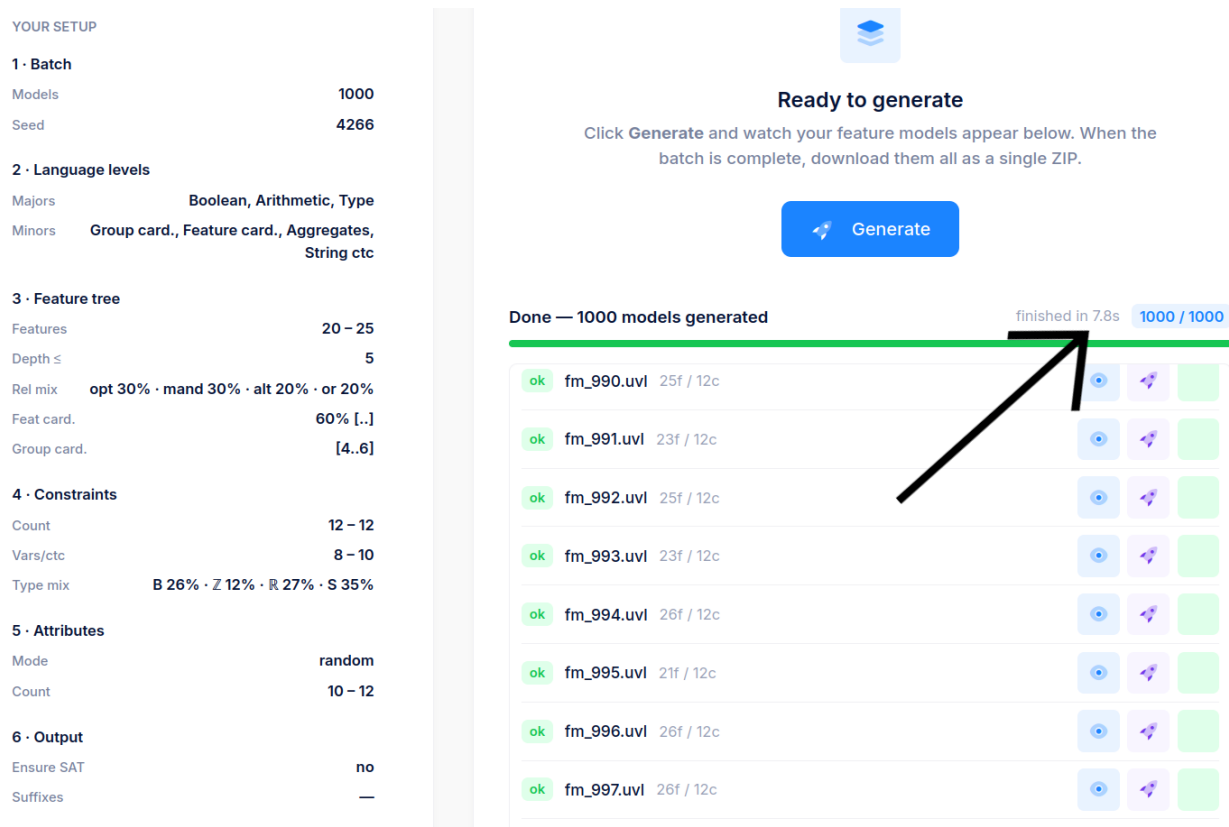


Figura 8.4: Resultado temporal de la generación de modelos. Elaboración propia

Los tiempos obtenidos durante la ejecución se dividen en dos fases diferenciadas: la carga inicial de las dependencias necesarias para ejecutar el generador y la generación posterior del conjunto de modelos UVL.

Fase	Tiempo obtenido
Carga inicial de dependencias e inicialización	4.05 s
Generación de 1000 modelos UVL	7.8 s

Tabla 8.1: Tiempos obtenidos durante la evaluación temporal del generador. Elaboración propia

Los resultados obtenidos muestran que el coste asociado a la generación de un volumen elevado de modelos es reducido una vez finalizada la fase inicial de carga e inicialización. En este escenario, la generación de 1000 modelos UVL se completa en 8.4 segundos, mostrando un comportamiento adecuado para el uso previsto del generador. No obstante, estos tiempos dependen de la complejidad de los modelos solicitados, pudiendo incrementarse en configuraciones con un mayor número de características, restricciones o atributos. Además, los valores obtenidos pueden variar en función de los recursos computacionales disponibles en el equipo donde se ejecute el generador.

Resultados sobre el objetivo O5

El objetivo O5 consistía en validar el correcto funcionamiento del generador mediante una batería de pruebas automatizadas que cubriese tanto la lógica interna del motor de generación como el flujo de configuración integrado en UVLHub.

Para verificar este objetivo se ha ejecutado la suite de pruebas del módulo *generator* dentro del entorno Docker de la aplicación. Estas pruebas validan diferentes componentes del sistema, incluyendo la generación de modelos UVL a partir de distintas configuraciones, la aplicación de los parámetros del asistente de generación, la persistencia del estado entre etapas del flujo, las opciones de salida generadas y el comportamiento del generador ante diferentes niveles de expresividad del lenguaje UVL, verificando que las configuraciones relativas a atributos, restricciones, cardinalidades y operadores avanzados producen los resultados esperados.

La ejecución completa de la suite mediante *rosemary* finalizó correctamente, obteniéndose 134 pruebas superadas. En la Figura 8.5 se muestra el resultado de dicha ejecución.

```

app/modules/generator/tests/test_wizard_outputs.py::test_attribute_fixed_count_observed[7] PASSED [ 77%]
app/modules/generator/tests/test_wizard_outputs.py::test_attribute_type_dominance[dist0-boolean] PASSED [ 78%]
app/modules/generator/tests/test_wizard_outputs.py::test_attribute_type_dominance[dist1-integer] PASSED [ 79%]
app/modules/generator/tests/test_wizard_outputs.py::test_attribute_type_dominance[dist2-real] PASSED [ 79%]
app/modules/generator/tests/test_wizard_outputs.py::test_attribute_type_dominance[dist3-string] PASSED [ 80%]
app/modules/generator/tests/test_wizard_outputs.py::test_filename_suffix_combinations[flags0-^fm\\.\\d+\\.\\uvl$] PASSED [ 81%]
app/modules/generator/tests/test_wizard_outputs.py::test_filename_suffix_combinations[flags1-^fm\\.\\d+f\\.\\uvl$] PASSED [ 82%]
app/modules/generator/tests/test_wizard_outputs.py::test_filename_suffix_combinations[flags2-^fm\\.\\d+c\\.\\uvl$] PASSED [ 82%]
app/modules/generator/tests/test_wizard_outputs.py::test_filename_suffix_combinations[flags3-^fm\\.\\d+f\\.\\d+c\\.\\uvl$] PASSED [ 83%]
app/modules/generator/tests/test_wizard_outputs.py::test_same_seed_gives_same_models[1] PASSED [ 84%]
app/modules/generator/tests/test_wizard_outputs.py::test_same_seed_gives_same_models[7] PASSED [ 85%]
app/modules/generator/tests/test_wizard_outputs.py::test_same_seed_gives_same_models[42] PASSED [ 85%]
app/modules/generator/tests/test_wizard_outputs.py::test_same_seed_gives_same_models[1234] PASSED [ 86%]
app/modules/generator/tests/test_wizard_outputs.py::test_same_seed_gives_same_models[99999] PASSED [ 87%]
app/modules/generator/tests/test_wizard_outputs.py::test_prev_nav_goes_to_previous_step[2-/step1] PASSED [ 88%]
app/modules/generator/tests/test_wizard_outputs.py::test_prev_nav_goes_to_previous_step[3-/step2] PASSED [ 88%]
app/modules/generator/tests/test_wizard_outputs.py::test_prev_nav_goes_to_previous_step[4-/step3] PASSED [ 89%]
app/modules/generator/tests/test_wizard_outputs.py::test_prev_nav_goes_to_previous_step[5-/step4] PASSED [ 90%]
app/modules/generator/tests/test_wizard_outputs.py::test_prev_nav_goes_to_previous_step[6-/step5] PASSED [ 91%]
app/modules/generator/tests/test_wizard_outputs.py::test_back_navigation_preserves_all_choices PASSED [ 91%]
app/modules/generator/tests/test_wizard_outputs.py::test_everything_on_every_family_represented PASSED [ 92%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step2_type_enables_arithmetic PASSED [ 93%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step3_normalizes_relation_distribution PASSED [ 94%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step4_disables_arithmetic_probs_when_arithmetic_off PASSED [ 94%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step5_random_attrs_normalizes_distribution PASSED [ 95%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step3_normalizes_relation_distribution_with_group_cardinality PASSED [ 96%]
app/modules/generator/tests/test_wizard_persisters.py::test_apply_step5_masks_unavailable_attribute_types_when_levels_are_off PASSED [ 97%]
app/modules/generator/tests/test_wizard_state.py::test_save_step_state_preserves_checkbox_false_values PASSED [ 97%]
app/modules/generator/tests/test_wizard_state.py::test_load_step_state_merges_saved_values_over_defaults PASSED [ 98%]
app/modules/generator/tests/test_wizard_state.py::test_save_step_state_does_not_overwrite_other_steps PASSED [ 99%]
app/modules/generator/tests/test_wizard_state.py::test_clear_step_state_only_removes_target_step PASSED [100%]

===== 134 passed in 93.37s (0:01:33) =====

```

Figura 8.5: Resultado de la ejecución de la suite de pruebas automatizadas.

Elaboración propia

Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O5, demostrando la estabilidad de las funcionalidades desarrolladas mediante pruebas automatizadas.

8.3 Comparación con trabajos previos

[Retoma la Tabla ?? del Capítulo 2, ahora con tus resultados en la mano: ¿qué celdas marcadas como «Parcial» o «No» en los otros trabajos cubre ya tu solución, con qué evidencia de la Sección 8.2? ¿Y en qué queda tu trabajo por detrás de ellos? Una

comparación honesta en ambas direcciones es lo que distingue un análisis riguroso de un alegato.]

[Si los trabajos relacionados publican métricas comparables (mismos datos o condiciones equiparables), añade aquí una tabla numérica con una fila por métrica y una columna por trabajo, citando la fuente de cada cifra [8]. Las cifras propias deben ir acompañadas de su dispersión (desviación típica o intervalo de confianza), no solo de la media. Si no existen métricas comparables, no la fuerces: justifica por qué la comparación directa no es posible (datos, entorno o alcance distintos) y mantén la comparación cualitativa por criterios.]

La Tabla ?? del Capítulo ?? recoge una comparación cualitativa entre diferentes herramientas relacionadas con la generación automática de modelos de características. En ella se consideran tres aspectos principales: la capacidad de generación automática, el grado de personalización ofrecido y la incorporación de construcciones de expresividad avanzada.

Los resultados obtenidos durante la evaluación del generador desarrollado permiten observar que la solución propuesta cubre los tres criterios analizados. A diferencia de S.P.L.O.T. y FeatureIDE, cuyas capacidades de generación están principalmente orientadas a modelos basados en construcciones booleanas, el generador desarrollado permite producir modelos UVL incorporando elementos adicionales como atributos tipados, restricciones aritméticas, restricciones sobre cadenas, funciones agregadas y diferentes mecanismos de cardinalidad.

En comparación con BeTTy, la solución desarrollada mantiene la capacidad de configuración flexible de los parámetros de generación, pero amplía el rango de expresividad de los modelos generados. Mientras que BeTTy permite definir escenarios experimentales con diferentes niveles de complejidad estructural, el generador desarrollado incorpora además construcciones propias de lenguajes de variabilidad más recientes como UVL.

Además, la integración realizada dentro del ecosistema Flamapy y UVLHub supone una diferencia respecto a las herramientas analizadas, ya que permite utilizar el generador directamente dentro de una plataforma web orientada a la gestión y análisis de modelos de características, manteniendo la compatibilidad con las herramientas existentes del ecosistema.

No se ha realizado una comparación cuantitativa directa en términos de tiempo de generación o volumen máximo de modelos, debido a que las herramientas estudiadas no proporcionan escenarios de evaluación homogéneos que permitan establecer una comparación justa, por lo que la comparación se ha centrado en las capacidades funcionales y de expresividad.

En consecuencia, los resultados obtenidos muestran que el trabajo desarrollado amplía las capacidades de los generadores existentes, especialmente en cuanto a la generación

de modelos con mayor expresividad y su integración dentro del ecosistema UVLHub, lo cual permite afirmar que efectivamente el trabajo realizado en este TFG no queda por detrás del resto a pesar de que herramientas como BeTTY cuenten con una mayor madurez histórica y una validación más extensa en escenarios experimentales.

8.4 Discusión

[Interpretación global, sin repetir los números anteriores: (1) ¿qué significan los resultados en conjunto? ¿responden al problema de la Sección 1.2? (2) ¿qué resultados fueron inesperados y a qué los atribuyes (hipótesis razonadas, no excusas)? (3) ¿qué compromisos (trade-offs) revelan: rendimiento frente a simplicidad, coste frente a precisión, generalidad frente a especialización...?]

[Cierra con un párrafo breve de amenazas a la validez: qué podría debilitar estas conclusiones, distinguiendo validez interna (factores del propio experimento: datos sintéticos, selección de casos, configuración), externa (hasta qué punto generalizan los resultados a otros entornos, datos o usuarios) y de constructo (¿miden las métricas elegidas realmente lo que se quiere evaluar?). Basta un párrafo: el tratamiento completo de las limitaciones se hace en la Sección 9.2, no lo dupliques aquí.]

Los resultados obtenidos muestran que la solución desarrollada responde al problema planteado inicialmente: la necesidad de disponer de un generador capaz de producir modelos de características con una mayor expresividad que las aproximaciones tradicionales basadas principalmente en construcciones booleanas. La integración del generador dentro del ecosistema Flamapy y UVLHub permite disponer de una herramienta flexible para la generación de modelos UVL destinados a tareas posteriores de análisis y experimentación.

Durante el desarrollo se encontraron diferentes dificultades relacionadas con la integración y adaptación de los componentes existentes. Sin embargo, la evaluación final confirmó que el generador soporta correctamente las funcionalidades planteadas inicialmente, incluyendo los diferentes niveles de expresividad y las distintas configuraciones disponibles.

La incorporación de construcciones avanzadas implica ciertos compromisos. Por un lado, aumentar la expresividad mediante atributos tipados, restricciones aritméticas, funciones agregadas o restricciones sobre cadenas incrementa la complejidad interna del generador. Por otro lado, ofrecer una configuración más flexible permite adaptar la generación a distintos escenarios, aunque aumenta el número de parámetros que el usuario debe comprender. Asimismo, una mayor riqueza en los modelos generados puede suponer un incremento del coste computacional en configuraciones más complejas.

Finalmente, existen algunas amenazas a la validez de los resultados. La evaluación se ha realizado utilizando modelos sintéticos y configuraciones diseñadas para cubrir las funcionalidades principales del generador, por lo que los resultados podrían variar en otros escenarios. Además, los tiempos obtenidos dependen del entorno de ejecución y no se ha realizado una comparación cuantitativa directa con otras herramientas bajo condiciones equivalentes.

[Cierra el capítulo con un párrafo puente de 2–4 líneas: qué queda establecido aquí y qué pregunta abre el capítulo siguiente. Por ejemplo: «Evaluados los resultados frente a los criterios de éxito y discutido su alcance, el Capítulo 9 sintetiza el grado de cumplimiento de los objetivos planteados en el Capítulo 1, las limitaciones del trabajo y las líneas de trabajo futuro que estos resultados abren».]

Los resultados obtenidos y la discusión realizada permiten establecer el grado de cumplimiento de los objetivos planteados y validar la solución desarrollada frente a los criterios de éxito definidos. A partir de este análisis, el Capítulo 9 aborda las conclusiones finales del trabajo, las limitaciones identificadas y las posibles líneas de trabajo futuro derivadas de los resultados obtenidos.

9 CONCLUSIONES Y TRABAJO FUTURO

[El último capítulo cierra la memoria en tres pasos: las conclusiones (qué se ha conseguido, apoyado en la evidencia del Capítulo 8, que el lector acaba de leer), las limitaciones (qué no se ha conseguido o no se ha podido validar) y el trabajo futuro (qué líneas abre todo lo anterior).]

9.1 Conclusiones

[Estructura recomendada:]

*[1. **Recapitulación del problema:** una o dos frases recordando qué problema se planteaba al inicio (sin entrar en detalles, ya que el lector viene del Capítulo 8).]*

*[2. **Cumplimiento de los objetivos:** revisa los objetivos específicos planteados en el Capítulo 1 y argumenta cómo se han cubierto, **apoyando cada afirmación en evidencia concreta:** la tabla, figura o sección del Capítulo 8 que lo demuestra (o, si procede, de los Capítulos 4 a 7). Evita el «se ha cumplido» sin soporte: todo cumplimiento debe poder verificarse en un capítulo anterior. Dedicar un párrafo a cada objetivo, en el mismo orden en que se enunciaron.]*

- *[El **primer objetivo** ... se ha cumplido mediante ..., como muestra la Tabla/Figura ... del Capítulo 8.]*
- *[El **segundo objetivo** ... se ha cumplido mediante ..., como muestra ...]*
- *[El **tercer objetivo** ... se ha cumplido mediante ..., como muestra ...]*
- *[El **cuarto objetivo** ... se ha cumplido mediante ..., como muestra ...]*
- *[El **quinto objetivo** ... se ha cumplido parcialmente mediante ..., como se discute en la Sección 9.2.]*

[La siguiente tabla sintetiza el cumplimiento y cierra la cadena abierta por la Tabla 1.1 del Capítulo 1: mismos objetivos, mismos criterios de éxito, ahora con su grado de cumplimiento y la evidencia que lo acredita. La columna «Grado» usa la escala Completo / Parcial / No alcanzado: «Completo» significa que el criterio se cumple íntegramente con la evidencia citada; «Parcial», que se cumple solo en parte, explicando qué falta en la Sección 9.2; «No alcanzado», que el criterio no se ha cumplido, analizando la causa en la Sección 9.2. Admitir un «Parcial» con remisión honesta a la Sección 9.2 es señal de madurez, no de debilidad.]

Obj.	Criterio de éxito	Grado	Evidencia (ref.)
O1	[Criterio de éxito del objetivo 1.]	[Completo]	[Tabla/Figura/Sección que lo acredita.]
O2	[Criterio de éxito del objetivo 2.]	[Completo]	[Evidencia.]
O3	[Criterio de éxito del objetivo 3.]	[Completo]	[Evidencia.]
O4	[Criterio de éxito del objetivo 4.]	[Completo]	[Evidencia.]
O5	[Criterio de éxito del objetivo 5.]	[Parcial]	[Evidencia y remisión a la Sección 9.2.]

Tabla 9.1: Grado de cumplimiento de los objetivos y evidencia asociada.

[3. **Aportaciones más amplias:** comenta brevemente la contribución del trabajo más allá del producto en sí. Por ejemplo, aplicación de un paradigma a un dominio nuevo, integración con un ecosistema, validación empírica de una técnica, etc.]

[4. **Reflexión crítica y lecciones aprendidas:** resume en un párrafo la desviación global de horas y en qué fases se concentró, remitiendo al cierre del Capítulo 3 (Sección 3.10) para el detalle, sin duplicarlo. Añade aquí una o dos lecciones nuevas, técnicas o personales: qué decisión repetirías, qué harías de otra forma si empezaras hoy, qué sabes ahora que no sabías al comenzar. Dos o tres frases honestas valen más que un párrafo genérico.]

[5. **Cierre:** una frase que cierre el capítulo conectando las limitaciones identificadas en la introducción con la solución desarrollada (“el sistema desarrollado resuelve los X problemas estructurales...”).]

El objetivo principal de este trabajo era desarrollar un generador automático de modelos de características capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, integrándose dentro del ecosistema Flamapy y UVLHub. Para ello, se ha desarrollado una solución configurable que permite generar modelos con construcciones avanzadas del lenguaje UVL y validar su correcto funcionamiento mediante pruebas automatizadas.

Los resultados obtenidos en el Capítulo 8 permiten concluir que los objetivos planteados inicialmente han sido alcanzados. El objetivo O1, relacionado con la generación automática de modelos UVL con distintos niveles de expresividad, se ha cumplido mediante la generación y validación de modelos con diferentes configuraciones, como se muestra en la Sección 8.2.1.

El objetivo O2 se ha cumplido mediante la incorporación de un sistema de configuración flexible que permite adaptar el proceso de generación a distintos escenarios. Por

su parte, el objetivo O3 se ha alcanzado mediante la integración del generador como plugin de Flamapy y su utilización desde UVLHub, permitiendo su incorporación dentro del flujo habitual de la plataforma (Secciones 8.2.2 y 8.2.2).

El objetivo O4, centrado en la evaluación del rendimiento del generador, se ha verificado mediante la medición del coste de generación en escenarios con un volumen elevado de modelos, obteniendo resultados adecuados para el uso previsto de la herramienta (Sección 8.2.2). Finalmente, el objetivo O5 se ha cumplido mediante la ejecución de una batería de pruebas automatizadas que valida las principales funcionalidades del generador y sus diferentes flujos de configuración (Sección 8.2.2). La Tabla 9.2 resume el grado de cumplimiento de los objetivos definidos.

Obj.	Objetivo	Grado
O1	Generación automática de modelos UVL con expresividad avanzada.	Completo
O2	Configuración flexible del proceso de generación.	Completo
O3	Integración con Flamapy y UVLHub.	Completo
O4	Evaluación temporal del generador.	Completo
O5	Validación mediante pruebas automatizadas.	Completo

Tabla 9.2: Grado de cumplimiento de los objetivos. Elaboración propia

Como aportación principal, este trabajo proporciona una solución integrada para la generación automática de modelos UVL con mayor expresividad que los generadores tradicionales centrados en construcciones booleanas. Además, la integración dentro del ecosistema Flamapy y UVLHub permite utilizar esta funcionalidad en escenarios de experimentación y análisis de variabilidad.

A nivel personal y técnico, el desarrollo ha permitido identificar la importancia de realizar una planificación temporal con margen para imprevistos, así como la necesidad de utilizar desde el inicio herramientas que faciliten un entorno reproducible, como Docker. También se ha comprobado la relevancia de realizar refactorizaciones progresivas y acompañar los cambios funcionales con pruebas automatizadas para evitar la acumulación de errores durante la evolución del software.

En conjunto, los resultados obtenidos permiten concluir que la solución desarrollada cumple los objetivos establecidos y proporciona una base extensible para futuras mejoras relacionadas con la generación y análisis de modelos de características.

9.2 Limitaciones

[Reflexión honesta sobre lo que el trabajo no hace o no ha podido demostrar. Reconocer limitaciones no resta nota: demuestra que comprendes el alcance real de lo construido. Para cada limitación sigue el patrón causa (decisión, restricción de tiempo o de recursos) + efecto práctico + por qué es asumible o cómo se mitigaría. Organízalas en tres categorías:]

- **[Limitación técnica:]** *[qué no hace el sistema o dónde rinde peor: p. ej., rendimiento no evaluado bajo carga real, dependencia de un servicio externo, escalabilidad no contrastada... Causa, efecto y mitigación.]*
- **[Limitación de validación:]** *[qué no se ha podido medir o con qué amenazas a la validez: p. ej., datos sintéticos, sin usuarios finales, una sola configuración de prueba... Conecta con la discusión de la Sección 8.4 del Capítulo 8.]*
- **[Limitación de alcance:]** *[qué quedó fuera del trabajo y por qué, de forma coherente con lo declarado en la Sección 1.5.]*

[Cada limitación es candidata natural a una línea de la Sección 9.3: si una limitación no genera línea futura ni se justifica como asumible, queda coja.]

A pesar de los resultados obtenidos, existen ciertas limitaciones que deben tenerse en cuenta al interpretar el alcance del trabajo realizado. Estas limitaciones no afectan al cumplimiento de los objetivos planteados, pero delimitan los escenarios en los que se han validado las capacidades del generador.

Desde un punto de vista técnico, la evaluación del generador se ha realizado sobre configuraciones representativas, pero no se ha estudiado su comportamiento en escenarios extremos con modelos de características de gran tamaño o con una complejidad significativamente superior. Esto puede afectar al coste computacional en configuraciones con un número elevado de características, atributos o restricciones. No obstante, la arquitectura desarrollada permite continuar optimizando y ampliando el generador para este tipo de escenarios.

Respecto a la validación, las pruebas realizadas se han basado principalmente en modelos sintéticos generados mediante la propia herramienta y en escenarios diseñados para cubrir las funcionalidades implementadas. Por tanto, aunque la suite de pruebas automatizadas permite validar el comportamiento esperado del sistema, no se ha evaluado todavía su comportamiento con modelos reales procedentes de dominios específicos ni mediante estudios con usuarios finales. Además, no se ha realizado una comparación cuantitativa directa con otras herramientas debido a la ausencia de condiciones experimentales homogéneas que permitieran obtener resultados comparables.

Finalmente, en cuanto al alcance del trabajo, la solución desarrollada se ha centrado en la generación automática de modelos UVL y su integración dentro del ecosistema Flamapy y UVLHub. No se han abordado otras líneas relacionadas, como la optimización avanzada del proceso de generación, nuevas estrategias de búsqueda o análisis posterior de los modelos generados, ya que quedan fuera de los objetivos definidos inicialmente.

9.3 Trabajo futuro

[Líneas de extensión del trabajo, ordenadas de mayor a menor inmediatez. Cada línea debe responder a tres preguntas: qué se haría, por qué merece la pena (anclándola a una limitación de la Sección 9.2 o a una tendencia del Capítulo 2, idealmente con cita) y qué viabilidad o dificultad estimada tiene. Al menos una línea debe ser genuinamente innovadora (no «añadir más tests»), explicando qué la hace novedosa.]

[Línea 1 (corto plazo): título corto.] [Qué: ... Por qué: resuelve la limitación técnica identificada en la Sección 9.2. Viabilidad: encaja con la arquitectura actual porque ...; dificultad estimada baja.]

[Línea 2 (corto plazo): título corto.] [Qué / por qué (limitación de validación) / viabilidad-dificultad.]

[Línea 3 (medio plazo): título corto.] [Qué / por qué (funcionalidad excluida del alcance) / viabilidad-dificultad.]

[Línea 4 (innovadora): título corto.] [Qué: extensión ambiciosa (nueva técnica, dominio o integración). Por qué: conecta con la tendencia identificada en el estado del arte [8]. Qué la hace novedosa: ... Viabilidad: qué haría falta para abordarla y por qué este trabajo es un buen punto de partida.]

A partir de las limitaciones identificadas y de las posibilidades de extensión del sistema desarrollado, se plantean diferentes líneas de trabajo futuro orientadas a mejorar la reutilización de configuraciones, la integración con UVLHub y la capacidad del generador para adaptarse a nuevos escenarios de uso.

Exportación e importación de configuraciones

Una primera línea de trabajo consistiría en desarrollar una transformación de Flamapy que permita exportar la configuración seleccionada del generador en un formato estándar como JSON, permitiendo posteriormente importar dicha configuración desde UVLHub para reproducir una generación concreta.

Esta extensión mejoraría la reproducibilidad de los experimentos y facilitaría el intercambio de configuraciones entre usuarios, evitando la necesidad de definir manualmente conjuntos complejos de parámetros. Además, permitiría compartir

escenarios de generación de forma sencilla dentro del ecosistema de herramientas de variabilidad.

La implementación de esta funcionalidad presenta una viabilidad elevada, ya que los parámetros del generador ya se encuentran estructurados internamente. Sería necesario únicamente definir un esquema estable de serialización y añadir los mecanismos de exportación e importación correspondientes.

Publicación directa de modelos en UVLHub

Otra posible extensión consistiría en ampliar la integración con UVLHub para permitir subir directamente los modelos generados a la plataforma, pudiendo establecer su visibilidad y compartirlos con otros usuarios.

Actualmente, el generador permite producir modelos UVL, pero la gestión posterior de estos modelos requiere pasos adicionales. La incorporación de esta funcionalidad permitiría centralizar el flujo completo, desde la generación hasta el almacenamiento y distribución de modelos de características.

Esta línea presenta una viabilidad media-alta, ya que requeriría ampliar la comunicación entre el generador y los servicios existentes de UVLHub, además de definir aspectos relacionados con permisos, almacenamiento y gestión de modelos públicos.

Evaluación con modelos reales y benchmarking

Una tercera línea de trabajo consistiría en realizar una evaluación experimental del generador utilizando modelos de características procedentes de dominios reales y comparando sus resultados con otras herramientas existentes bajo unas condiciones experimentales comunes.

Esta extensión permitiría superar una de las limitaciones identificadas en la evaluación actual, basada principalmente en modelos sintéticos, y proporcionaría una medida más precisa de la utilidad del generador en escenarios reales.

Su viabilidad es media, ya que requeriría disponer de conjuntos de datos adecuados, establecer una metodología experimental común y adaptar las herramientas comparadas para garantizar unas condiciones equivalentes.

Optimización del proceso de generación

Una posible línea de trabajo futuro consistiría en optimizar el proceso interno de generación para mejorar el rendimiento en configuraciones con una mayor complejidad estructural o un volumen elevado de modelos generados.

Aunque los resultados obtenidos muestran un comportamiento adecuado en los escenarios evaluados, el coste computacional puede incrementarse al aumentar el número de características, atributos o restricciones. Por ello, sería interesante analizar

los puntos críticos del proceso de generación y aplicar técnicas de optimización, como mejoras en la gestión de estructuras internas, reducción de operaciones redundantes o paralelización de determinadas etapas.

Esta extensión presenta una viabilidad media-alta, ya que no requiere modificar la arquitectura general del sistema, sino realizar mejoras sobre los componentes existentes del motor de generación. La principal dificultad reside en identificar aquellas operaciones que tienen mayor impacto sobre el tiempo de ejecución y mantener la corrección de los modelos generados tras las optimizaciones.

Generación guiada mediante aprendizaje automático

Como línea de investigación más ambiciosa, se plantea la incorporación de técnicas de aprendizaje automático para aprender patrones presentes en modelos de características reales y utilizarlos durante el proceso de generación.

Esta aproximación permitiría evolucionar desde una generación basada únicamente en parámetros definidos manualmente hacia una generación capaz de producir modelos sintéticos adaptados a las características de dominios concretos. De esta forma, se podrían obtener modelos más realistas para tareas de evaluación y experimentación. La viabilidad de esta línea es más reducida, ya que requeriría disponer de un volumen suficiente de modelos reales, definir representaciones adecuadas de los modelos de características y desarrollar técnicas capaces de capturar sus patrones estructurales. No obstante, constituye una posible evolución innovadora del generador desarrollado.

BIBLIOGRAFÍA

- [1] J. A. Zamudio. *Líneas de Producto Software para acelerar el desarrollo de aplicaciones relacionadas*. 2023. URL: <https://ingenieriadesoftware.es/lineas-producto-software/> (visitado 05-08-2026).
- [2] A. Felfernig, A. Falkner y D. Benavides. *Feature Models. AI-Driven Design, Analysis and Applications*. 1.^a ed. Springer, 2024. ISBN: 978-3-031-61874-1.
- [3] D. Benavides, C. Sundermann, K. Feichtinger, J. A. Galindo, R. Rabiser y T. Thüm. «UVL: Feature modelling with the Universal Variability Language». En: *ELSEVIER* 225 (2025). DOI: 10.1016/j.jss.2024.112326.
- [4] U. de Sevilla, U. de Málaga, U. of Graz y UNED. *Flamapy: The cutting-edge Python-based tool for Automated Analysis of Feature Models using UVL and more*. URL: <https://www.flamapy.org/> (visitado 05-08-2026).
- [5] *TASOVA Plus: Red en nuevas Tendencias en Arquitectura Software y Variabilidad*. 2023. URL: <https://tasovaplus.github.io/> (visitado 05-08-2026).
- [6] *UVLHub, a repository of feature models in UVL (Universal Variability Language) format, integrated with Zenodo and flamapy, and developed following Open Science*. 2023. URL: <https://docs.uvlhub.io/> (visitado 05-08-2026).
- [7] C. Sundermann, T. Heß, R. Sundermann, E. Kuitert, S. Krieter y T. Thüm. «Generating Feature Models with UVL’s Full Expressiveness». En: *Proceedings of the 28th ACM International Systems and Software Product Line Conference. SPLC ’24*. New York, NY, USA: Association for Computing Machinery, 2024, págs. 61-65. DOI: 10.1145/3646548.3676602.
- [8] N. Apellido y N. Apellido. «Título de ejemplo de un artículo de revista». En: *Nombre de la revista* 12.3 (2024), págs. 45-67. DOI: 10.1000/ejemplo.2024.
- [9] N. Apellido. *Título de ejemplo de un libro*. 2.^a ed. Editorial, 2021. ISBN: 978-0-00-000000-0.
- [10] P. Clements y L. Northrop. *Software Product Lines: Practices and Patterns*. 2002. ISBN: 9780201703320.
- [11] I. de Software. *Líneas de Producto Software*. 2025. URL: <https://ingenieria desoftware.es/lineas-producto-software/> (visitado 15-08-2026).
- [12] K. Pohl, G. Böckle y F. J. van der Linden. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer, 2005.

- [13] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak y A. S. Peterson. «Feature-Oriented Domain Analysis (FODA) Feasibility Study». En: Software Engineering Institute, Carnegie Mellon University, 1990.
- [14] D. Benavides, S. Segura y A. Ruiz-Cortés. «Automated Analysis of Feature Models 20 Years Later: A Literature Review». En: *Information Systems* 35.6 (2010), págs. 615-636.
- [15] D. Benavides, C. Sundermann, K. Feichtinger, J. A. Galindo, R. Rabiser y T. Thüm. «UVL: Feature modelling with the Universal Variability Language». En: *The Journal of Systems and Software* 225 (2025), pág. 112326. DOI: 10.1016/j.jss.2025.112326.
- [16] R. E. Bryant. «Graph-Based Algorithms for Boolean Function Manipulation». En: *IEEE Transactions on Computers* C-35.8 (1986), págs. 677-691. DOI: 10.1109/TC.1986.1676819.
- [17] U. de Sevilla. *Proyecto - 24/25: Documentación general de UVLHub*. Wiki de la asignatura Evolución y Gestión de la Configuración. URL: https://1984.1si.us.es/wiki-egc/index.php/Proyecto_-_24/25#Documentaci.C3.B3n_general.
- [18] M. Mendonça, M. Branco y D. Cowan. «S.P.L.O.T. - Software Product Lines Online Tools». En: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. 2009, págs. 761-762.
- [19] T. Thüm, C. Kästner, F. Benduhn, J. Meinicke, G. Saake y T. Leich. «FeatureIDE: An Extensible Framework for Feature-Oriented Software Development». En: *Science of Computer Programming* 79 (2014), págs. 70-85.
- [20] S. Segura, J. Galindo, D. Benavides, J. A. Parejo y A. Ruiz-Cortés. «BeTTY: Benchmarking and Testing on the Automated Analysis of Feature Models». En: *Proceedings of the 6th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS)*. 2012, págs. 63-70.
- [21] Wikipedia. *Pseudoaleatoriedad*. <https://es.wikipedia.org/wiki/Pseudoaleatoriedad>. Accedido: 29-08-2026. 2026.
- [22] UVLHub. *Rosemary: UVLHub Generation Framework Documentation*. 2026. URL: <https://docs.uvlhub.io/rosemary> (visitado 05-09-2026).

A APÉNDICE

[Los apéndices recogen material de referencia que rompería la lectura del cuerpo principal pero que conviene tener disponible: especificaciones largas, ejemplos completos de artefactos generados, manuales de uso, datasets, representaciones gráficas o textuales del modelo del dominio, etc.]

[Sugerencias de contenido típico:]

[

- *[Especificación formal completa (gramática, esquema, modelo).]*
- *[Ejemplo extenso de un artefacto que produce el sistema.]*
- *[Manual de instalación / despliegue / uso.]*
- *[Tablas de referencia muy extensas.]*
- *[Figuras grandes que requieren orientación apaisada.]*

]

[Si el TFG necesita varios apéndices, duplica este archivo (`B_apendice.tex`, `C_apendice.tex...`) y añade los correspondientes `\include` en `main.tex` dentro del bloque `\appendix`.]

[Para incluir bloques de código o ficheros largos, usa el entorno `verbatim` (ya configurado con numeración de líneas):]

```
1 features
2     Smartwatch
3         mandatory
4             Display
5                 alternative
6                     LCD
7                     OLED
8                     "E-Ink"
9             Connectivity
10            or
11                Bluetooth
12                "Wi-Fi"
13                LTE
14        Sensors
15            mandatory
```

```

16         "Heart Rate Monitor"
17     optional
18         GPS
19         "Blood Oxygen Sensor" {Accuracy 95}
20         Accelerometer
21     Battery
22         alternative
23             Standard {Capacity 300}
24             "Long Life" {Capacity 450}
25 optional
26     "Music Player" {Storage 2}
27     NFC
28     "Voice Assistant"
29     "Water Resistance"
30         alternative
31             IP67
32             IP68
33             IPX8
34     "Watch Faces" cardinality [5..50]
35     Accessories
36         [0..2]
37         "Leather Strap"
38         "Metal Strap"
39         "Silicone Strap"
40     String Color
41     Float Weight
42
43 constraints
44     LTE => (GPS & Bluetooth)
45     IPX8 => (!LCD)
46     "Fall Detection" => (Accelerometer & Gyroscope)
47     "Voice Assistant" <=> "Wi-Fi"
48     sum(Capacity) >= 400
49     avg(Accuracy) >= 90
50     len("Serial Number") == 10
51     Weight > 30.0
52     Color != "Pink"

```

Código A.1: Ejemplo de modelo de características en UVL. Usado en la Sección 2.2

B MANUAL DE INSTALACIÓN Y DESPLIEGUE

[Este apéndice describe los pasos para instalar y desplegar el sistema en un entorno limpio. Debe ser lo bastante detallado para que un tercero (p. ej. otro estudiante o un evaluador) pueda reproducir el despliegue sin contacto con el autor/a.]

B.1 Requisitos previos

[Lista del software, hardware y credenciales necesarios antes de empezar. Indica versiones concretas siempre que sea posible.]

[

- **Sistema operativo:** *[p. ej. Linux Ubuntu 24.04 LTS / macOS / Windows 11.]*
- **Lenguaje y runtime:** *[p. ej. Python 3.x / Node 20 / Java 17.]*
- **Gestor de paquetes:** *[pip / npm / maven...]*
- **Servicios externos:** *[base de datos, broker de mensajería, almacenamiento...]*
- **Otras herramientas:** *[Docker, Git, etc.]*

]

- **Sistema operativo:** el entorno utilizado durante el desarrollo y validación del sistema ha sido Ubuntu 24.04 LTS. También es posible utilizar otros sistemas compatibles con Docker.
- **Lenguaje y runtime:** no es necesario instalar manualmente Python ni sus dependencias en el sistema anfitrión, ya que el entorno de ejecución se encuentra definido dentro de los contenedores del proyecto.
- **Gestor de paquetes:** las dependencias necesarias para ejecutar la aplicación y el generador se instalan automáticamente durante la construcción de las imágenes Docker.
- **Servicios externos:** no es necesario instalar manualmente servicios adicionales. La configuración Docker del proyecto incluye los contenedores necesarios para los servicios utilizados por UVLHub, entre ellos la base de datos MariaDB, Redis, Elasticsearch, MailHog y Selenium.

- **Otras herramientas:** es necesario disponer de Git para obtener el código fuente y Docker junto con Docker Compose para construir y ejecutar el entorno de desarrollo.

B.2 Obtención del código fuente

[Indica el repositorio donde está alojado el código y cómo clonarlo:]
En el siguiente enlace se puede encontrar el repositorio del proyecto:
<https://github.com/antjimort/uvlhub>

```
1 git clone git@github.com:antjimort/uvlhub.git
2 cd uvlhub
```

B.3 Instalación de dependencias y ejecución

[Comandos para instalar las dependencias del proyecto en un entorno aislado (virtualenv, contenedor, etc.). Si hay variables de entorno o ficheros de configuración que el usuario debe rellenar, lístalos aquí.]

Las dependencias del proyecto se gestionan mediante Docker, por lo que no es necesaria una instalación manual de paquetes. La construcción de los contenedores instala automáticamente las dependencias definidas en la configuración del proyecto. Este comando permite construir las imágenes utilizando el archivo de configuración correspondiente que contiene todos los subcomandos internos que lo prepara todo. Posteriormente, levanta los contenedores.

```
1 docker compose -f docker/docker-compose.dev.yml up --build
```

Para detener y eliminar los contenedores se usa el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml down
```

B.4 Despliegue en producción

[Si el sistema se despliega en un entorno productivo (servidor propio, nube, PaaS...), describe el proceso paso a paso: construcción de la imagen, publicación en el registro, despliegue en el host, configuración del proxy/HTTPS, etc.]

La herramienta utilizada para el despliegue de la app principal ha sido Render, pero ha sido usada únicamente para las revisiones del tutor. También se ha usado Redis, utilizado como almacenamiento externo para la gestión de las sesiones de Flask para conservar el estado del proceso de configuración del generador.

No obstante, la plataforma utilizada no constituye un requisito del sistema, ya que la aplicación puede ser desplegada mediante otras alternativas compatibles con contenedores.

El despliegue oficial de UVLHub es gestionado por DiversoLab.

B.5 Verificación de la instalación

[Cómo comprobar que la instalación ha sido correcta: ejecución de la suite de pruebas, acceso a una pantalla concreta, llamada a un endpoint de health check, etc.]

B.6 Verificación de la instalación

Una vez levantados los contenedores mediante el comando indicado en la Sección B.3, se puede comprobar que la instalación se ha realizado correctamente accediendo a la dirección local proporcionada por la aplicación durante el arranque.

La consola muestra la dirección en la que se encuentra disponible el servicio web, normalmente mediante la dirección `http://172.18.0.10:5000` (la segunda que aparece). Se puede acceder a ella mediante cualquier navegador. Un ejemplo de la salida generada durante el inicio de la aplicación se muestra en la Figura B.1.

```
web_app_container | [DEBUG] Indexing document with ID 'hubfile-12' in 'search_index'
web_app_container | [SUCCESS] Document 'hubfile-12' indexed successfully.
web_app_container | [REINDEX] Reindexing completed.
web_app_container | [✓] Reindexing completed successfully!
mariadb_container | 2026-09-07 11:15:13 9 [Warning] Aborted connection 9 to db: 'uVL
web_app_container | [🔍] Looking for hubfiles without FactLabels...
web_app_container | [✓] All hubfiles already have FactLabels!
mariadb_container | 2026-09-07 11:15:14 10 [Warning] Aborted connection 10 to db: 'u
web_app_container | * Serving Flask app 'app'
web_app_container | * Debug mode: on
web_app_container | [WARNING: This is a development server. Do not use it in a product
web_app_container | * Running on all addresses (0.0.0.0)
web_app_container | * Running on http://127.0.0.1:5000
web_app_container | * Running on http://172.18.0.10:5000
web_app_container | Press CTRL+C to quit
web_app_container | * Restarting with stat
web_app_container | * Debugger is active!
web_app_container | * Debugger PIN: 114-430-192
w Enable Watch d Detach
```

Figura B.1: Salida de consola tras el arranque correcto del contenedor web.
Elaboración propia

C MANUAL DE USUARIO

[Este apéndice está dirigido al usuario final del sistema. A diferencia del manual de instalación, no asume conocimientos técnicos y se centra en cómo usar el sistema una vez en marcha. Incluye capturas de pantalla de las acciones principales cuando completes la memoria final.]

C.1 Acceso al sistema

[Indica la URL (o forma de acceso) del sistema y los pasos para registrarse e iniciar sesión.]

[

- *[URL de acceso: `https://example.org`]*
- *[Registro de cuenta nueva.]*
- *[Inicio de sesión.]*
- *[Recuperación de contraseña (si aplica).]*

]

[(Espacio reservado para una captura de la pantalla de inicio.)]

El sistema puede utilizarse mediante dos alternativas. En primer lugar, es posible ejecutar la aplicación en el entorno local siguiendo los pasos descritos en la Sección B.6 del manual de instalación y despliegue.

Como alternativa, se puede acceder al despliegue oficial disponible públicamente mediante la siguiente dirección:

- `https://www.uvlhub.io/`

La generación de modelos no requiere que el usuario disponga de una cuenta ni que realice un proceso de autenticación previo.

La pantalla inicial de UVLHub se muestra en la Figura C.1.

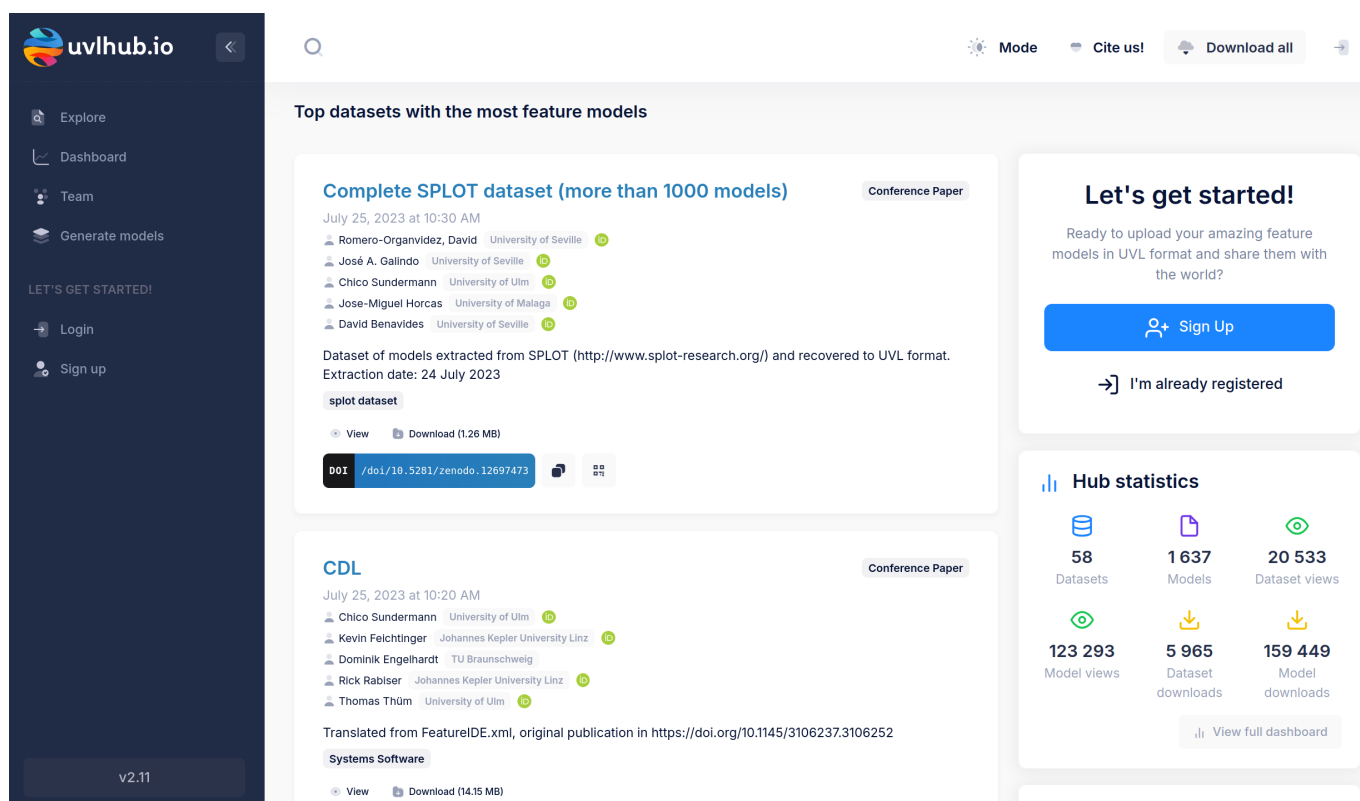


Figura C.1: Pantalla inicial de UVLHub. Elaboración propia

C.2 Funcionalidad principal

[Describe paso a paso cómo realizar la tarea principal del sistema. Conviene incluir capturas o anotaciones que destaquen los botones y campos relevantes.]

[

1. *[Paso 1: dónde hacer clic, qué introducir.]*
2. *[Paso 2: qué pantalla aparece y qué hacer en ella.]*
3. *[Paso 3: cómo confirmar la acción.]*
4. *[Paso 4: cómo verificar el resultado.]*

]

La funcionalidad principal del sistema consiste en la generación automática de modelos de características mediante un asistente de configuración (*wizard*). La estructura completa del flujo de generación y las pantallas correspondientes a cada etapa pueden consultarse en la Sección 5.4.1.

El funcionamiento del asistente consiste en configurar los diferentes parámetros de generación según las necesidades del usuario. En cada etapa del flujo se pueden modificar las opciones disponibles y avanzar al siguiente paso mediante el botón

Next, situado en la parte inferior de la pantalla. En caso de querer modificar una configuración anterior, es posible volver a los pasos previos utilizando el botón *Previous*.

Una vez completada la configuración, en el sexto y último paso del asistente se generan los modelos de características solicitados. Desde esta pantalla el usuario dispone de diferentes opciones sobre los modelos generados:

- Descargar el conjunto de modelos generado mediante el botón *Download*.
- Visualizar un modelo directamente desde la aplicación mediante el botón con el icono del ojo.
- Descargar individualmente cada modelo mediante el botón verde disponible junto a cada resultado.
- Abrir un modelo mediante el botón morado con el icono del cohete en FlamapyIDE, un entorno externo que permite visualizar, editar y realizar tareas de análisis automático sobre modelos de características (<https://github.com/flamapy/flamapy-ide>).

C.3 Mensajes de error y resolución de incidencias

[Lista de errores frecuentes que puede encontrar el usuario, con su significado y la acción recomendada para resolverlos.]

[

- *[Mensaje 1]: [causa habitual y solución.]*
- *[Mensaje 2]: [causa habitual y solución.]*
- *[Mensaje 3]: [causa habitual y solución.]*

]

Durante el proceso de configuración y generación, el sistema muestra mensajes de error descriptivos cuando no se cumplen las reglas de negocio definidas en la Sección 4.5, indicando al usuario la condición que debe modificarse para poder continuar.

Existe un caso particular durante la generación de modelos en el que puede aparecer un error no asociado a una configuración incorrecta, sino a una exigencia excesiva sobre los recursos disponibles. Este problema puede ocurrir, por ejemplo, al solicitar un número elevado de modelos con un gran número de características, restricciones, variables por restricción o atributos.

Para solucionar esta situación se recomienda revisar los parámetros de generación configurados, especialmente:

- Número de modelos a generar.
- Número de características.
- Número de restricciones.
- Número de variables por restricción.
- Número de atributos.

Un ejemplo del mensaje mostrado en esta situación se presenta en la Figura C.2.

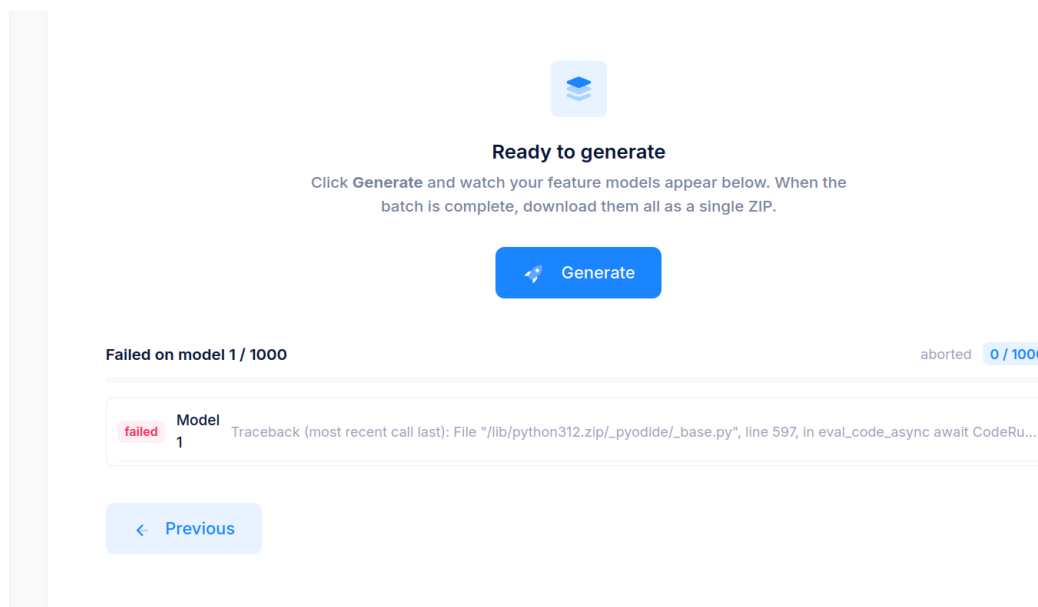


Figura C.2: Ejemplo de error durante la generación de modelos. Elaboración propia

C.4 Contacto y soporte

[Vía de contacto para incidencias o soporte (correo, formulario, repositorio de issues), si la hubiese.]

Para cualquier duda relacionada con el funcionamiento del sistema o para reportar errores no contemplados en este manual, se puede contactar con el desarrollador mediante las siguientes direcciones de correo electrónico:

- antjimort@alum.us.es
- ajimenez34@us.es